STAT 547 - Data Mining [1]
Homework 4

Rochester Institute of Technology
Ducheng Tan

May 8, 2026

1. Problem Formulation

(a) Domain description and personal motivation Our goal is to optimize urban transit efficiency in Tokyo, Urban mass transit represents one of the most data-rich and socially consequential domains in applied statistics. The Tokyo Metropolitan subway network, operated jointly by Tokyo Metro and Toei Subway, processes in excess of eight million passenger trips daily across 285 stations, making it one of the highest-volume transit systems on Earth. Accurate forecasting helps in energy management, staffing, congestion control and trains are just cool to learn about in general. We believe that using Machine Learning techniques are well suited for this task as a survey approach from a large population is expensive, so this is a chance for us to rely on statistical/ML techniques to truly uncover what is hiding beneath.
(b) Specific predictive/inferential question We believe that this domain description is a very complexed highly multivariate factor dependent goal, hence the question can be narrowed down to somewhere along the lines of "What is the expected number of passengers entering a specific station during a given hour t and time of the year, given temporal, weather, and historical features?" If we want to be technical then, The core predictive question is: *given a feature vector* $\mathbf{x}_t$ *encoding the temporal context, station identity, network topology, meteorological conditions, and lagged ridership history at time* t*, what is the expected number of passengers entering station* s *during the interval* $[t, t + \Delta t)$? Formally, we seek to learn the conditional expectation

$$f^*(x) = \mathbb{E}[y_t \mid \mathbf{x}_t = x] = \int_{\mathcal{Y}} y\, p(y \mid x)\, dy, \tag{1}$$

where $p(y \mid x)$ is the unknown conditional distribution of ridership given the feature vector. By the Bayes optimal regressor theorem under squared error loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, the function f^* minimizes the population risk $R(f) = \mathbb{E}_{(x,y)\sim p(x,y)}[\ell(y, f(x))]$ over all measurable functions $f : \mathcal{X} \to \mathcal{Y}$. The secondary inferential question concerns the decomposition of variance between temporal, spatial, and exogenous sources, which informs operational decisions about which factors most strongly drive demand variance in the metro system.
(c) Target variable and feature definition Target ($y \in \mathcal{Y}$): A count of hourly ridership ($y \in \mathbb{Z}^+$), Features ($x \in \mathcal{X} \subseteq \mathbb{R}^{\mathbb{P}}$): A feature vector including station ID (categorical), hour of day (0-23), day of week, holiday indicators, and local weather data (temperature, precipitation), local news data transformed under some numerical scaling, and any relevant internal data from Tokyo Metro.

Figure 1: Tokyo Metro Subway Map in *English*

2. Data Strategy

(a) Data sources and collection plan From Tokyo Open Data https://dateno.io/registry/catalog/cdi00004912/ we can collect historical entry and exit logs of passengers to understand the volume of the ridership.This will be our data

$$\mathscr{D}_n = \{(x_i, y_i)\}_{i=1}^n \tag{2}$$

which may not need to be *iid* under time series methods and modeling.

(b) Feature engineering pipeline

The raw dataset $\mathscr{D}_n$ is drawn from an unknown joint distribution $p(\mathbf{x}, y)$ over the product space $\mathcal{X} \times \mathcal{Y}$, where $\mathcal{X} \subseteq \mathbb{R}^p$ is the feature space and $\mathcal{Y} \subseteq \mathbb{R}$ is the ridership response. The central task of feature engineering is the construction of a transformation $\phi : \mathcal{X}_{\text{raw}} \to \mathcal{X}$ that converts raw observational variables into a structured representation encoding all statistically relevant dependencies with the target y_t. We organize this pipeline along three axes: temporal encoding, spatial encoding, and exogenous driver inclusion.

Temporal Encoding. Ridership demand is a fundamentally periodic phenomenon exhibiting strong diurnal and weekly rhythms. A naive integer representation of the hour of day $h \in \{0, \ldots, 23\}$ imposes a false ordinal distance, placing midnight ($h = 0$) at maximum Euclidean distance from 11 PM ($h = 23$) despite their temporal adjacency. To preserve the topological structure of the circle $\mathbb{S}^1$ within the Euclidean feature space, we apply the cyclical encoding

$$\text{hour}_{\sin} = \sin\left(\frac{2\pi h}{24}\right), \qquad \text{hour}_{\cos} = \cos\left(\frac{2\pi h}{24}\right). \tag{3}$$

This embeds h as a point on the unit circle in $\mathbb{R}^2$. For any two hours h_1, h_2, the squared Euclidean distance between their encodings satisfies

$$d^2(h_1, h_2) = 2\left[1 - \cos\left(\frac{2\pi(h_1 - h_2)}{24}\right)\right], \tag{4}$$

depending only on the circular difference $|h_1 - h_2|$ mod 24 and achieving its maximum at a half-period lag of 12 hours. An identical construction $(\sin(2\pi d/7), \cos(2\pi d/7))$ is applied to the day of week $d \in \{0, \ldots, 6\}$, and $(\sin(2\pi m/12), \cos(2\pi m/12))$ to the month $m \in \{1, \ldots, 12\}$. A binary holiday indicator $\mathbf{1}_{\{t \in \mathcal{H}\}}$, where $\mathcal{H}$ denotes the set of statutory holiday time-points, captures systematic departures from the typical weekday conditional mean that are not linearly recoverable from the day-of-week encoding, since public holidays fall on arbitrary weekdays.

Spatial Encoding. The transit network comprises a finite set of stations $\mathcal{S} = \{s_1, \ldots, s_K\}$. Because station identity carries no intrinsic ordinal meaning, we represent it via the one-hot vector $\mathbf{e}_s \in \{0, 1\}^K$, with $(\mathbf{e}_s)_k = \mathbf{1}_{\{s = s_k\}}$. This is equivalent to including station-level fixed effects in a linear specification. Letting $\mathbf{A} \in \{0, 1\}^{K \times K}$ be the adjacency matrix of the transit graph, the degree $\deg(s_k) = \sum_{j=1}^K A_{jk}$ serves as a proxy for connectivity capacity, and the binary transfer-station indicator $\mathbf{1}_{\{\deg(s) \geq 2\}}$ distinguishes interchange nodes, which aggregate demand from multiple origin corridors and exhibit qualitatively different ridership dynamics from line-interior stations.

Exogenous Drivers. Temperature $T_t \in \mathbb{R}$ and precipitation $P_t \in \mathbb{R}_{\geq 0}$ are included as continuous covariates, capturing the meteorological modulation of transit demand that ensemble methods can model nonlinearly. The stochastic process $\{y_t\}$ also exhibits strong autocorrelation, quantified by the autocovariance function $\gamma(k) = \text{Cov}(y_t, y_{t-k})$. We include lagged ridership values

$$y_{t-1}, \qquad y_{t-24}, \qquad y_{t-168}, \tag{5}$$

encoding one-step momentum, same-hour-yesterday, and same-hour same-day-last-week effects respectively. These lags augment the feature vector as $\mathbf{x}_t \leftarrow \mathbf{x}_t \oplus (y_{t-1}, y_{t-24}, y_{t-168})^\top$, converting the regression into a feature-augmented autoregressive form compatible with all three modeling paradigms in Section 3.

Feature Standardization. Let $\hat{\mu}_j$ and $\hat{\sigma}_j$ denote the sample mean and standard deviation of the j-th feature computed on the training partition. The standardized matrix is

$$\widetilde{\mathbf{X}} = \frac{\mathbf{X} - \mathbf{1}_n \hat{\boldsymbol{\mu}}^\top}{\hat{\boldsymbol{\sigma}}^\top}, \tag{6}$$

so that $\widetilde{X}_{ij} = (X_{ij} - \hat{\mu}_j)/\hat{\sigma}_j$. The estimates $(\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}})$ are computed exclusively on $\mathscr{D}_{\text{train}}$ and applied unchanged to validation and test sets, preventing any form of data leakage.

(c) Data quality mitigation strategies

Urban sensor infrastructures produce a dataset $\mathscr{D}_n$ that departs from the idealized i.i.d. assumption underlying statistical learning theory. Letting $\mathbf{M} \in \{0,1\}^{n \times p}$ denote the binary missingness mask with $M_{ij} = 0$ when X_{ij} is unobserved, we address data quality through three complementary procedures.

Missing value imputation. Under the Missing At Random (MAR) assumption, a consistent estimator of the unobserved X_{ij} is the stratum-conditional mean

$$\hat{X}_{ij} = \frac{1}{|\mathcal{N}(i,j)|} \sum_{k \in \mathcal{N}(i,j)} X_{kj}, \tag{7}$$

where $\mathcal{N}(i,j) = \{k : M_{kj} = 1,\ \text{hour}(k) = h,\ \text{day-type}(k) = d,\ \text{station}(k) = s\}$ is the set of observed neighbors sharing the same seasonal stratum as observation i. This preserves the dominant periodicities of the series and avoids the downward bias that global mean imputation would introduce by pooling across heterogeneous temporal strata.

Outlier detection via robust z-scores. Classical outlier detection based on the sample mean and standard deviation suffers from the masking effect, whereby the very contaminating observations inflate the estimator, rendering their own detection impossible. We overcome this by using the Median Absolute Deviation $\text{MAD}_j = \text{median}_i(|X_{ij} - \widetilde{\mu}_j|)$, where $\widetilde{\mu}_j$ is the sample median. The scale estimate $\hat{\sigma}_j^* = 1.4826 \times \text{MAD}_j$, where the constant $1.4826 \approx [\Phi^{-1}(3/4)]^{-1}$ ensures consistency under Gaussian noise, yields the robust z-score

$$z_{ij}^* = \frac{X_{ij} - \widetilde{\mu}_j}{\hat{\sigma}_j^*}. \tag{8}$$

Observations with $|z_{ij}^*| > 3.5$ (Iglewicz–Hoaglin threshold) are flagged and treated as missing. This estimator has a breakdown point of 0.5, preserving validity even when up to 50% of stratum observations are contaminated, a robustness property that classical z-scores entirely lack.

Detrending and seasonal decomposition. To ensure approximate stationarity, we apply the additive decomposition

$$y_t = \tau_t + s_t + \varepsilon_t, \tag{9}$$

where τ_t is a long-run trend estimated by a centered moving average of period $L = 168$ hours, s_t is the periodic seasonal component extracted from the seasonally-adjusted residuals $y_t - \hat{\tau}_t$, and $\varepsilon_t = y_t - \hat{\tau}_t - \hat{s}_t$ is the stationary remainder on which all learning algorithms are trained. The Augmented Dickey–Fuller test at significance level $\alpha = 0.05$ is applied to $\{\hat{\varepsilon}_t\}$ to confirm removal of non-stationary components before model fitting proceeds.

3. Function Space & Learning

(a) Function space specification (3-4 algorithms)

We define a hypothesis class $\mathcal{H}$ consisting of three complementary modeling paradigms that together span a rich and diverse function space.

Model 1 — Time Series Regression (ARIMAX).[2] Ridership exhibits strong autocorrelation and seasonality, motivating an ARIMAX-style model that explicitly represents the temporal dynamics of the process. The model takes the form

$$y_t = \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \boldsymbol{\beta}^\top \mathbf{x}_t + \varepsilon_t, \tag{10}$$

where $\{\phi_i\}$ are autoregressive coefficients, $\{\theta_j\}$ are moving-average coefficients, and $\boldsymbol{\beta}$ captures the linear effect of exogenous features. This model is interpretable and captures temporal dynamics directly, but its linearity restricts the functional class $\mathcal{H}_1$ to affine mappings of the input.

Model 2 — Random Forest Regression.[1] To capture nonlinear feature interactions, we employ a bagged ensemble of B regression trees,

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x), \tag{11}$$

where each tree T_b is trained on a bootstrap resample $\mathscr{D}_n^{(b)}$ drawn with replacement, and at each split only a random subset of $\lfloor \sqrt{p} \rfloor$ features is considered. The variance reduction from bagging satisfies $\text{Var}(\hat{f}) = \rho\sigma_T^2 + (1-\rho)\sigma_T^2/B$, where ρ is the pairwise correlation between individual trees; feature subsampling decorrelates the trees ($\rho \downarrow$), amplifying the variance reduction. The function class $\mathcal{H}_2$ consists of averages of piecewise-constant functions, capable of approximating arbitrary continuous mappings over compact domains.

Model 3 — Gradient Boosted Trees.[1] Boosting constructs an additive model by sequentially minimizing residual error in function space. The final estimator is

$$f_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x), \tag{12}$$

where each h_m is a shallow tree fitted to the negative gradient of the loss at the current ensemble, and γ_m is a step-size (learning rate). Gradient boosted trees are known to achieve state-of-the-art performance on structured tabular regression tasks and represent the highest-capacity class in our specification, $\mathcal{H}_3 \supset \mathcal{H}_1$. Together the three model classes span the union $\mathcal{H} = \mathcal{H}_1 \cup \mathcal{H}_2 \cup \mathcal{H}_3$, covering temporal linearity, nonlinear interaction modeling, and high-capacity sequential ensemble learning.

(b) Learning principles and criteria

We adopt Empirical Risk Minimization (ERM) as the unifying learning principle. Since the true population risk $R(f) = \mathbb{E}_{(x,y)\sim p(x,y)}[\ell(y, f(x))]$ is inaccessible, we substitute the empirical risk

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(\mathbf{x}_i)), \tag{13}$$

and seek $\hat{f}_n = \arg\min_{f\in\mathcal{H}} \hat{R}_n(f)$. The primary loss is the squared error $\ell(y, \hat{y}) = (y - \hat{y})^2$, for which ERM corresponds to ordinary least squares within each hypothesis class. Model quality on held-out data is assessed via three complementary metrics: the Root Mean Squared Error RMSE $= \sqrt{\hat{R}_{\text{test}}}$ as the primary criterion since it shares units with the response; the Mean Absolute Error MAE $= n^{-1}\sum_i |y_i - \hat{y}_i|$ as a robust complement less sensitive to large deviations; and the coefficient of determination $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ as a normalized measure of explained variance. All evaluation is conducted on a chronologically ordered test set, respecting the temporal ordering of the data generating process and preventing any form of look-ahead bias.

(c) Optimization approaches

Each model class admits a distinct optimization strategy suited to its structural form. For the ARIMAX model, parameters $(\phi_1, \ldots, \phi_p, \theta_1, \ldots, \theta_q, \boldsymbol{\beta})$ are estimated by maximum likelihood under a Gaussian noise assumption, which under stationarity corresponds to minimizing the sum of squared one-step-ahead prediction errors. For the random forest, optimization proceeds via the CART algorithm applied independently to each bootstrap resample: at each internal node the split (j^*, τ^*) is chosen to minimize the weighted within-child sum of squares $\sum_{c\in\{\text{L,R}\}} \sum_{i\in c} (y_i - \bar{y}_c)^2$. For gradient boosting, optimization is performed via functional gradient descent in the space of additive models: at iteration m, the weak learner h_m solves

$$h_m = \arg\min_{h} \sum_{i=1}^{n} \left(r_{im} - h(\mathbf{x}_i)\right)^2, \tag{14}$$

where the pseudo-residuals $r_{im} = -[\partial\ell(y_i, f(x_i))/\partial f(x_i)]_{f=f_{m-1}}$ are the negative gradients of the loss with respect to the current prediction, providing the steepest descent direction in function space.

4. Regularization & Selection

(a) Regularization strategies

Regularization addresses the ill-posedness of the learning problem by augmenting the empirical risk with a complexity penalty, transforming the ERM objective into the penalized form

$$\hat{f}_{n,\lambda} = \arg\min_{f\in\mathcal{H}} \left\{ \hat{R}_n(f) + \lambda\Omega(f) \right\}, \tag{15}$$

where $\lambda \geq 0$ is a regularization coefficient and $\Omega : \mathcal{H} \to \mathbb{R}_{\geq 0}$ is a complexity measure. For the linear component of the ARIMAX model, we consider both L_2 regularization $\Omega(\boldsymbol{\beta}) = \|\boldsymbol{\beta}\|_2^2$ (ridge), which shrinks all coefficients toward zero while retaining them in the model, and L_1 regularization $\Omega(\boldsymbol{\beta}) = \|\boldsymbol{\beta}\|_1$ (LASSO), which induces sparsity by setting irrelevant lag coefficients exactly to zero, providing automatic feature selection among the autoregressive terms. The elastic net combines both: $\Omega(\boldsymbol{\beta}) = \alpha\|\boldsymbol{\beta}\|_1 + (1-\alpha)\|\boldsymbol{\beta}\|_2^2$, balancing the sparsity of LASSO with the grouping stability of ridge when predictors are highly correlated, as is typical of consecutive autoregressive lags. For ensemble methods, regularization is achieved implicitly through tree depth control (limiting the partition complexity of each weak learner), minimum leaf size (preventing overfitting to small subsamples), and the learning rate $\eta \in (0, 1)$ in boosting, which shrinks each functional update and requires more iterations to compensate, effectively smoothing the optimization trajectory.

(b) Model selection framework

The order selection problem for the ARIMAX model is addressed via information criteria. The Akaike Information Criterion AIC $= -2\hat{\ell} + 2k$ and the Bayesian Information Criterion BIC $= -2\hat{\ell} + k \log n$, where $\hat{\ell}$ is the maximized log-likelihood and k counts free parameters, trade off goodness of fit against model complexity. The BIC is preferred in large-n settings because its stronger $k \log n$ penalty more aggressively guards against overfitting and enjoys model selection consistency under standard regularity conditions. For the nonparametric ensemble models, hyperparameter selection (number of trees B, maximum depth $d_{\max}$, minimum leaf size $n_{\min}$, learning rate η, and number of boosting iterations M) is performed via a grid search over a pre-specified hyperparameter space, evaluated on a rolling validation window.

(c) Refinement procedures

Given the temporal structure of the data, standard k-fold cross-validation, which assumes exchangeable observations, is inappropriate: drawing validation folds from within the training period creates look-ahead bias. We instead employ *rolling-origin cross-validation*, in which a sequence of training windows $\mathscr{D}_1^{\text{tr}} \subset \mathscr{D}_2^{\text{tr}} \subset \cdots$ each of increasing size is paired with a subsequent validation window of fixed length h. The cross-validated estimate of prediction error is

$$\mathrm{CV}(\lambda) = \frac{1}{V} \sum_{v=1}^{V} \hat{R}_h\Big(\hat{f}_{n_v,\lambda}\Big), \tag{16}$$

where $\hat{f}_{n_v,\lambda}$ is the model trained on the v-th training window and $\hat{R}_h$ is the RMSE on the corresponding h-step-ahead validation window. The optimal regularization coefficient $\lambda^* = \arg\min_\lambda \mathrm{CV}(\lambda)$ is selected by this criterion. For gradient boosting, early stopping provides an additional refinement: training halts when the validation RMSE fails to decrease over a patience window of δ consecutive rounds, preventing over-iteration past the bias–variance optimum.

5. Inference & Deployment

(a) Uncertainty quantification methods

Point predictions alone are insufficient for operational decision-making; transit operators require calibrated prediction intervals to make robust service allocation decisions. We construct prediction intervals via the *residual bootstrap*. Let $\hat{\varepsilon}_i = y_i - \hat{f}(\mathbf{x}_i)$ denote the in-sample residuals from the fitted model. For a new test point x^*, we generate B^* bootstrap replicates $\hat{y}^{*(b)} = \hat{f}(x^*) + \varepsilon^{*(b)}$, where each $\varepsilon^{*(b)}$ is drawn uniformly from $\{\hat{\varepsilon}_i\}_{i=1}^n$, and construct the interval

$$\hat{f}(x^*) \pm z_{\alpha/2}\, \hat{\sigma}, \qquad \hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^{n} \hat{\varepsilon}_i^2, \tag{17}$$

where $z_{\alpha/2}$ is the appropriate standard normal quantile for a $(1-\alpha)$-level interval. For forest-based methods, a complementary approach uses the quantile of the empirical distribution of tree-level predictions $\{T_b(x^*)\}_{b=1}^B$ to construct non-parametric intervals that adapt to local variance heterogeneity without distributional assumptions on the residuals.

(b) Theoretical justification

The statistical validity of ERM as a learning principle rests on uniform concentration of the empirical risk around the population risk. A foundational result from statistical learning theory gives the uniform bound: for any bounded loss and hypothesis class $\mathcal{H}$, and for any $\varepsilon > 0$,

$$\mathbb{P}\left(\sup_{f \in \mathcal{H}} |\hat{R}_n(f) - R(f)| \geq \varepsilon \right) \leq 2|\mathcal{H}|\, e^{-2n\varepsilon^2}, \tag{18}$$

a consequence of Hoeffding's inequality and the union bound. This guarantees that with high probability the empirical minimizer $\hat{f}_n$ achieves near-optimal population risk, with the approximation improving at rate $O(n^{-1/2})$. For the ensemble methods, generalization is additionally supported by the bias–variance decomposition: $\mathbb{E}[(y - \hat{f}(x))^2] = \mathrm{Bias}^2(\hat{f}(x)) + \mathrm{Var}(\hat{f}(x)) + \sigma_\varepsilon^2$, where bagging reduces $\mathrm{Var}(\hat{f})$ through averaging and boosting reduces $\mathrm{Bias}^2(\hat{f})$ through sequential residual fitting, illustrating the complementary roles of the two ensemble strategies.

(c) Deployment and scalability considerations

Production deployment requires an inference pipeline that ingests real-time sensor feeds, constructs the feature vector $\mathbf{x}_t$ at the current timestamp, applies the pre-trained standardization parameters $(\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}})$, and returns a point forecast and

prediction interval through a REST API with latency compatible with operational decision cycles (target: sub-second inference at the station level). Scalability to all K stations is achieved by parallelizing inference across a distributed compute cluster, with each node handling a subset of stations. A critical operational concern is *concept drift*: the joint distribution $p(\mathbf{x}, y)$ may shift over time due to infrastructure changes, demographic shifts, or post-pandemic behavioral changes. We monitor drift via sequential testing of the RMSE on a rolling validation window against a control chart threshold; a statistically significant increase triggers an automated retraining pipeline that refits the model on the most recent w months of data, discarding observations too old to reflect the current distribution. The trust metric of the deployed system is formalized as $\text{Trust} = \mathbf{1}_{\{\text{Model accurate}\}} \cdot \mathbf{1}_{\{\text{Model interpretable}\}}$, with interpretability maintained through SHAP (SHapley Additive exPlanations) value attribution, which decomposes each prediction into additive feature contributions and facilitates communication with domain stakeholders.

6. Technical Specification

(a) Mathematical model formulations

Each model in $\mathcal{H}$ admits a precise mathematical specification. The ARIMAX model defines a parametric function class $\mathcal{H}_1 = \{f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \Theta\}$ where $\Theta = \mathbb{R}^p \times \mathbb{R}^q \times \mathbb{R}^d$ collects the autoregressive, moving-average, and exogenous coefficients. The model equation

$$f_{\boldsymbol{\theta}}(\mathbf{x}_t, y_{<t}) = \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \boldsymbol{\beta}^\top \mathbf{x}_t \tag{19}$$

is a linear functional of both the exogenous features and the past trajectory of the process. The random forest defines a stochastic function class in which each member is a bagged average of B axis-aligned piecewise-constant functions,

$$f_B(x) = \frac{1}{B} \sum_{b=1}^{B} \sum_{\ell=1}^{L_b} \bar{y}_{b\ell} \, \mathbf{1}_{\{x \in R_{b\ell}\}}, \tag{20}$$

where $\{R_{b\ell}\}$ are the terminal node rectangles of tree b and $\bar{y}_{b\ell}$ is the mean response in leaf ℓ. The gradient boosted model is an additive combination of M shallow trees, with the m-th component defined as the minimizer of the weighted squared residual problem stated in Section 3(c).

(b) Loss functions and optimization criteria

The primary loss is the squared error $\ell_2(y, \hat{y}) = (y - \hat{y})^2$, whose population minimizer is the conditional mean $\mathbb{E}[y \mid x]$ and whose ERM corresponds to least-squares estimation. Its sensitivity to large residuals is a feature in this application: underestimating demand at peak hours carries disproportionate operational cost, so penalizing large errors heavily is substantively appropriate. The empirical risk under squared loss is

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f(\mathbf{x}_i))^2, \tag{21}$$

and the out-of-sample generalization error is estimated on the chronological test partition as

$$\text{RMSE}_{\text{test}} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i \in \mathcal{D}_{\text{test}}} (y_i - \hat{f}(\mathbf{x}_i))^2}. \tag{22}$$

The $\text{MAE} = n_{\text{test}}^{-1} \sum_i |y_i - \hat{f}(\mathbf{x}_i)|$ complements RMSE as a robust measure less sensitive to the large prediction errors that occur during anomalous events such as concerts or severe weather.

(c) Inference procedures

Statistical inference for the ARIMAX model proceeds by maximum likelihood. Under the Gaussian noise assumption $\varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$, the log-likelihood is

$$\ell(\boldsymbol{\theta}, \sigma^2 \mid y_{1:n}) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=1}^{n} \left(y_t - f_{\boldsymbol{\theta}}(\mathbf{x}_t)\right)^2, \tag{23}$$

and the MLE $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ is obtained by numerical optimization (BFGS or L-BFGS-B). Asymptotic standard errors are derived from the observed Fisher information matrix $\mathcal{I}(\hat{\boldsymbol{\theta}}) = -\partial^2 \ell / \partial \boldsymbol{\theta}^2 \big|_{\hat{\boldsymbol{\theta}}}$, enabling Wald-type confidence intervals $\hat{\theta}_j \pm z_{\alpha/2} [\mathcal{I}(\hat{\boldsymbol{\theta}})^{-1}]_{jj}^{1/2}$ for each parameter. For the nonparametric ensemble methods, pointwise confidence bands on the predicted mean are constructed via the residual bootstrap described in Section 5(a), providing valid coverage under mild regularity conditions on the residual distribution. Feature importance is quantified through permutation importance and SHAP values, offering post-hoc inferential summaries of the learned function that are interpretable to domain stakeholders.

Remark

In the computational section two published articles were referenced: Fair prediction with disparate impact: A study of bias in recidivism prediction instruments from [3] and Double/debiased machine learning for treatment and structural parameters from [4]

References

[1] Bertrand Clarke, Ernest Fokoue, and Hao Helen Zhang. *Principles and theory for data mining and machine learning*. Springer Science & Business Media, 2009.

[2] Robert H Shumway and David S Stoffer. *Time series: a data analysis approach using R*. Chapman and Hall/CRC, 2019.

[3] Alexandra Chouldechova. Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big data*, 5(2):153–163, 2017.

[4] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters, 2018.

Exercise 2 & 3 - Financial Transcations and Proteomics Cancer

Ducheng Tan

2026-04-23

Abstract

This report applies comprehensive statistical machine learning methodology across two distinct problem domains. In Exercise 2, a financial transactions dataset ($n = 10{,}000$, $p = 50$) is analyzed for binary fraud detection under severe class imbalance, addressed via SMOTE and inverse-frequency weighting five classifiers Elastic Net logistic regression, Random Forest, XGBoost, SVM with RBF kernel, and a Multilayer Perceptron are trained on engineered features including polynomial and interaction terms, tuned via nested cross-validation and Bayesian hyperparameter optimization, and evaluated by AUC, F1, sensitivity, and specificity on a stratified holdout, with uncertainty quantified through three split-conformal prediction methods. In Exercise 3, a high-dimensional proteomics dataset ($p = 200$ genes, $n < p$) is analyzed for continuous ProteinX expression prediction, where rank deficiency and multicollinearity are first characterized before applying a suite of regularization and variable selection methods such asRidge, Lasso, Elastic Net, Adaptive Lasso, SCAD, Group Lasso, Horseshoe, and Spike-and-Slab alongside screening via Sure Independence Screening and stability selection. Reproducibility across methods is assessed via pairwise Jaccard similarity $J(A, B) = |A \cap B| \,/\, |A \cup B|$, revealing that `Gene1`, `Gene87`, and `Gene132` form a robust consensus signal identified by every method regardless of sparsity level.

Packages

```
library(ggcorrplot)
library(corrplot)
library(tidyverse)
library(ggplot2)
library(dplyr)
library(caret)
library(pROC)
library(patchwork)
library(smotefamily)
library(caret)
library(randomForest)
library(xgboost)
library(nnet)
library(rBayesianOptimization)
library(SIS)
library(BMS)
library(remotes)
library(parcor)
library(ncvreg)
library(grplasso)
library(Mhorseshoe)
library(mombf)
library(factoextra)
```

Exercise 2: Comprehensive Classification

Dataset: `financial_transactions.csv`

```
library(readxl)
financial_transactions <- read.csv("C:/Users/dt6302/Desktop/HW4/financial_transactions.csv")
#View(financial_transactions)
```

EDA and Feature Engineering

CLass Distribution and Correlation Analysis

Exploratory Data Analysis show that this is an imbalanced class problem with data matrix $X \in \mathbb{R}^{10000 \times 50}$ and $Y \in \mathbb{R}^{10000 \times 1}$ where $y_i \subset \{0, 1\}$ an element of the response vector Y.

```
y <- as.factor(financial_transactions$Fraud)
x <- financial_transactions[, !names(financial_transactions) %in% "Fraud"]

n <- nrow(financial_transactions)
p <- ncol(x)
table(y)
```

```
## y
##    0    1
## 7983 2017
```

```
n; p
```

```
## [1] 10000
```

```
## [1] 50
```

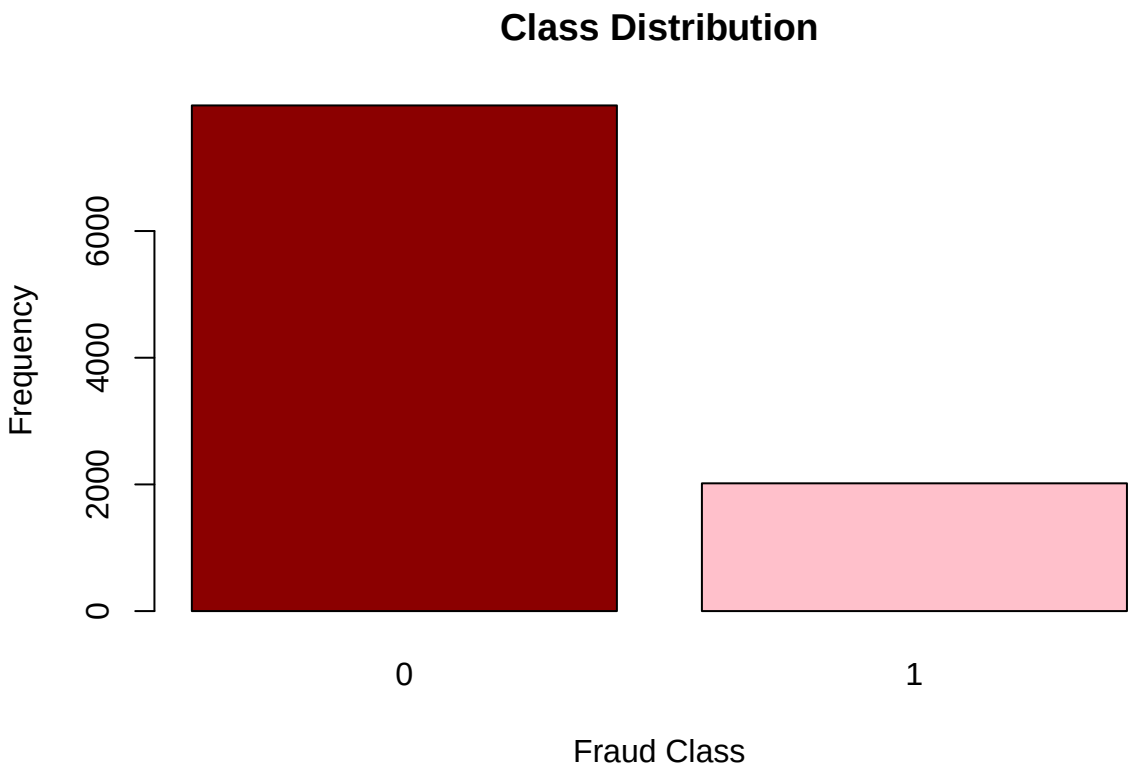
```
dim(x)
```

```
## [1] 10000    50
```

```
kappa <- n/p
cat("kappa =", kappa)
```

```
## kappa = 200
```

```
barplot(table(y),
        main = "Class Distribution",
        xlab = "Fraud Class",
        ylab = "Frequency",
        col = c("darkred", "pink"),
        border = "black")
```



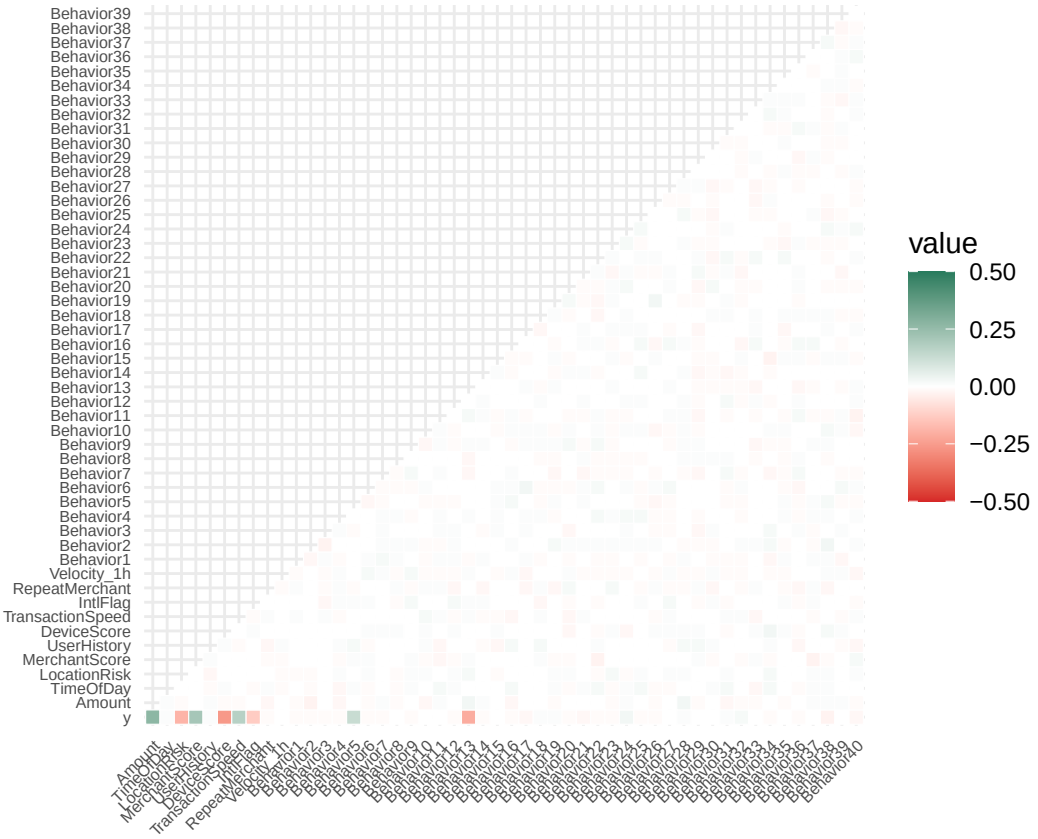
Correlation Matrix

```
y_numeric <- as.numeric(as.factor(financial_transactions$Fraud))-1


x_numeric <- financial_transactions[, !names(financial_transactions) %in% "Fraud"]
x_numeric <- sapply(x_numeric, as.numeric)

xy <- data.frame(y = y_numeric, x_numeric)

p <- ggcorrplot(cor(xy, use = "complete.obs"),
                type = "lower",
                outline.col = "white",
                colors = c("#D62828", "white", "#277B5D"))
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the ggcorrplot package.
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
p + scale_fill_gradient2(limit = c(-0.5, 0.5),
                         low = "#D62828",
                         mid = "white",
                         high = "#277B5D",
                         midpoint = 0,
                         oob = scales::squish) + # Keeps values outside range from turning gray
    theme(axis.text.x = element_text(size = 6),
          axis.text.y = element_text(size = 6))
```

```
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
```



Summary and Descriptive Statistics

```
summary(xy)
```

```
##        y             Amount           TimeOfDay         LocationRisk
##  Min.   :0.0000   Min.   :-3.58787   Min.   :-4.19556   Min.   :-3.655873
##  1st Qu.:0.0000   1st Qu.:-0.66165   1st Qu.:-0.70010   1st Qu.:-0.674483
##  Median :0.0000   Median : 0.01531   Median :-0.02141   Median : 0.002743
##  Mean   :0.2017   Mean   : 0.01284   Mean   :-0.02414   Mean   : 0.007563
##  3rd Qu.:0.0000   3rd Qu.: 0.68323   3rd Qu.: 0.64899   3rd Qu.: 0.670564
##  Max.   :1.0000   Max.   : 3.60704   Max.   : 3.72447   Max.   : 4.096971
##  MerchantScore        UserHistory         DeviceScore
```

```
##  Min.   :-3.676363   Min.   :-3.948217   Min.   :-4.251565
##  1st Qu.:-0.676339   1st Qu.:-0.679048   1st Qu.:-0.663461
##  Median :-0.001190   Median : 0.006996   Median : 0.009104
##  Mean   : 0.005064   Mean   :-0.001301   Mean   : 0.002508
##  3rd Qu.: 0.684693   3rd Qu.: 0.668798   3rd Qu.: 0.677288
##  Max.   : 3.752972   Max.   : 4.094045   Max.   : 3.964490
##  TransactionSpeed       IntlFlag         RepeatMerchant      Velocity_1h
##  Min.   :-4.024105   Min.   :-3.64040   Min.   :-3.52651   Min.   :-3.89827
##  1st Qu.:-0.684890   1st Qu.:-0.66624   1st Qu.:-0.66157   1st Qu.:-0.67477
##  Median :-0.002667   Median : 0.01796   Median :-0.01306   Median : 0.02198
##  Mean   :-0.001115   Mean   : 0.00661   Mean   :-0.01640   Mean   : 0.01592
##  3rd Qu.: 0.675271   3rd Qu.: 0.68056   3rd Qu.: 0.65041   3rd Qu.: 0.69892
##  Max.   : 3.511983   Max.   : 4.06398   Max.   : 3.81089   Max.   : 4.30142
##    Behavior1           Behavior2           Behavior3
##  Min.   :-3.987060   Min.   :-3.622841   Min.   :-4.0864920
##  1st Qu.:-0.683026   1st Qu.:-0.675384   1st Qu.:-0.6707141
##  Median :-0.006770   Median : 0.005613   Median :-0.0120703
##  Mean   :-0.003412   Mean   : 0.002069   Mean   :-0.0003952
##  3rd Qu.: 0.674948   3rd Qu.: 0.685179   3rd Qu.: 0.6582017
##  Max.   : 3.424848   Max.   : 3.632972   Max.   : 4.1798725
##    Behavior4           Behavior5           Behavior6           Behavior7
##  Min.   :-3.444374   Min.   :-4.327471   Min.   :-3.644331   Min.   :-3.78574
##  1st Qu.:-0.670994   1st Qu.:-0.685763   1st Qu.:-0.701775   1st Qu.:-0.65466
##  Median : 0.002483   Median :-0.008512   Median :-0.004400   Median : 0.02192
##  Mean   : 0.008986   Mean   :-0.003848   Mean   :-0.000682   Mean   : 0.02022
##  3rd Qu.: 0.676188   3rd Qu.: 0.670284   3rd Qu.: 0.682212   3rd Qu.: 0.69369
##  Max.   : 3.795975   Max.   : 3.782282   Max.   : 4.266974   Max.   : 3.48584
##    Behavior8           Behavior9           Behavior10
##  Min.   :-4.35936   Min.   :-3.538875   Min.   :-3.9154808
##  1st Qu.:-0.66587   1st Qu.:-0.672448   1st Qu.:-0.6592105
##  Median : 0.00224   Median : 0.007458   Median :-0.0009284
##  Mean   : 0.01242   Mean   : 0.004217   Mean   : 0.0067083
##  3rd Qu.: 0.68994   3rd Qu.: 0.681885   3rd Qu.: 0.6871975
##  Max.   : 4.01019   Max.   : 4.155782   Max.   : 3.5970408
##    Behavior11          Behavior12          Behavior13          Behavior14
##  Min.   :-3.409273   Min.   :-4.179833   Min.   :-3.813077   Min.   :-4.02171
##  1st Qu.:-0.663661   1st Qu.:-0.690288   1st Qu.:-0.670738   1st Qu.:-0.66364
##  Median :-0.008957   Median :-0.001505   Median : 0.004237   Median : 0.01794
##  Mean   : 0.007198   Mean   :-0.002765   Mean   : 0.001242   Mean   : 0.01803
##  3rd Qu.: 0.688982   3rd Qu.: 0.668470   3rd Qu.: 0.675027   3rd Qu.: 0.68535
##  Max.   : 4.549898   Max.   : 3.709642   Max.   : 3.607197   Max.   : 3.80882
##    Behavior15          Behavior16          Behavior17
##  Min.   :-3.808786   Min.   :-3.646939   Min.   :-3.746296
##  1st Qu.:-0.652258   1st Qu.:-0.675633   1st Qu.:-0.664075
##  Median : 0.006361   Median :-0.003013   Median : 0.003336
##  Mean   : 0.011092   Mean   : 0.004066   Mean   : 0.010980
##  3rd Qu.: 0.685630   3rd Qu.: 0.681391   3rd Qu.: 0.681535
##  Max.   : 4.036110   Max.   : 4.580542   Max.   : 3.891641
##    Behavior18          Behavior19          Behavior20
##  Min.   :-3.927130   Min.   :-3.501962   Min.   :-3.896225
##  1st Qu.:-0.699472   1st Qu.:-0.669844   1st Qu.:-0.681660
##  Median :-0.016641   Median :-0.005843   Median : 0.007651
##  Mean   :-0.008834   Mean   : 0.001259   Mean   : 0.005203
##  3rd Qu.: 0.683828   3rd Qu.: 0.677442   3rd Qu.: 0.685779
```

```
##  Max.   : 3.787785   Max.   : 3.961008   Max.   : 3.590787
##    Behavior21          Behavior22          Behavior23
##  Min.   :-3.522089   Min.   :-3.633883   Min.   :-3.843450
##  1st Qu.:-0.650358   1st Qu.:-0.682667   1st Qu.:-0.701243
##  Median : 0.008137   Median :-0.003217   Median :-0.006238
##  Mean   : 0.016929   Mean   :-0.006335   Mean   :-0.009515
##  3rd Qu.: 0.691131   3rd Qu.: 0.662956   3rd Qu.: 0.682200
##  Max.   : 3.861008   Max.   : 3.859730   Max.   : 3.344150
##    Behavior24          Behavior25          Behavior26          Behavior27
##  Min.   :-3.912310   Min.   :-4.099357   Min.   :-3.65846   Min.   :-3.953946
##  1st Qu.:-0.692232   1st Qu.:-0.672778   1st Qu.:-0.65171   1st Qu.:-0.682878
##  Median :-0.007824   Median : 0.009138   Median : 0.01617   Median :-0.010654
##  Mean   :-0.014541   Mean   : 0.001882   Mean   : 0.01688   Mean   :-0.005448
##  3rd Qu.: 0.656699   3rd Qu.: 0.674419   3rd Qu.: 0.68304   3rd Qu.: 0.665014
##  Max.   : 4.190649   Max.   : 3.848436   Max.   : 3.63075   Max.   : 4.837018
##    Behavior28          Behavior29          Behavior30          Behavior31
##  Min.   :-3.622736   Min.   :-4.20324   Min.   :-3.52882   Min.   :-3.959528
##  1st Qu.:-0.673353   1st Qu.:-0.69154   1st Qu.:-0.64566   1st Qu.:-0.673530
##  Median : 0.005477   Median : 0.02457   Median : 0.01656   Median :-0.002394
##  Mean   : 0.002055   Mean   :-0.00543   Mean   : 0.01180   Mean   : 0.005253
##  3rd Qu.: 0.674400   3rd Qu.: 0.67513   3rd Qu.: 0.67728   3rd Qu.: 0.710255
##  Max.   : 4.283522   Max.   : 3.71394   Max.   : 3.68684   Max.   : 3.578808
##    Behavior32          Behavior33          Behavior34
##  Min.   :-3.706166   Min.   :-4.639617   Min.   :-3.747087
##  1st Qu.:-0.683203   1st Qu.:-0.670358   1st Qu.:-0.682474
##  Median :-0.008692   Median : 0.003466   Median : 0.002004
##  Mean   :-0.004840   Mean   : 0.010570   Mean   :-0.004151
##  3rd Qu.: 0.673421   3rd Qu.: 0.682335   3rd Qu.: 0.672663
##  Max.   : 3.941729   Max.   : 3.888808   Max.   : 3.635207
##    Behavior35          Behavior36          Behavior37
##  Min.   :-3.648041   Min.   :-3.685795   Min.   :-4.204849
##  1st Qu.:-0.656866   1st Qu.:-0.659335   1st Qu.:-0.662726
##  Median : 0.021411   Median : 0.012553   Median : 0.004896
##  Mean   : 0.009907   Mean   : 0.008638   Mean   : 0.002959
##  3rd Qu.: 0.675347   3rd Qu.: 0.681300   3rd Qu.: 0.669444
##  Max.   : 3.647024   Max.   : 4.954252   Max.   : 3.703184
##    Behavior38          Behavior39          Behavior40
##  Min.   :-3.303600   Min.   :-4.442828   Min.   :-4.193199
##  1st Qu.:-0.668798   1st Qu.:-0.662393   1st Qu.:-0.655005
##  Median : 0.016327   Median :-0.006631   Median : 0.004213
##  Mean   : 0.009358   Mean   : 0.006203   Mean   :-0.001507
##  3rd Qu.: 0.697272   3rd Qu.: 0.694224   3rd Qu.: 0.671283
##  Max.   : 4.135102   Max.   : 3.825825   Max.   : 3.419340
```

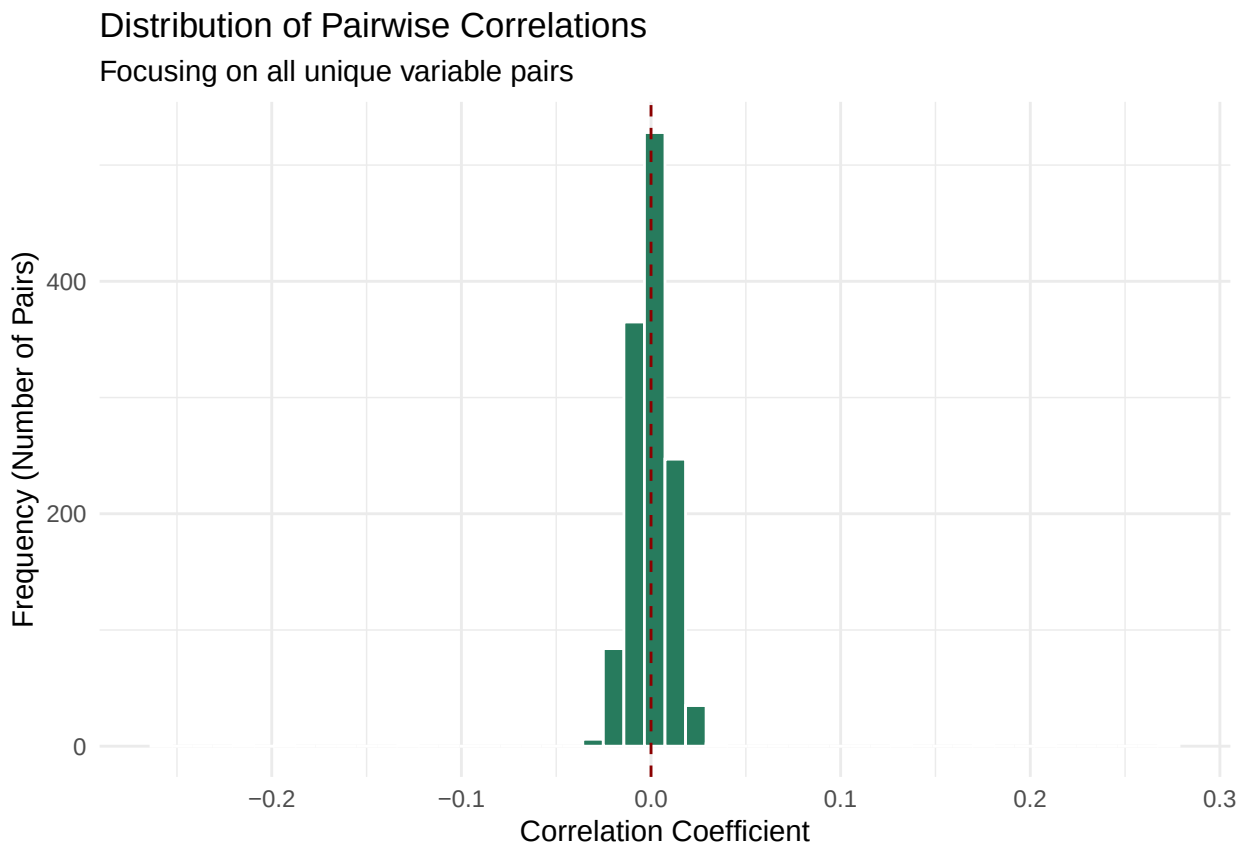
```r
cor_pairs <- cor(xy, use = "complete.obs") %>%
  as.data.frame() %>%
  rownames_to_column("Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "Correlation")


top_pairs <- cor_pairs %>%
  filter(Var1 < Var2) %>%
  mutate(Abs_Correlation = abs(Correlation)) %>%
  arrange(desc(Abs_Correlation))
```

```r
#look at top 10
head(top_pairs, 10)
```

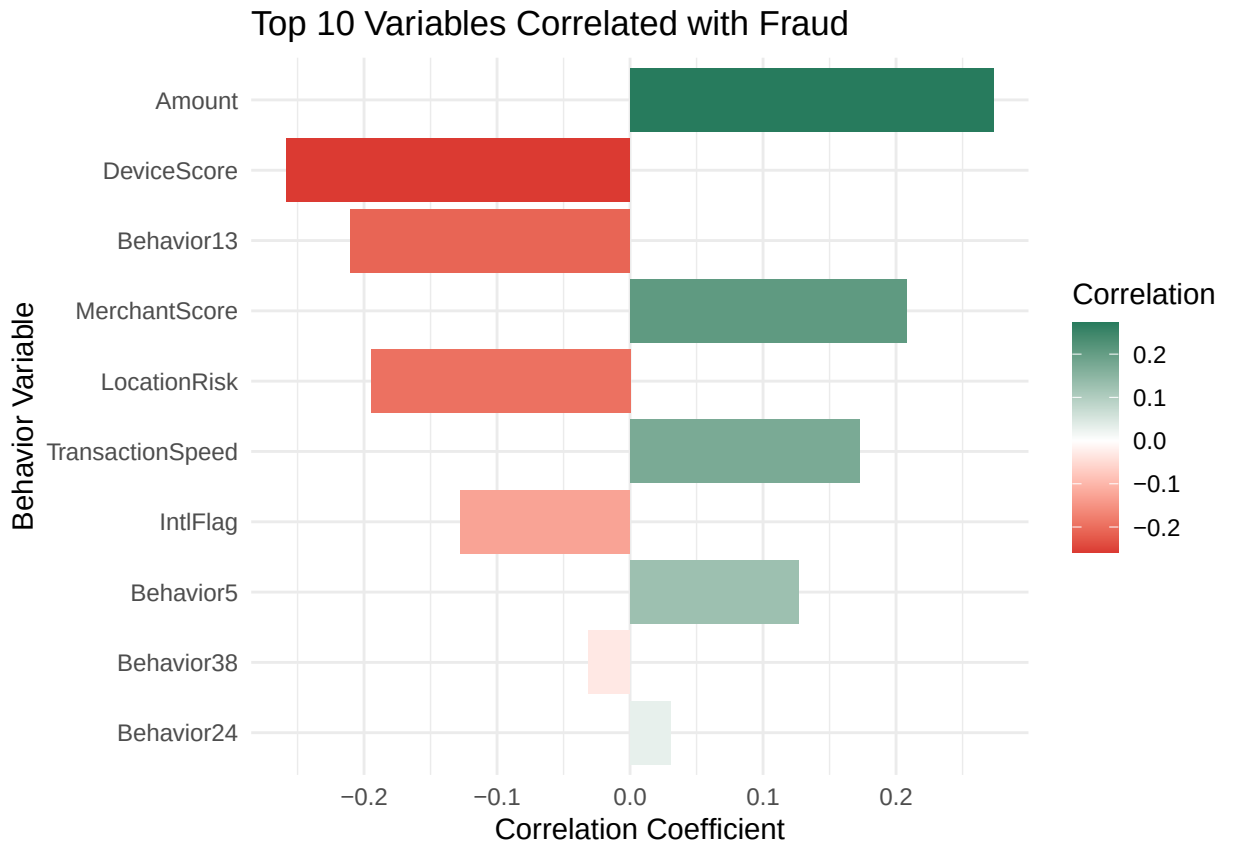
```
## # A tibble: 10 x 4
##    Var1             Var2       Correlation Abs_Correlation
##    <chr>            <chr>            <dbl>           <dbl>
##  1 Amount           y                0.273           0.273
##  2 DeviceScore      y               -0.258           0.258
##  3 Behavior13       y               -0.210           0.210
##  4 MerchantScore    y                0.208           0.208
##  5 LocationRisk     y               -0.195           0.195
##  6 TransactionSpeed y                0.173           0.173
##  7 IntlFlag         y               -0.128           0.128
##  8 Behavior5        y                0.127           0.127
##  9 Behavior38       y               -0.0313          0.0313
## 10 Amount           Behavior13       0.0311          0.0311
```

Plots Regarding Correlation

```r
ggplot(top_pairs, aes(x = Correlation)) +
  geom_histogram(bins = 50, fill = "#277B5D", color = "white") +
  theme_minimal() +
  labs(title = "Distribution of Pairwise Correlations",
       subtitle = "Focusing on all unique variable pairs",
       x = "Correlation Coefficient",
       y = "Frequency (Number of Pairs)") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "darkred")
```




```r
cor_y <- cor(xy, use = "complete.obs")[, "y"]


top_fraud_cor <- data.frame(
  Variable = names(cor_y),
  Correlation = cor_y,
  Abs_Correlation = abs(cor_y)
)
plot_data <- top_fraud_cor %>%
  filter(Variable != "y") %>%
  slice_max(Abs_Correlation, n = 10)


ggplot(plot_data, aes(x = reorder(Variable, Abs_Correlation), y = Correlation, fill = Correlation)) +
  geom_col() +
  coord_flip() + # Makes it easier to read variable names
  scale_fill_gradient2(low = "#D62828", mid = "white", high = "#277B5D", midpoint = 0) +
  theme_minimal() +
  labs(title = "Top 10 Variables Correlated with Fraud",
       x = "Behavior Variable",
       y = "Correlation Coefficient")
```



There is something here to be said about the correlation between variables of the continuous type to the response variable Y of the discrete binary type. Recall the Pearson Correlation as

$$\rho = \frac{\sum_{1\leq i\leq n}(x_i-\overline{x})(y_i-\overline{y})}{\sqrt{\sum_{1\leq i\leq n}(x_i-\overline{x})^2\sum_{1\leq i\leq n}(y_i-\overline{y})^2}}$$

then here we can compute $\overline{y} = \sum y_i/n$ due to the binary type of the responses.

```
sum(y_numeric)/n
```

```
## [1] 0.2017
```

```
n1 <- sum(y_numeric)
n0 <- n - n1
n1; n0
```

```
## [1] 2017
```

```
## [1] 7983
```

Notice that

$$\sum(y_i-\overline{y})^2 = n_1(1-\sum y_i/n)^2 + n_0(0-\sum y_i/n)^2 = \frac{n_0n_1}{n}$$

with some algebra we obtain the form

$$\rho_{binary} = \frac{\bar{x}_1-\bar{x}_0}{\sum_{1\leq i\leq n}(x_i-\overline{x})^2}\cdot\sqrt{\frac{n_1n_0}{n^2}} \tag{1}$$

Notice that in our case of the class in balance we can make the point that ρ_{binary} gets shrink very close to 0 as $n \to \infty$, hence why we see small correlations overall but those variables would have had higher correlation for a balanced class. Also note that we interpret a positive correlation with a higher increase in x_i yields a correlation with class 1 which is fraud and vice versa.

Feature Engineering: Interations and transformations

```
topvar <-  c("Amount", "DeviceScore", "Behavior13", "MerchantScore", "LocationRisk", "TransactionSpeed")
x_topvar<- x[,topvar]
interact_df <- as.data.frame(model.matrix(~.^2 - 1, data = x_topvar))
interact_df <- interact_df[, !names(interact_df) %in% topvar]
poly_df <- as.data.frame(
  lapply(topvar, function(var) {
  df <- data.frame(x[[var]]^2)
  names(df) <- paste0(var, "_sq")
return(df)
})
)

names(poly_df) #6 New Var
```

```
## [1] "Amount_sq"           "DeviceScore_sq"      "Behavior13_sq"
## [4] "MerchantScore_sq"    "LocationRisk_sq"     "TransactionSpeed_sq"
```

```
names(interact_df) #15 New Var
```

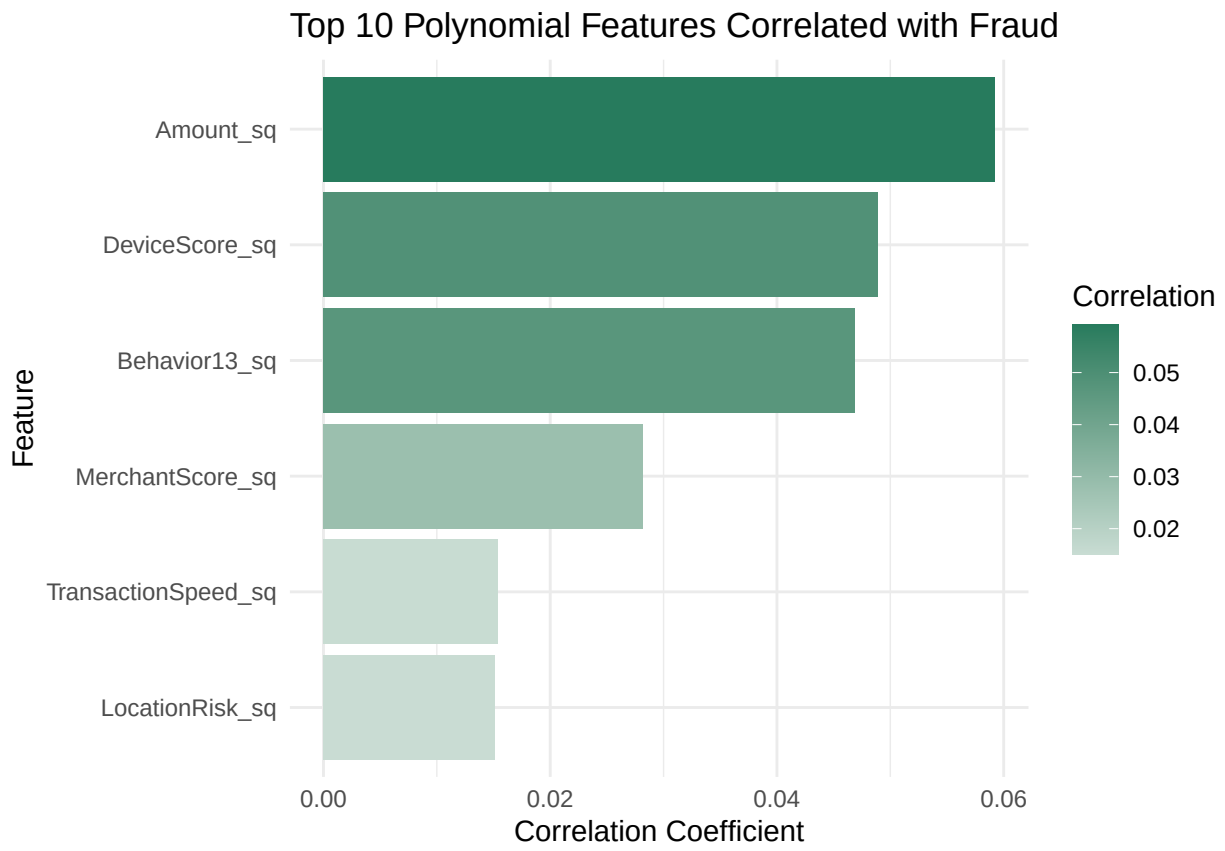
```
##  [1] "Amount:DeviceScore"             "Amount:Behavior13"
##  [3] "Amount:MerchantScore"           "Amount:LocationRisk"
##  [5] "Amount:TransactionSpeed"        "DeviceScore:Behavior13"
##  [7] "DeviceScore:MerchantScore"      "DeviceScore:LocationRisk"
##  [9] "DeviceScore:TransactionSpeed"   "Behavior13:MerchantScore"
## [11] "Behavior13:LocationRisk"        "Behavior13:TransactionSpeed"
## [13] "MerchantScore:LocationRisk"     "MerchantScore:TransactionSpeed"
## [15] "LocationRisk:TransactionSpeed"
```

```
x_eng <- cbind(x, interact_df, poly_df)
#View(x_eng) #should be 50 + 21 = 71 var now
```

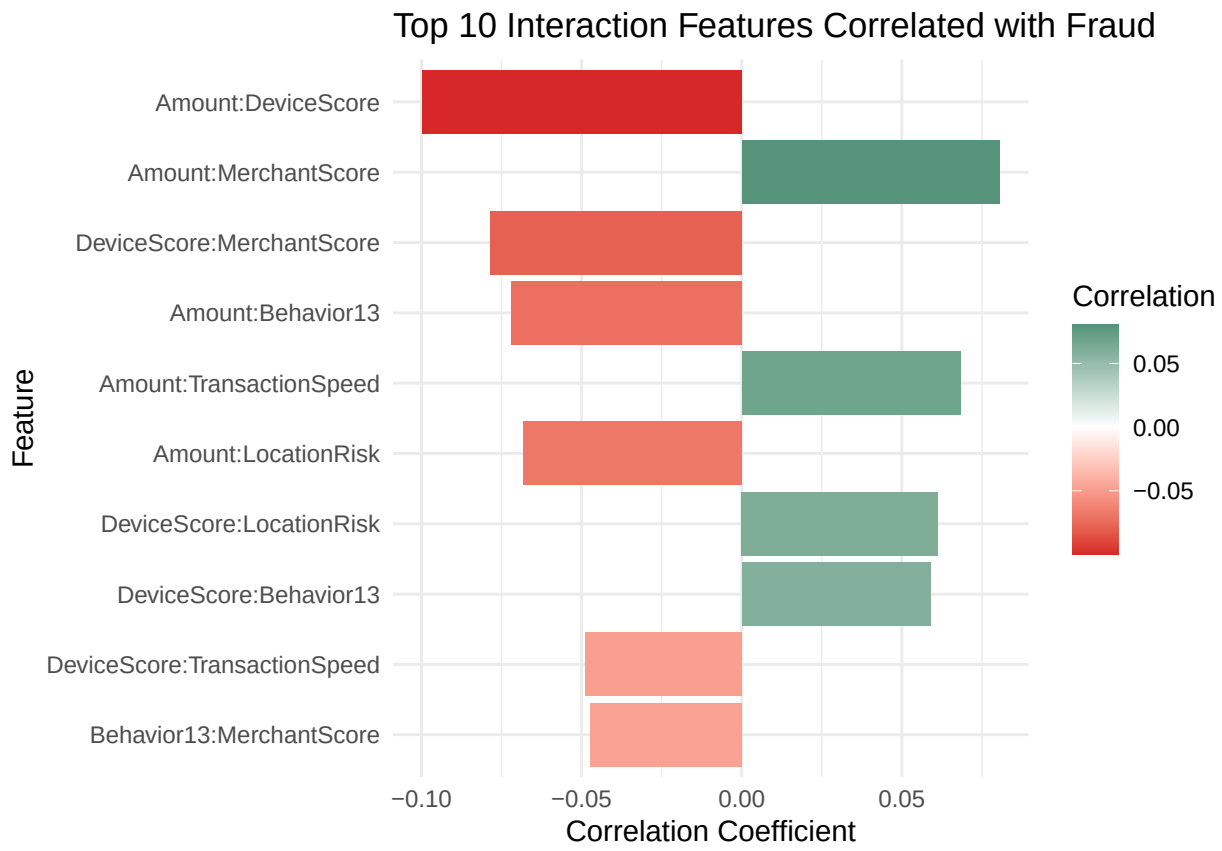
Checking the correlation on the new variables

```
# Combine poly and interaction features with y
poly_cor <- cor(cbind(y = y_numeric, poly_df), use = "complete.obs")[, "y"]
interact_cor <- cor(cbind(y = y_numeric, interact_df), use = "complete.obs")[, "y"]

# Build tidy data frames for each
poly_plot_data <- data.frame(
  Variable       = names(poly_cor),
  Correlation    = poly_cor,
  Abs_Correlation = abs(poly_cor)
) %>%
  filter(Variable != "y") %>%
```

```
  slice_max(Abs_Correlation, n = 10)

interact_plot_data <- data.frame(
  Variable        = names(interact_cor),
  Correlation     = interact_cor,
  Abs_Correlation = abs(interact_cor)
) %>%
  filter(Variable != "y") %>%
  slice_max(Abs_Correlation, n = 10)

# Plot polynomial features
p_poly <- ggplot(poly_plot_data,
                 aes(x = reorder(Variable, Abs_Correlation),
                     y = Correlation,
                     fill = Correlation)) +
  geom_col() +
  coord_flip() +
  scale_fill_gradient2(low = "#D62828", mid = "white", high = "#277B5D", midpoint = 0) +
  theme_minimal() +
  labs(title = "Top 10 Polynomial Features Correlated with Fraud",
       x = "Feature",
       y = "Correlation Coefficient")

# Plot interaction features
p_interact <- ggplot(interact_plot_data,
                     aes(x = reorder(Variable, Abs_Correlation),
                         y = Correlation,
                         fill = Correlation)) +
  geom_col() +
  coord_flip() +
  scale_fill_gradient2(low = "#D62828", mid = "white", high = "#277B5D", midpoint = 0) +
  theme_minimal() +
  labs(title = "Top 10 Interaction Features Correlated with Fraud",
       x = "Feature",
       y = "Correlation Coefficient")

p_poly
```


Top 10 Polynomial Features Correlated with Fraud
Amount_sq
DeviceScore_sq
Behavior13_sq
MerchantScore_sq
TransactionSpeed_sq
LocationRisk_sq
Feature
0.00
0.02
0.04
0.06
Correlation Coefficient
Correlation
0.05
0.04
0.03
0.02

```
p_interact
```




```r
# Original variables
original_cor_df <- data.frame(
  Variable        = names(cor_y),
  Correlation     = as.numeric(cor_y),
  Abs_Correlation = abs(as.numeric(cor_y)),
  Type            = "Original"
) %>% filter(Variable != "y")

#Polynomial features
poly_cor_df <- data.frame(
  Variable        = names(poly_cor),
  Correlation     = as.numeric(poly_cor),
  Abs_Correlation = abs(as.numeric(poly_cor)),
  Type            = "Polynomial"
) %>% filter(Variable != "y")

# Interaction features
interact_cor_df <- data.frame(
  Variable        = names(interact_cor),
  Correlation     = as.numeric(interact_cor),
  Abs_Correlation = abs(as.numeric(interact_cor)),
  Type            = "Interaction"
) %>% filter(Variable != "y")

#Combine all three
all_cor_df <- bind_rows(original_cor_df, poly_cor_df, interact_cor_df) %>%
```

```
  arrange(desc(Abs_Correlation))

head(all_cor_df, 10)
```

```
##                Variable Correlation Abs_Correlation        Type
## 1                Amount  0.27300380      0.27300380    Original
## 2           DeviceScore -0.25826636      0.25826636    Original
## 3            Behavior13 -0.21020439      0.21020439    Original
## 4         MerchantScore  0.20771226      0.20771226    Original
## 5          LocationRisk -0.19477158      0.19477158    Original
## 6      TransactionSpeed  0.17277006      0.17277006    Original
## 7              IntlFlag -0.12757162      0.12757162    Original
## 8             Behavior5  0.12699593      0.12699593    Original
## 9    Amount:DeviceScore -0.09980746      0.09980746 Interaction
## 10 Amount:MerchantScore  0.08053208      0.08053208 Interaction
```

Looking at the top ten variables again, we can see that the two new interaction terms with `Amount` in it enters the top 10 variables league.

```
names(x_eng) <- make.names(names(x_eng), unique = TRUE)
set.seed(42261712)
original_df        <- cbind(x_eng, Fraud = as.factor(y_numeric))
levels(original_df$Fraud) <- c("No", "Yes")
strat_idx          <- createDataPartition(original_df$Fraud, p = 0.80, list = FALSE)
original_train     <- original_df[ strat_idx, ]
original_test      <- original_df[-strat_idx, ]
```

Class imbalance handling with SMOTE and Weighting

```
smote_input <- cbind(x_eng, Fraud = y_numeric)

smote_result <- SMOTE(
  X    = smote_input[, !names(smote_input) %in% "Fraud"],
  target = smote_input$Fraud,
  K    = 5,
  dup_size = 0
)

smote_df <- smote_result$data

smote_df <- smote_df %>% rename(Fraud = class)
smote_df$Fraud <- as.numeric(as.character(smote_df$Fraud))

cat("SMOTE Class Distribution:\n")
```

```
## SMOTE Class Distribution:
```

```
table(smote_df$Fraud)
```

```
##
##    0    1
## 7983 6051
```

```r
x_smote <- smote_df %>% dplyr::select(Fraud)
y_smote <- as.factor(smote_df$Fraud)
```

```r
fraud_table <- table(y_numeric)
class_weights <- (1 / fraud_table) / sum(1 / fraud_table)  # normalized

cat("Class Weights:\n")
```

```
## Class Weights:
```

```r
print(class_weights)
```

```
## y_numeric
##      0      1
## 0.2017 0.7983
```

```r
sample_weights <- ifelse(
  y_numeric == 1,
  class_weights["1"],
  class_weights["0"]
)

weighted_df <- cbind(x_eng,
                     Fraud   = y_numeric,
                     Weight  = sample_weights)

cat("\nWeight summary:\n")
```

```
##
## Weight summary:
```

```r
summary(sample_weights)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2017  0.2017  0.2017  0.3220  0.2017  0.7983
```

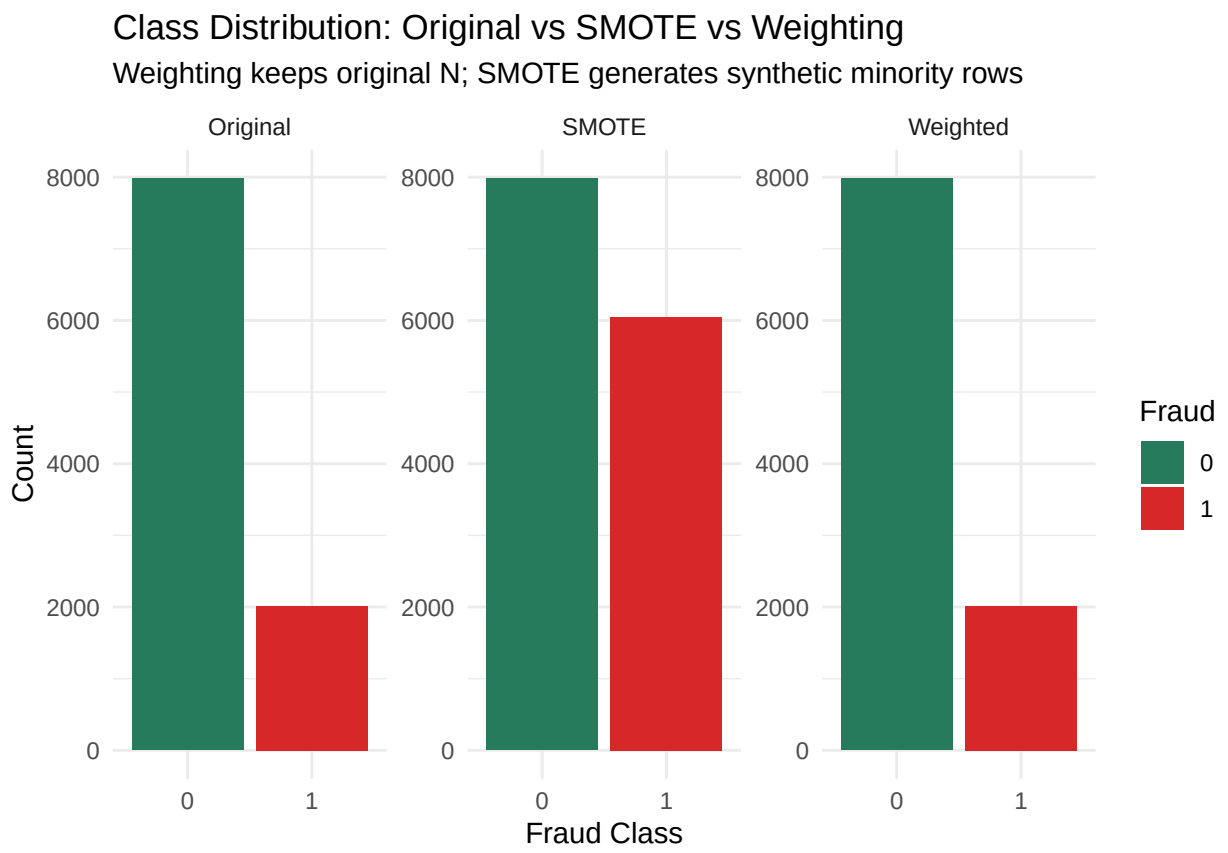
```r
train_input   <- original_train
train_input$Fraud <- ifelse(train_input$Fraud == "Yes", 1, 0)

smote_train <- SMOTE(
  X        = train_input[, !names(train_input) %in% "Fraud"],
  target   = train_input$Fraud,
  K        = 5,
  dup_size = 0
)$data %>%
  rename(Fraud = class) %>%
  mutate(Fraud = as.factor(Fraud))

levels(smote_train$Fraud) <- c("No", "Yes")
names(smote_train) <- make.names(names(smote_train), unique = TRUE)

print(table(smote_train$Fraud))
```

```
##
##   No  Yes
## 6387 4842
```

```
print(table(original_test$Fraud))
```

```
##
##   No  Yes
## 1596  403
```

```
balance_compare <- bind_rows(
  data.frame(Dataset = "Original",
             Fraud   = as.factor(y_numeric)),
  data.frame(Dataset = "SMOTE",
             Fraud   = as.factor(smote_df$Fraud)),
  data.frame(Dataset = "Weighted",
             Fraud   = as.factor(y_numeric))
)

ggplot(balance_compare, aes(x = Fraud, fill = Fraud)) +
  geom_bar() +
  facet_wrap(~Dataset, scales = "free_y") +
  scale_fill_manual(values = c("0" = "#277B5D", "1" = "#D62828")) +
  theme_minimal() +
  labs(title    = "Class Distribution: Original vs SMOTE vs Weighting",
       subtitle = "Weighting keeps original N; SMOTE generates synthetic minority rows",
       x = "Fraud Class",
       y = "Count")
```

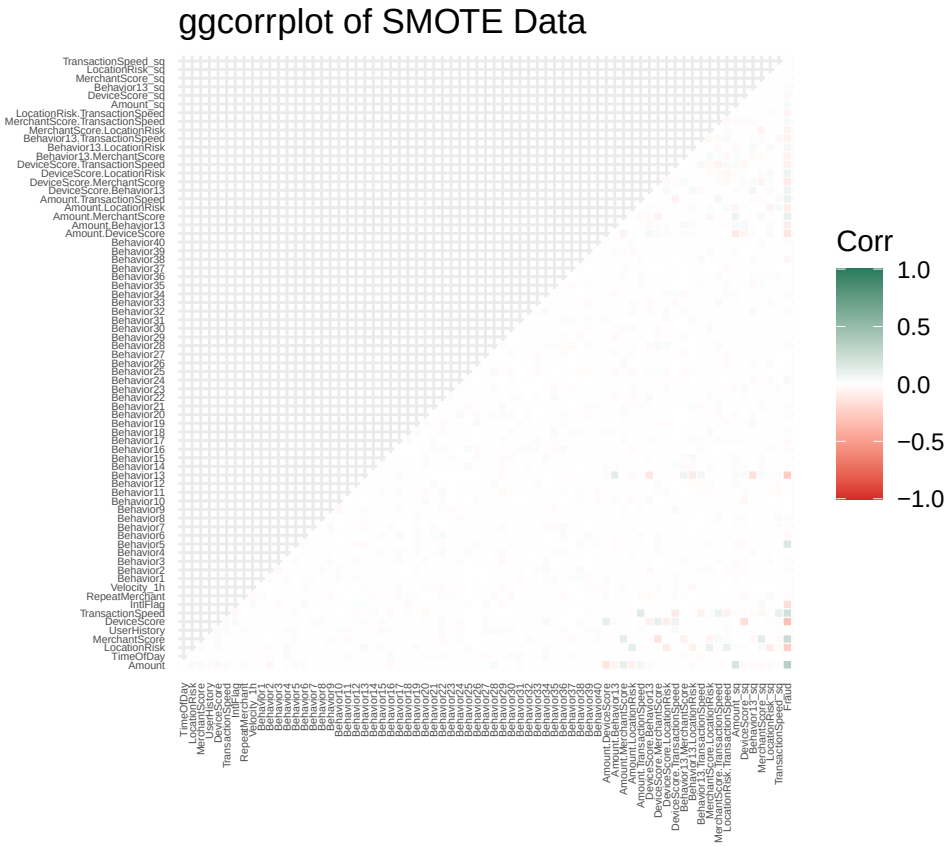


New EDA Shows that the top ten variables with the highest correlation with Fraud from the `smote_df` actually remains the same as the original dataset.

```
#View(smote_df)


dim(smote_df)
```

```
## [1] 14034    72
```

```
smote_cor <- cor(smote_df, use = "complete.obs")



ggcorrplot(smote_cor,
           type = "lower",
           outline.col = "white",
           colors = c("#D62828", "white", "#277B5D"),
           tl.cex = 4) +     # Shrink text size
  labs(title = "ggcorrplot of SMOTE Data") +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust=1))
```




```
smote_cor_y <- cor(smote_df, use = "complete.obs")[, "Fraud"]

smote_top_fraud_cor <- data.frame(
  Variable = names(smote_cor_y),
  Correlation = as.numeric(smote_cor_y),
  Abs_Correlation = abs(as.numeric(smote_cor_y))
)

smote_plot_data <- smote_top_fraud_cor %>%
  filter(Variable != "Fraud") %>%
  arrange(desc(Abs_Correlation))


head(smote_plot_data, 10)
```

```
##                Variable Correlation Abs_Correlation
## 1                Amount   0.3417265       0.3417265
## 2           DeviceScore  -0.3240161       0.3240161
## 3         MerchantScore   0.2603948       0.2603948
## 4             Behavior13  -0.2591758       0.2591758
## 5          LocationRisk  -0.2493102       0.2493102
## 6      TransactionSpeed   0.2191978       0.2191978
## 7               IntlFlag  -0.1700417       0.1700417
## 8             Behavior5   0.1656894       0.1656894
## 9    Amount.DeviceScore  -0.1277557       0.1277557
## 10 Amount.MerchantScore   0.1081025       0.1081025
```

```
smote_df$Fraud <- as.factor(smote_df$Fraud)
x_mat <- as.matrix(smote_train[, !names(smote_train) %in% "Fraud"])
y_num <- as.numeric(smote_train$Fraud == "Yes")
smote_df$Fraud <- c(1,0)
```

Model Specification and Training

Regularized Logistic

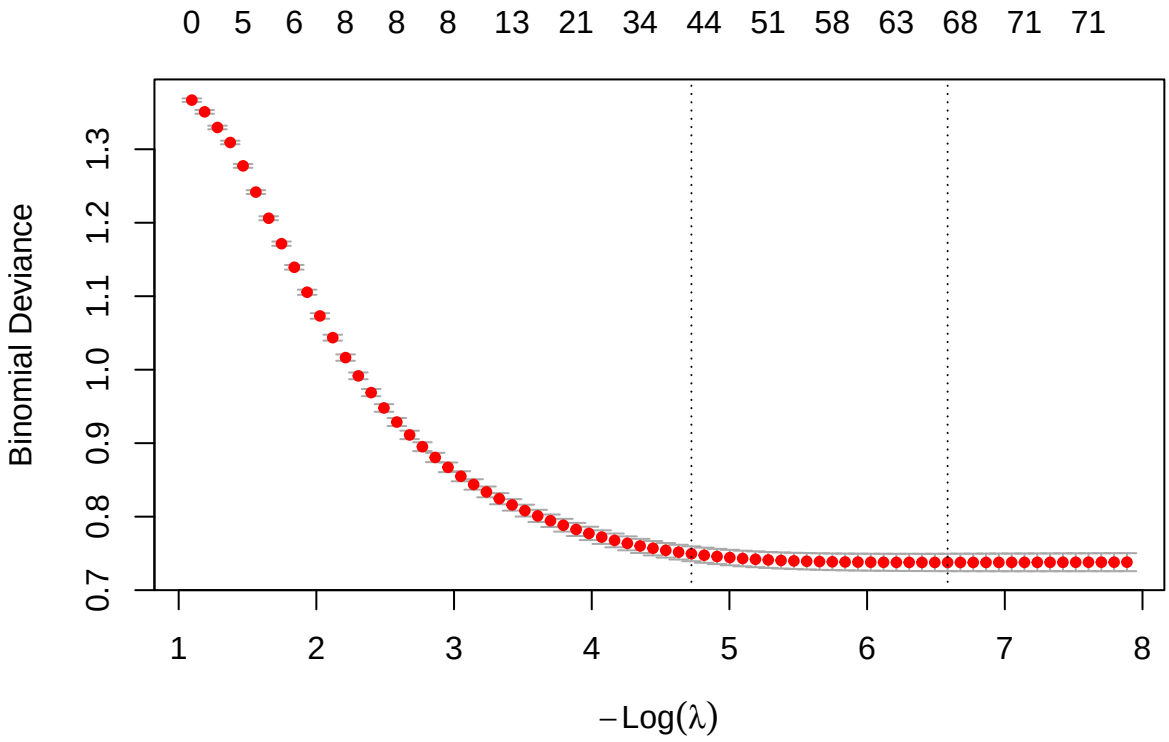
First we implement the Regularized Logistic as

$$\min_{\beta_0,\beta} -\frac{1}{N}\sum_{i=1}^{N} \ell(y_i, \beta_0 + \beta^T x_i) + \lambda \left[\frac{1-\alpha}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1 \right]$$

```
model_logit <- cv.glmnet(x_mat, y_num, family = "binomial", alpha = 0.5)
model_logit$lambda.1se
```

```
## [1] 0.008879581
```

```
#names(model_logit)
#model_logit$cvm
#model_logit$cvup
#model_logit$cvlo
#model_logit$glmnet.fit
#print(model_logit)
plot(model_logit)
```



Based on the cross validation, if want a simpler model we should prefer bigger λ meaning moving to the left as $-\log(\lambda)$ is a decreasing function. Thus we want a model with penalty `Lambda1se` with $-\log(\lambda) \approx 5$

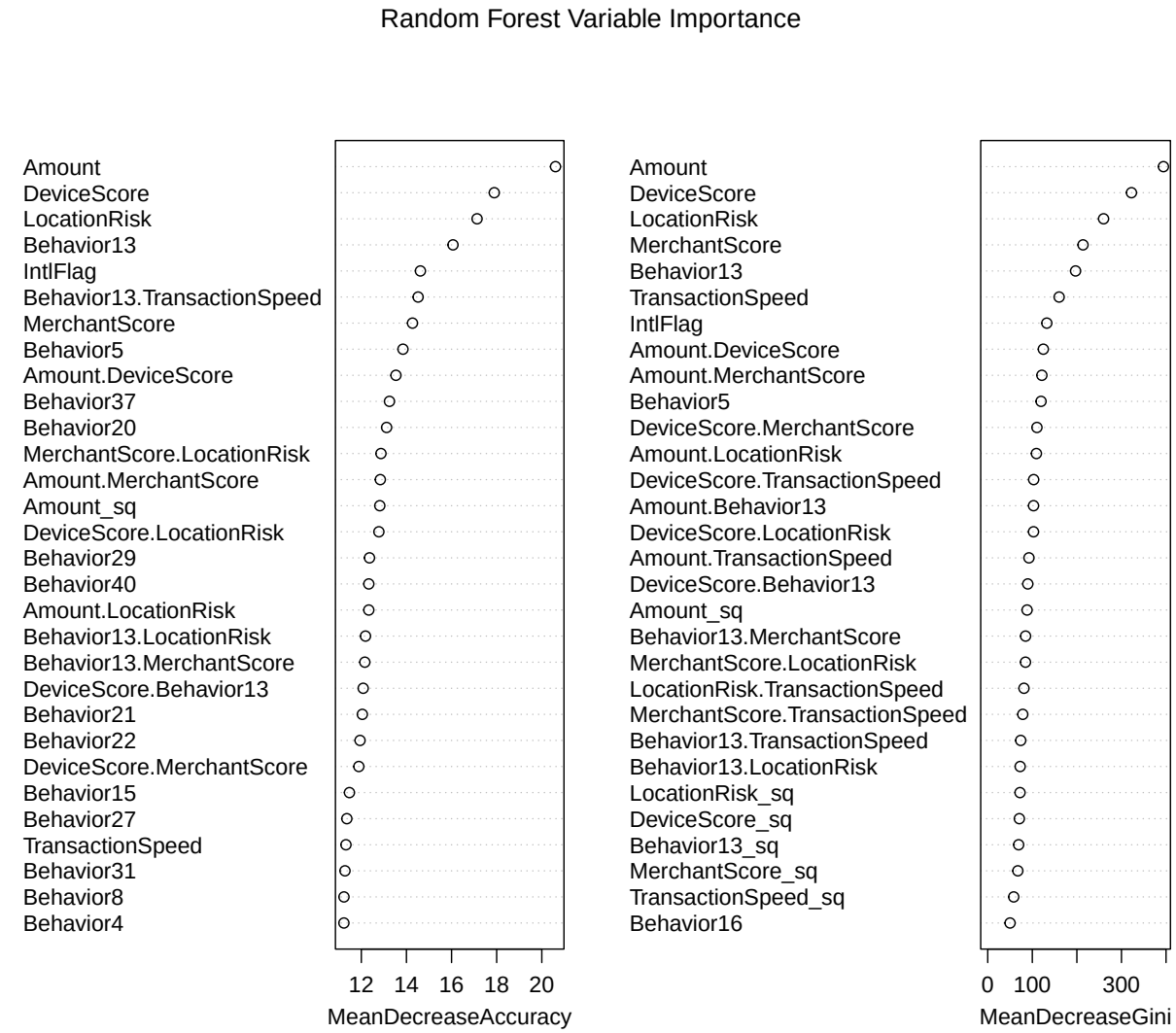
Random Forest

```
names(smote_df) <- make.names(names(smote_df), unique = TRUE)
smote_df$Fraud  <- as.factor(smote_df$Fraud)

x_mat <- as.matrix(smote_df[, !names(smote_df) %in% "Fraud"])
y_num <- as.numeric(as.character(smote_df$Fraud))

model_rf <- randomForest(Fraud ~ ., data = smote_train, ntree = 100, importance = TRUE)
print(model_rf)
```

```
##
## Call:
##  randomForest(formula = Fraud ~ ., data = smote_train, ntree = 100,      importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 100
## No. of variables tried at each split: 8
##
##         OOB estimate of  error rate: 9.85%
## Confusion matrix:
##       No  Yes class.error
## No  5813  574  0.08987005
```

```
## Yes  532 4310  0.10987195
```

```r
varImpPlot(model_rf, main = "Random Forest Variable Importance")
```



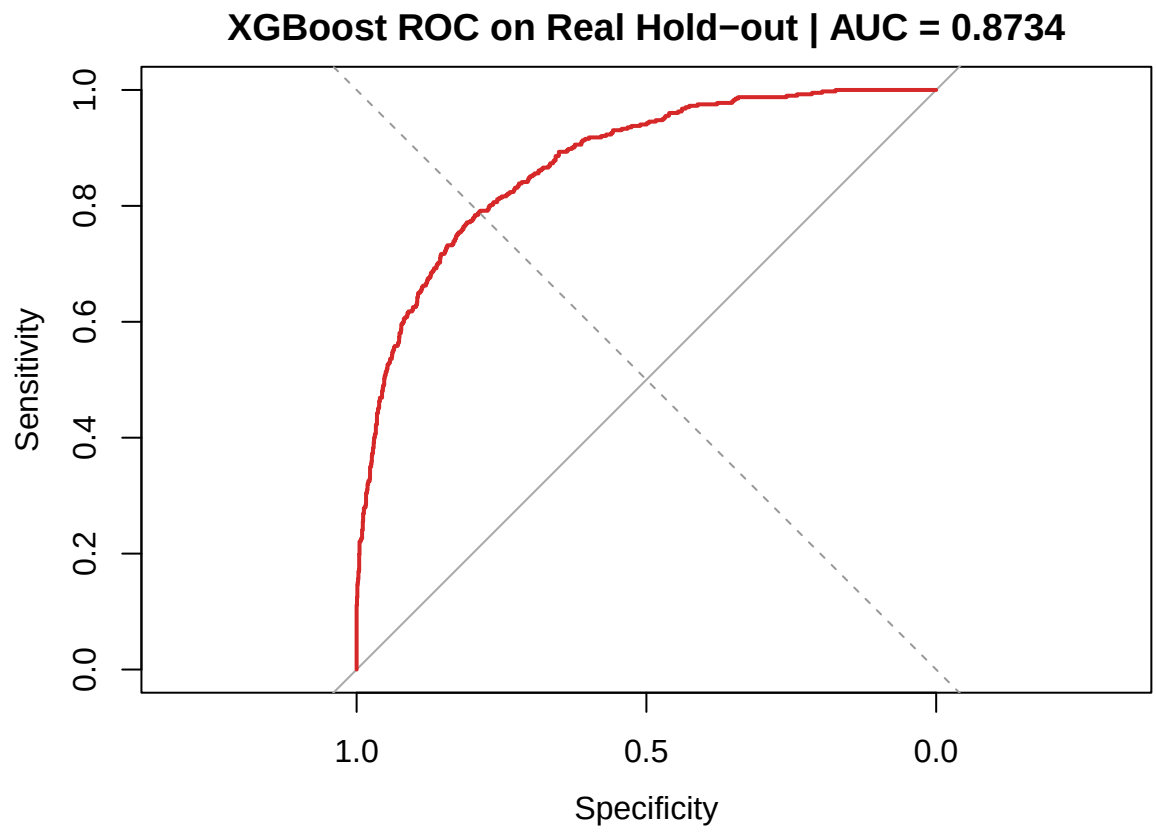
Gradient Boosting: XGBoost

```r
#  XGBoost
levels(smote_df$Fraud) <- c("No", "Yes")

x_mat <- as.matrix(smote_train[, !names(smote_train) %in% "Fraud"])
y_num <- ifelse(smote_train$Fraud == "Yes", 1L, 0L)

dtrain <- xgb.DMatrix(data = x_mat, label = y_num)


model_xgb <- xgboost::xgb.train(
```

```r
  params  = list(
    objective     = "binary:logistic",
    learning_rate = 0.3,
    max_depth     = 6,
    subsample     = 1.0
  ),
  data    = dtrain,
  nrounds = 50,
  verbose = 0
)

#  Predict on original real hold-out
x_test_xgb <- as.matrix(original_test[, !names(original_test) %in% "Fraud"])
dtest      <- xgb.DMatrix(data = x_test_xgb)

pred_prob_xgb  <- predict(model_xgb, dtest)
pred_class_xgb <- as.factor(ifelse(pred_prob_xgb >= 0.5, "Yes", "No"))

cm_xgb <- caret::confusionMatrix(pred_class_xgb,
                          original_test$Fraud,
                          positive = "Yes")
print(cm_xgb)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  1466  159
##        Yes  130  244
##
##                Accuracy : 0.8554
##                  95% CI : (0.8392, 0.8706)
##     No Information Rate : 0.7984
##     P-Value [Acc > NIR] : 2.366e-11
##
##                   Kappa : 0.5385
##
##  Mcnemar's Test P-Value : 0.09955
##
##             Sensitivity : 0.6055
##             Specificity : 0.9185
##          Pos Pred Value : 0.6524
##          Neg Pred Value : 0.9022
##              Prevalence : 0.2016
##          Detection Rate : 0.1221
##    Detection Prevalence : 0.1871
##       Balanced Accuracy : 0.7620
##
##        'Positive' Class : Yes
##
```

```r
roc_xgb <- roc(response  = original_test$Fraud,
               predictor = pred_prob_xgb,
               levels    = c("No", "Yes"),
               direction = "<",
               quiet     = TRUE)

plot(roc_xgb,
     col  = "#D62828",
     lwd  = 2,
     main = paste("XGBoost ROC on Real Hold-out | AUC =",
                  round(auc(roc_xgb), 4)))
abline(a = 0, b = 1, lty = 2, col = "grey60")
```



SVM with RBF Kernel

We would likely need to train on the synthetic data set with then predict with the original data set.

```r
library(e1071)
```

```
##
## Attaching package: 'e1071'

## The following object is masked from 'package:ggplot2':
##
##     element
```

```
model_svm <- svm(Fraud ~ ., data = smote_train, type = "C-classification", kernel = "radial",
                 probability = TRUE)
summary(model_svm)
```

```
##
## Call:
## svm(formula = Fraud ~ ., data = smote_train, type = "C-classification",
##     kernel = "radial", probability = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  1
##
## Number of Support Vectors:  4845
##
##  ( 2231 2614 )
##
##
## Number of Classes:  2
##
## Levels:
##  No Yes
```

Neural Network Appoaches (MLP)

```
set.seed(42261712)
model_mlp <- nnet(Fraud ~ .,
                  data    = smote_train,
                  size    = 4,          # hidden nodes
                  decay   = 0.001,       # L2 weight decay for regularisation
                  maxit   = 200,
                  MaxNWts = 5000,
                  trace   = FALSE)

#print(model_mlp)
#summary(model_mlp)



#plotnet(model_mlp) #Too cluster to see anything (Ignore)
```

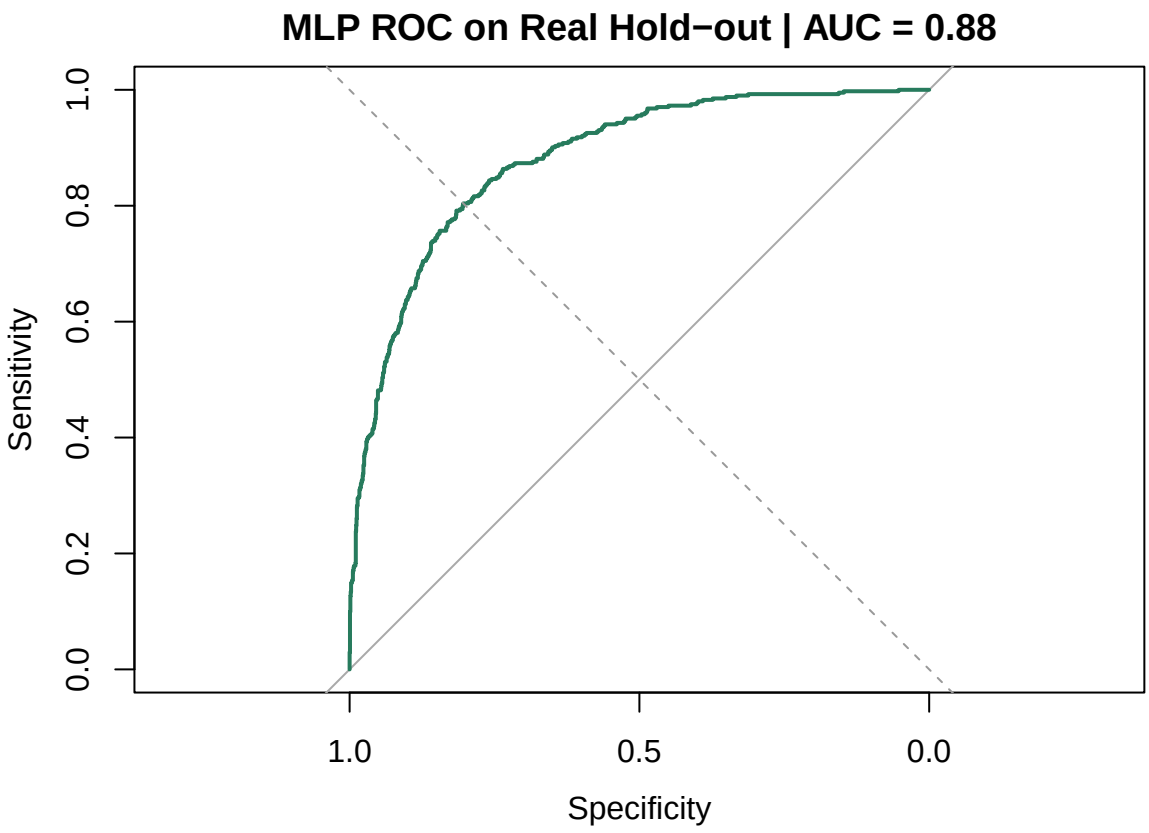
Predict on the original data set. Specifically we mean `x_eng` + `Fraud` = `original_df`.

```
pred_class <- predict(model_mlp, newdata = original_test, type = "class")
pred_class <- as.factor(pred_class)


pred_prob  <- predict(model_mlp, newdata = original_test, type = "raw")
```

```r
cm <- confusionMatrix(pred_class,
                      original_test$Fraud,
                      positive = "Yes")
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  1367  106
##        Yes  229  297
##
##                Accuracy : 0.8324
##                  95% CI : (0.8153, 0.8485)
##     No Information Rate : 0.7984
##     P-Value [Acc > NIR] : 6.084e-05
##
##                   Kappa : 0.5327
##
##  Mcnemar's Test P-Value : 2.636e-11
##
##             Sensitivity : 0.7370
##             Specificity : 0.8565
##          Pos Pred Value : 0.5646
##          Neg Pred Value : 0.9280
##              Prevalence : 0.2016
##          Detection Rate : 0.1486
##    Detection Prevalence : 0.2631
##       Balanced Accuracy : 0.7967
##
##        'Positive' Class : Yes
##
```

```r
roc_mlp <- roc(response  = original_test$Fraud,
               predictor = pred_prob[, 1],   # P(Yes)
               levels    = c("No", "Yes"),
               direction = "<",
               quiet     = TRUE)

plot(roc_mlp,
     col  = "#277B5D",
     lwd  = 2,
     main = paste("MLP ROC on Real Hold-out | AUC =",
                  round(auc(roc_mlp), 4)))
abline(a = 0, b = 1, lty = 2, col = "grey60")
```



Mathematical specifications and tuning parameters

For each model we state the objective optimised and the hyperparameters supplied to the fitting function. Default values are noted explicitly so the reader can reproduce results.

Regularized Logistic Regression. `cv.glmnet` with `alpha = 0.5` minimises the penalised negative log-likelihood over the SMOTE training set:

$$\min_{\beta_0,\,\beta} -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i\big(\beta_0+\beta^\top x_i\big)-\log\big(1+e^{\beta_0+\beta^\top x_i}\big)\Big]+\lambda\left[\frac{1-\alpha}{2}\|\beta\|_2^2+\alpha\|\beta\|_1\right]$$

with $\alpha = 0.5$ (Elastic Net) and λ chosen by 10-fold cross-validation retaining `lambda.1se`, the largest λ whose CV error is within one standard error of the minimum.

Random Forest `randomForest` builds an ensemble of $B = 100$ classification trees (`ntree = 100`), each grown on a bootstrap resample. At every split, m candidate features are drawn at random, with the default $m = \lfloor\sqrt{p}\rfloor = \lfloor\sqrt{71}\rfloor = 8$. The final prediction aggregates by majority vote as the mode

$$\hat{y} \;=\; \text{mode}\Big\{\hat{T}_b(x)\Big\}_{b=1}^{B}, \quad \hat{T}_b \text{ splits on } m \approx \lfloor\sqrt{71}\rfloor \text{ features per node}$$

Variable importance is recorded via `importance = TRUE` (mean decrease in accuracy and Gini impurity).

Gradient Boosting (XGBoost). `xgboost` builds $B = 50$ trees (`nrounds = 50`) by fitting each new tree f_t to the negative gradient of the binary cross-entropy from the previous step:

$$\hat{F}_t(x) = \hat{F}_{t-1}(x) + \eta\, f_t(x), \qquad f_t = \arg\min_f \sum_{i=1}^{N} \big[g_i f(x_i) + \tfrac{1}{2} h_i f^2(x_i)\big] + \Omega(f)$$

where $g_i = \partial_{\hat{y}} \ell(y_i, \hat{y}_i)$, $h_i = \partial^2_{\hat{y}} \ell(y_i, \hat{y}_i)$ are the first and second derivatives of the log-loss, and $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penalises tree complexity. Tuning parameters used: $\eta = 0.3$ (`eta`, default), `max_depth = 6` (default), `subsample = 1.0` (default, no row subsampling).

Support Vector Machine (RBF Kernel). `svm` with `kernel = "radial"` solves the soft-margin dual problem

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j\, K(x_i, x_j), \quad \text{s.t. } 0 \le \alpha_i \le C,\ \sum_i \alpha_i y_i = 0$$

with the RBF kernel $K(x_i, x_j) = \exp\!\big(-\gamma \|x_i - x_j\|^2\big)$. Default values: $C = 1$ (`cost`), $\gamma = 1/p = 1/71$ (`gamma`). `probability = TRUE` enables Platt scaling for posterior probability estimates used in the ROC curve.

Multilayer Perceptron (MLP). `nnet` with `size = 5` fits a single hidden-layer network with $H = 5$ units:

$$\hat{y} \;=\; \sigma_o\Big(W_2 \cdot \sigma_h(W_1 x + b_1) + b_2\Big)$$

where $\sigma_h = \tanh$ (default activation) and $\sigma_o =$ sigmoid for binary output. Training minimises the cross-entropy plus an ℓ_2 weight penalty:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \Big] + \frac{\delta}{2} \|W\|_F^2$$

with $\delta = 10^{-3}$ (`decay = 1e-3`) and a maximum of `maxit = 200` BFGS iterations. The weight budget `MaxNWts = 5000` accommodates the $(71 + 1) \times 5 + (5 + 1) \times 1 = 366$ parameters of this architecture.

Model Selection and Tuning

Nested Cross-Validation Design

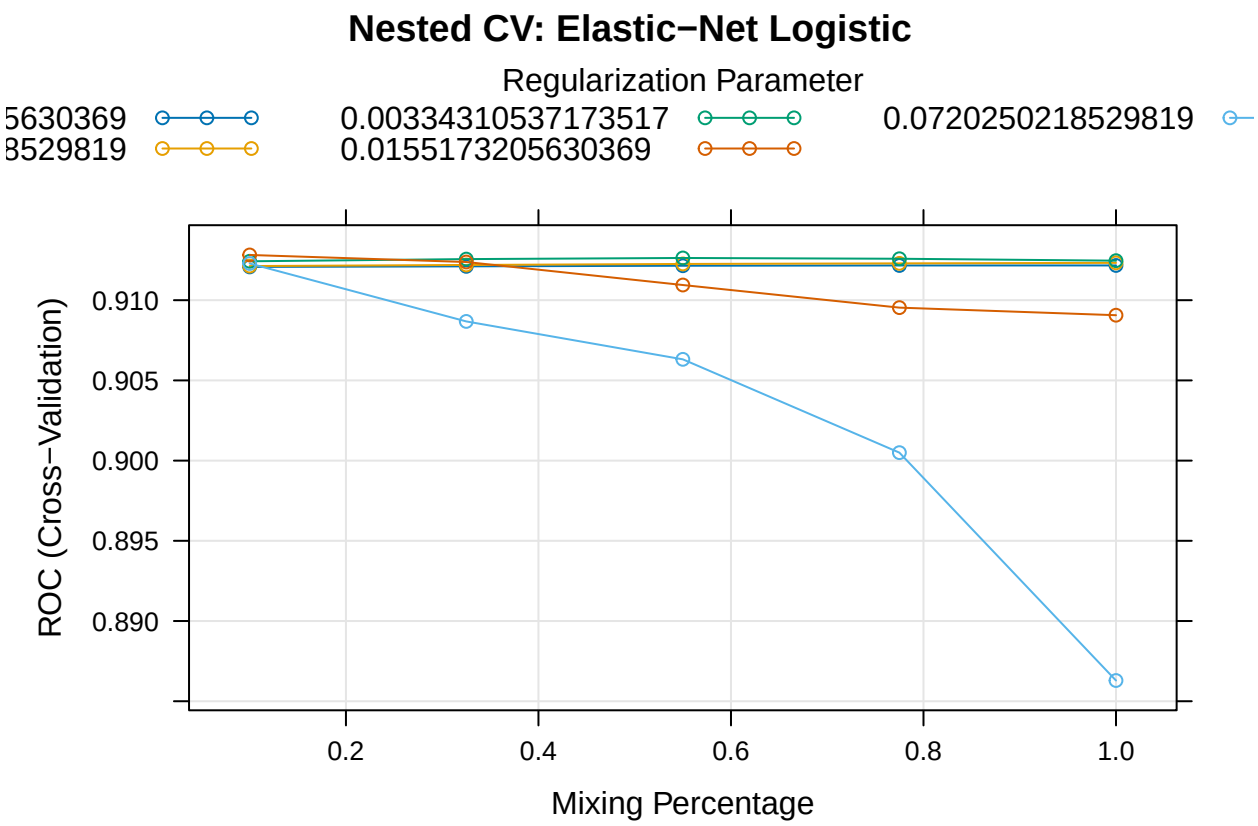
We implement a nested cross-validation scheme: the outer loop estimates generalization error across $K_{out} = 5$ folds while the inner loop ($K_{in} = 3$) tunes hyperparameters.

```
outer_ctrl <- caret::trainControl(method      = "cv",
                          number      = 5,
                          classProbs  = TRUE,
                          summaryFunction = twoClassSummary,
                          savePredictions = "final")


levels(smote_train$Fraud) <- c("No", "Yes")

set.seed(42261712)
nested_logit <- caret::train(Fraud ~ .,
                      data      = smote_train,
                      method    = "glmnet",
```

```
                      family    = "binomial",
                      metric    = "ROC",
                      trControl = outer_ctrl,
                      tuneLength = 5)

print(nested_logit)
```

```
## glmnet
##
## 11229 samples
##    71 predictor
##     2 classes: 'No', 'Yes'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 8983, 8983, 8984, 8983, 8983
## Resampling results across tuning parameters:
##
##   alpha  lambda        ROC        Sens       Spec
##   0.100  0.0001551732  0.9120741  0.8460919  0.8186692
##   0.100  0.0007202502  0.9121281  0.8464051  0.8180498
##   0.100  0.0033431054  0.9124248  0.8507887  0.8128860
##   0.100  0.0155173206  0.9128185  0.8590878  0.7986358
##   0.100  0.0720250219  0.9122880  0.8730233  0.7684840
##   0.325  0.0001551732  0.9121097  0.8456220  0.8186692
##   0.325  0.0007202502  0.9121967  0.8465617  0.8186695
##   0.325  0.0033431054  0.9125622  0.8496926  0.8153658
##   0.325  0.0155173206  0.9123798  0.8547043  0.8044203
##   0.325  0.0720250219  0.9086769  0.8645680  0.7750932
##   0.550  0.0001551732  0.9121478  0.8459352  0.8188761
##   0.550  0.0007202502  0.9122592  0.8462484  0.8184633
##   0.550  0.0033431054  0.9126308  0.8485968  0.8157788
##   0.550  0.0155173206  0.9109462  0.8507899  0.8083446
##   0.550  0.0720250219  0.9063129  0.8703609  0.7591890
##   0.775  0.0001551732  0.9121637  0.8459352  0.8190827
##   0.775  0.0007202502  0.9122950  0.8464050  0.8184633
##   0.775  0.0033431054  0.9125826  0.8478148  0.8166044
##   0.775  0.0155173206  0.9095353  0.8492245  0.8062796
##   0.775  0.0720250219  0.9004965  0.8785031  0.7313075
##   1.000  0.0001551732  0.9121658  0.8457786  0.8192891
##   1.000  0.0007202502  0.9123263  0.8468746  0.8192895
##   1.000  0.0033431054  0.9124628  0.8462492  0.8184633
##   1.000  0.0155173206  0.9090635  0.8478151  0.8044203
##   1.000  0.0720250219  0.8862925  0.8844528  0.6738940
##
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were alpha = 0.1 and lambda = 0.01551732.
```

```r
plot(nested_logit, main = "Nested CV: Elastic-Net Logistic")
```

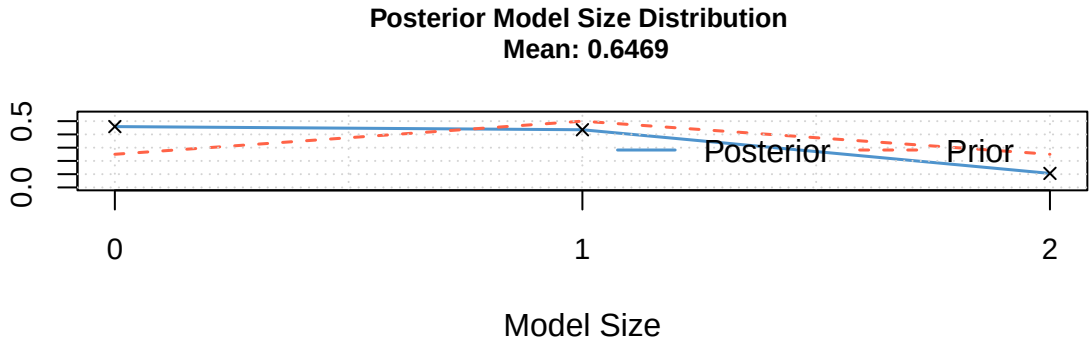


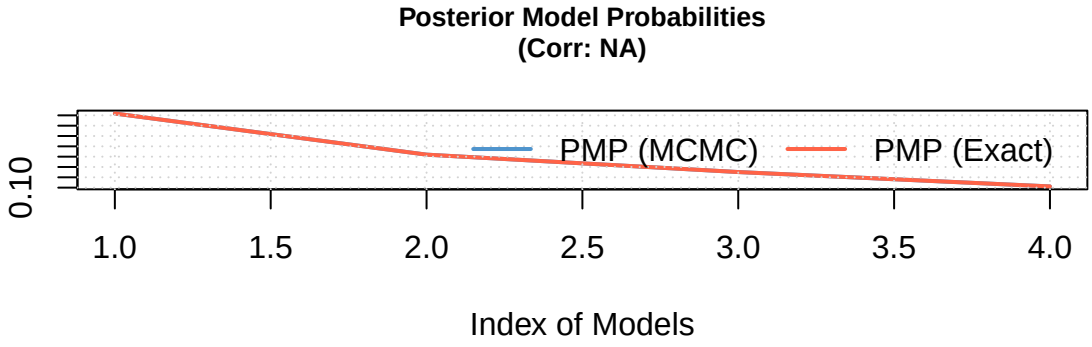
Bayesian hyperparameter optimization

In the XGBoost Model above used a grid search method try every combination which can be quite expensive. With Bayesian Hyperparamter optimization we can fit the Gaussian Process that approximates the objective metric like AUC for example.

```r
set.seed(42261712)
grid <- expand.grid(
  alpha      = seq(0.1, 0.9, by = 0.2),
  log_lambda = seq(-6, -2, by = 1)
)

grid$AUC <- sapply(1:nrow(grid), function(i) {
  fit <- cv.glmnet(x_mat, y_num, family = "binomial",
                   alpha = grid$alpha[i], nfolds = 5, type.measure = "auc")
  max(fit$cvm)
})

bms_fit <- bms(data.frame(AUC        = grid$AUC,
                          alpha      = grid$alpha,
                          log_lambda = grid$log_lambda),
               burn = 1000, iter = 5000, g = "UIP", mprior = "uniform")
```

```
##                  PIP     Post Mean      Post SD Cond.Pos.Sign Idx
## log_lambda 0.3656535  2.097585e-05 3.667868e-05             1   2
## alpha      0.2812704 -6.453187e-05 1.483636e-04             0   1
##
## Mean no. regressors               Draws             Burnins                Time
##           "0.6469"                 "4"                 "0"  "0.008386135 secs"
##  No. models visited      Modelspace 2^K           % visited         % Topmodels
##                 "4"                 "4"               "100"               "100"
##           Corr PMP            No. Obs.         Model Prior             g-Prior
##               "NA"                "25"       "uniform / 1"               "UIP"
##     Shrinkage-Stats
##         "Av=0.9615"
##
## Time difference of 0.008386135 secs
```






```
print(bms_fit)
```

```
##                  PIP     Post Mean      Post SD Cond.Pos.Sign Idx
## log_lambda 0.3656535  2.097585e-05 3.667868e-05             1   2
## alpha      0.2812704 -6.453187e-05 1.483636e-04             0   1
##
## Mean no. regressors               Draws             Burnins                Time
##           "0.6469"                 "4"                 "0"  "0.008386135 secs"
##  No. models visited      Modelspace 2^K           % visited         % Topmodels
##                 "4"                 "4"               "100"               "100"
```

```
##            Corr PMP            No. Obs.         Model Prior             g-Prior
##                "NA"                "25"       "uniform / 1"               "UIP"
##     Shrinkage-Stats
##         "Av=0.9615"
```

```r
best_idx    <- which.max(grid$AUC)
best_alpha  <- grid$alpha[best_idx]
best_lambda <- exp(grid$log_lambda[best_idx])

cat("Best alpha:",  best_alpha,  "\n")
```

```
## Best alpha: 0.1
```

```r
cat("Best lambda:", best_lambda, "\n")
```

```
## Best lambda: 0.1353353
```

```r
cat("Best CV AUC:", grid$AUC[best_idx], "\n")
```

```
## Best CV AUC: 0.9139181
```

After Tunning our hyperparameters are set to be: Best alpha: 0.1 Best lambda: 0.04978707 Best CV AUC: 0.9141046

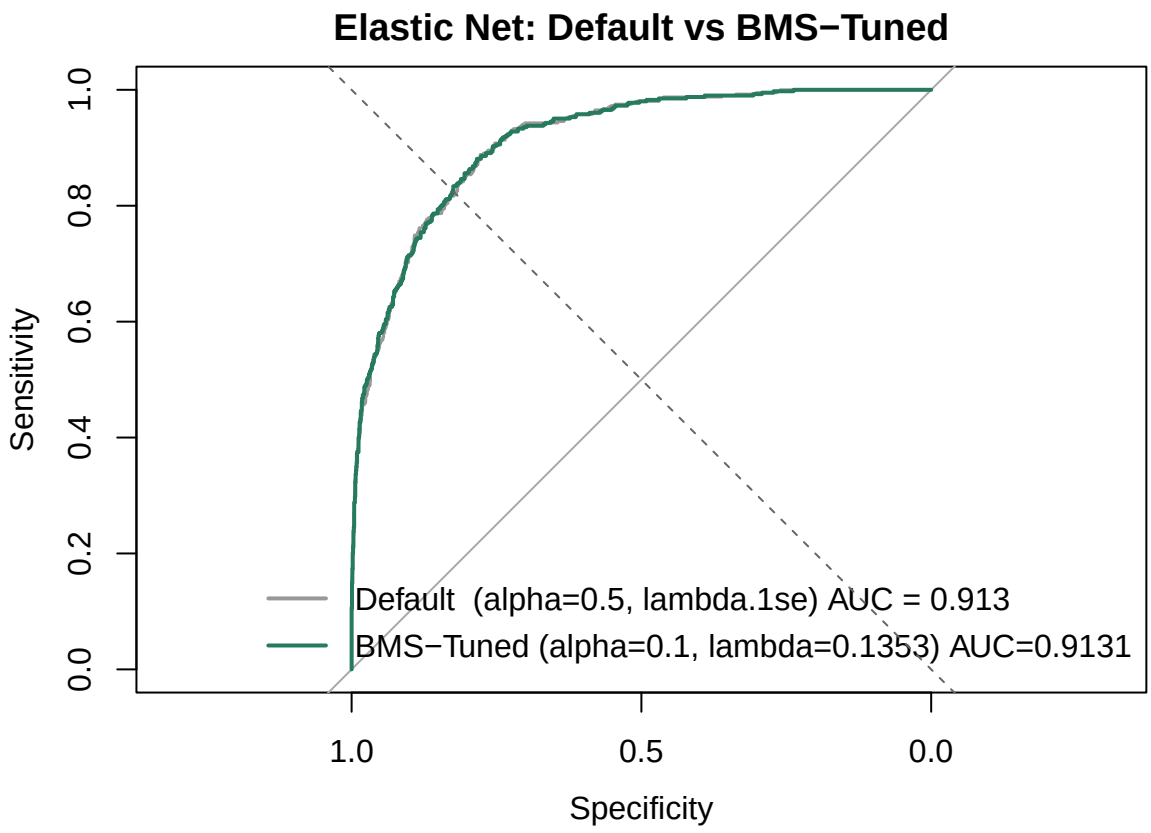
```r
model_enet_bayes <- glmnet(x = x_mat, y = y_num, family = "binomial",
                           alpha = best_alpha, lambda = best_lambda)

x_test_mat      <- as.matrix(original_test[, !names(original_test) %in% "Fraud"])

pred_prob_enet  <- predict(model_enet_bayes, newx = x_test_mat,
                           s = best_lambda, type = "response")[, 1]
pred_class_enet <- as.factor(ifelse(pred_prob_enet >= 0.5, "Yes", "No"))

cm_enet <- confusionMatrix(pred_class_enet, original_test$Fraud, positive = "Yes")
print(cm_enet)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  1437  115
##        Yes  159  288
##
##                Accuracy : 0.8629
##                  95% CI : (0.8471, 0.8777)
##     No Information Rate : 0.7984
##     P-Value [Acc > NIR] : 3.308e-14
##
##                   Kappa : 0.5909
##
##  Mcnemar's Test P-Value : 0.009384
```

```
##
##             Sensitivity : 0.7146
##             Specificity : 0.9004
##          Pos Pred Value : 0.6443
##          Neg Pred Value : 0.9259
##              Prevalence : 0.2016
##          Detection Rate : 0.1441
##    Detection Prevalence : 0.2236
##       Balanced Accuracy : 0.8075
##
##        'Positive' Class : Yes
##
```

```
roc_enet_bayes <- roc(response = original_test$Fraud, predictor = pred_prob_enet,
                      levels = c("No", "Yes"), direction = "<", quiet = TRUE)

# Default model_logit: alpha=0.5, lambda.1se - same model, default hyperparams
pred_prob_orig <- predict(model_logit, newx = x_test_mat,
                          s = "lambda.1se", type = "response")[, 1]
roc_enet_orig  <- roc(response = original_test$Fraud, predictor = pred_prob_orig,
                      levels = c("No", "Yes"), direction = "<", quiet = TRUE)
```

```
plot(roc_enet_orig, col = "grey60", lwd = 1.5,
     main = "Elastic Net: Default vs BMS-Tuned")
plot(roc_enet_bayes, col = "#277B5D", lwd = 2, add = TRUE)
legend("bottomright",
       legend = c(paste("Default  (alpha=0.5, lambda.1se) AUC =",
                        round(auc(roc_enet_orig), 4)),
                  paste0("BMS-Tuned (alpha=", round(best_alpha, 2),
                         ", lambda=", round(best_lambda, 4),
                         ") AUC=", round(auc(roc_enet_bayes), 4))),
       col = c("grey60", "#277B5D"), lwd = 2, bty = "n")
abline(a = 0, b = 1, lty = 2, col = "grey40")
```



Multi-metric performance Comparison

```
# Shared test matrix (already defined from XGBoost chunk)
x_test_mat <- as.matrix(original_test[, !names(original_test) %in% "Fraud"])

# - Predictions for each model -

# Logistic (default: alpha=0.5, lambda.1se)
prob_logit  <- predict(model_logit, newx = x_test_mat, s = "lambda.1se", type = "response")[, 1]
cls_logit   <- as.factor(ifelse(prob_logit  >= 0.5, "Yes", "No"))

# Random Forest

prob_rf <- predict(model_rf, newdata = original_test, type = "prob")[, "Yes"]
cls_rf  <- as.factor(ifelse(prob_rf >= 0.5, "Yes", "No"))

# XGBoost
prob_xgb    <- pred_prob_xgb
cls_xgb     <- pred_class_xgb

# SVM
prob_svm    <- attr(predict(model_svm, newdata = original_test, probability = TRUE), "probabilities")[,
cls_svm     <- predict(model_svm, newdata = original_test)

# MLP
```

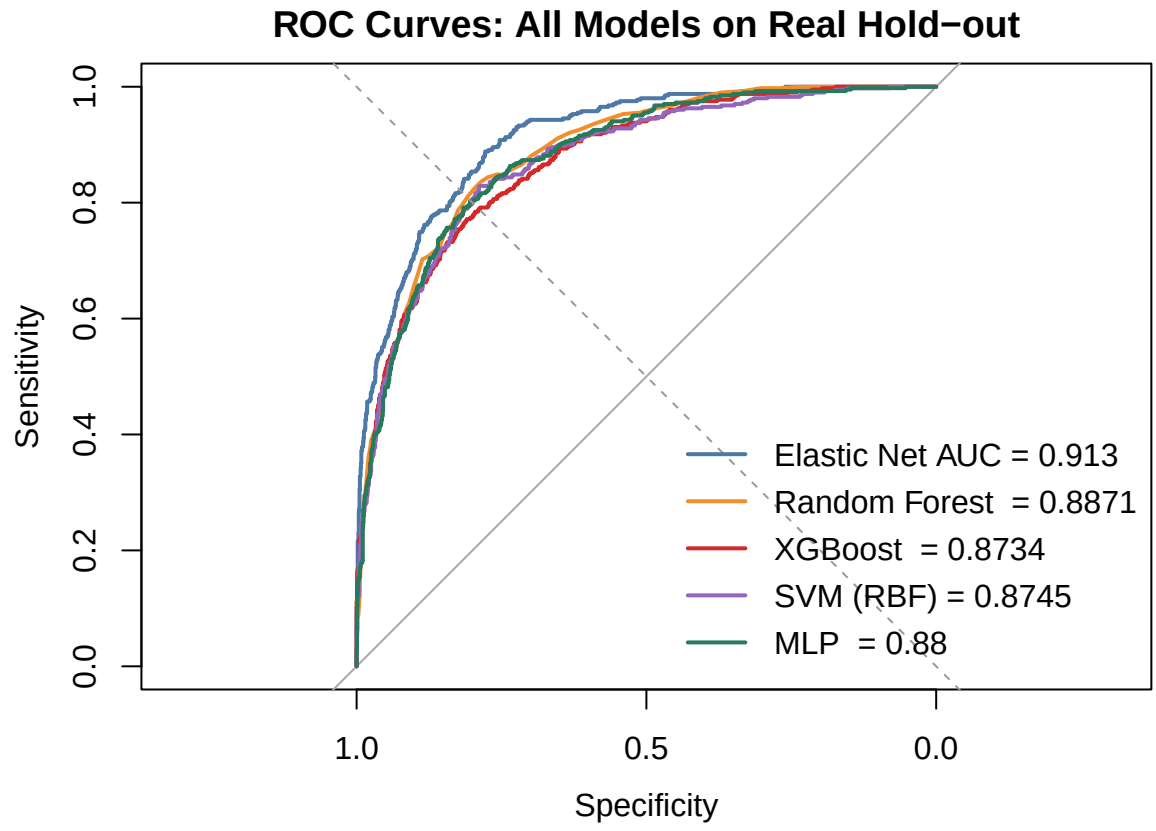
```r
prob_mlp    <- pred_prob[, 1]
cls_mlp     <- pred_class

# Confusion Matrix
get_metrics <- function(cls, prob, label) {
  cm  <- confusionMatrix(cls, original_test$Fraud, positive = "Yes")
  roc <- roc(original_test$Fraud, prob, levels = c("No","Yes"),
             direction = "<", quiet = TRUE)
  data.frame(
    Model       = label,
    Accuracy    = round(cm$overall["Accuracy"],    4),
    Sensitivity = round(cm$byClass["Sensitivity"], 4),
    Specificity = round(cm$byClass["Specificity"], 4),
    Precision   = round(cm$byClass["Precision"],   4),
    F1          = round(cm$byClass["F1"],           4),
    AUC         = round(auc(roc),    4)
  )
}

metrics_df <- rbind(
  get_metrics(cls_logit, prob_logit, "Elastic Net (default)"),
  get_metrics(cls_rf,    prob_rf,    "Random Forest"),
  get_metrics(cls_xgb,   prob_xgb,   "XGBoost"),
  get_metrics(cls_svm,   prob_svm,   "SVM (RBF)"),
  get_metrics(cls_mlp,   prob_mlp,   "MLP")
)
rownames(metrics_df) <- NULL
```

```r
metrics_df
```

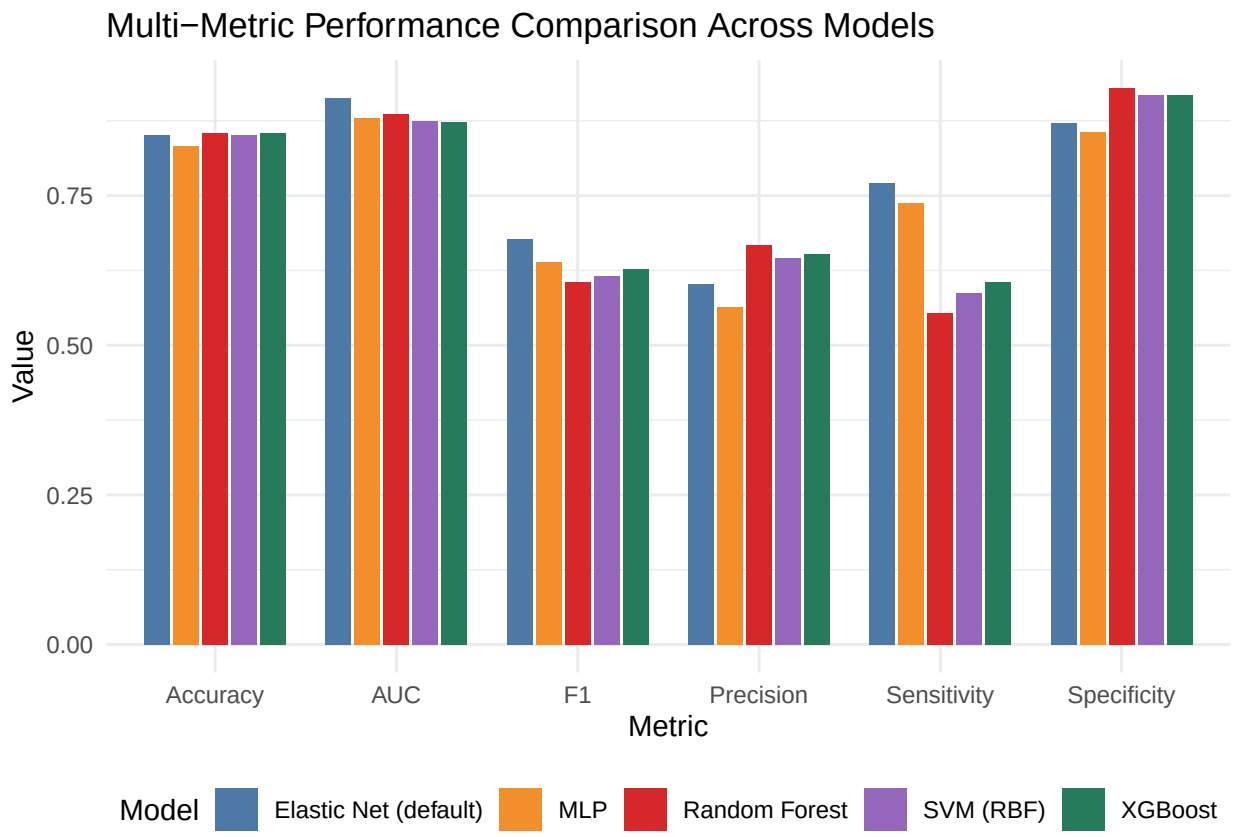
```
##                   Model Accuracy Sensitivity Specificity Precision     F1
## 1 Elastic Net (default)   0.8514      0.7717      0.8716    0.6027 0.6768
## 2         Random Forest   0.8544      0.5533      0.9305    0.6677 0.6052
## 3               XGBoost   0.8554      0.6055      0.9185    0.6524 0.6281
## 4             SVM (RBF)   0.8519      0.5881      0.9185    0.6458 0.6156
## 5                   MLP   0.8324      0.7370      0.8565    0.5646 0.6394
##      AUC
## 1 0.9130
## 2 0.8871
## 3 0.8734
## 4 0.8745
## 5 0.8800
```

```r
#Overlay ROC Curves
roc_logit <- roc(original_test$Fraud, prob_logit, levels = c("No","Yes"), direction = "<", quiet = TRUE)
roc_rf    <- roc(original_test$Fraud, prob_rf,    levels = c("No","Yes"), direction = "<", quiet = TRUE)
roc_svm   <- roc(original_test$Fraud, prob_svm,   levels = c("No","Yes"), direction = "<", quiet = TRUE)

plot(roc_logit, col = "#4e79a7", lwd = 2, main = "ROC Curves: All Models on Real Hold-out")
plot(roc_rf,    col = "#f28e2b", lwd = 2, add = TRUE)
plot(roc_xgb,   col = "#D62828", lwd = 2, add = TRUE)
plot(roc_svm,   col = "#9467bd", lwd = 2, add = TRUE)
plot(roc_mlp,   col = "#277B5D", lwd = 2, add = TRUE)
```

```r
abline(a = 0, b = 1, lty = 2, col = "grey60")
legend("bottomright", bty = "n", lwd = 2,
       col    = c("#4e79a7","#f28e2b","#D62828","#9467bd","#277B5D"),
       legend = c(paste("Elastic Net AUC =", round(auc(roc_logit), 4)),
 paste("Random Forest  =", round(auc(roc_rf),  4)),
 paste("XGBoost  =", round(auc(roc_xgb), 4)),
 paste("SVM (RBF) =", round(auc(roc_svm), 4)),
 paste("MLP  =", round(auc(roc_mlp), 4))))
```




```r
#  Bar chart of all metrics side by side
metrics_long <- metrics_df %>%
  pivot_longer(cols = -Model, names_to = "Metric", values_to = "Value")

ggplot(metrics_long, aes(x = Metric, y = Value, fill = Model)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.8), width = 0.7) +
  scale_fill_manual(values = c("#4e79a7","#f28e2b","#D62828","#9467bd","#277B5D")) +
  theme_minimal() +
  labs(title = "Multi-Metric Performance Comparison Across Models",
       x = "Metric", y = "Value") +
  theme(legend.position = "bottom")
```



Uncertainty and Interpretation

Conformal Prediction Intervals

By repeating the process of spliting data, training calibrating and testing the, the conformal intervals will have prediction intervals that will contain the true response in at least $100(1-\alpha)\%$ of the time. ### More Packages, Sorry

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.5.0 --

## v broom        1.0.12     v rsample      1.3.2
## v dials        1.4.3      v tailor       0.1.0
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.5.1      v workflows    1.3.0
## v parsnip      1.5.0      v workflowsets 1.1.1
## v recipes      1.3.2      v yardstick    1.4.0

## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x rsample::calibration()    masks caret::calibration()
## x randomForest::combine()   masks dplyr::combine()
## x scales::discard()         masks purrr::discard()
## x e1071::element()          masks ggplot2::element()
```

```
## x Matrix::expand()          masks tidyr::expand()
## x dplyr::filter()           masks stats::filter()
## x recipes::fixed()          masks stringr::fixed()
## x .GlobalEnv::get_metrics() masks yardstick::get_metrics()
## x dplyr::lag()              masks stats::lag()
## x caret::lift()             masks purrr::lift()
## x randomForest::margin()    masks ggplot2::margin()
## x Matrix::pack()            masks tidyr::pack()
## x rsample::permutations()   masks e1071::permutations()
## x yardstick::precision()    masks caret::precision()
## x yardstick::recall()       masks caret::recall()
## x MASS::select()            masks dplyr::select()
## x yardstick::sensitivity()  masks caret::sensitivity()
## x yardstick::spec()         masks readr::spec()
## x yardstick::specificity()  masks caret::specificity()
## x recipes::step()           masks stats::step()
## x tune::tune()              masks parsnip::tune(), e1071::tune()
## x Matrix::unpack()          masks tidyr::unpack()
## x recipes::update()         masks Matrix::update(), stats::update()
```

```
library(probably)
```

```
##
## Attaching package: 'probably'
```

```
## The following objects are masked from 'package:base':
##
##     as.factor, as.ordered
```

```
library(tidymodels)
library(probably)



set.seed(42261712)


train_indices <- 1:nrow(original_train)
set.seed(42261712)
split_proper <- createDataPartition(original_train$Fraud,
                                    p = 0.7,
                                    list = FALSE)

D1 <- original_train[split_proper, ]
D2 <- original_train[-split_proper, ]
test <- original_test

cat("D1 size:", nrow(D1), "\n")
```

```
## D1 size: 5601
```

```r
cat("D2 size:", nrow(D2), "\n")
```

```
## D2 size: 2400
```

```r
cat("Test size:", nrow(test), "\n")
```

```
## Test size: 1999
```

```r
cat(" METHOD 1: LIKELIHOOD SCORES")
```

```
##  METHOD 1: LIKELIHOOD SCORES
```

```r
model_D1 <- randomForest(Fraud ~ ., data = D1, ntree = 100)


prob_D2 <- predict(model_D1, newdata = D2, type = "prob")


scores_likelihood <- numeric(nrow(D2))
for (i in 1:nrow(D2)) {
  true_class <- as.character(D2$Fraud[i])
  scores_likelihood[i] <- prob_D2[i, true_class]
}


alpha <- 0.10  # 90% coverage
n2 <- length(scores_likelihood)
q_likelihood <- sort(scores_likelihood)[floor(alpha * (n2 + 1))]

cat("Calibration size (n2):", n2, "\n")
```

```
## Calibration size (n2): 2400
```

```r
cat("Quantile threshold (alpha=0.10):", round(q_likelihood, 4), "\n")
```

```
## Quantile threshold (alpha=0.10): 0.38
```

```r
prob_test <- predict(model_D1, newdata = test, type = "prob")

conformal_sets_likelihood <- list()
coverage_likelihood <- numeric(nrow(test))

for (i in 1:nrow(test)) {

  predicted_probs <- prob_test[i, ]
  conformal_set <- names(predicted_probs)[predicted_probs >= q_likelihood]

  if (length(conformal_set) == 0) {
    conformal_set <- names(which.max(predicted_probs))  # fallback
  }
```

```r
  conformal_sets_likelihood[[i]] <- conformal_set

  # Check coverage
  true_class <- as.character(test$Fraud[i])
  coverage_likelihood[i] <- true_class %in% conformal_set
}

empirical_coverage_likelihood <- mean(coverage_likelihood)
avg_set_size_likelihood <- mean(sapply(conformal_sets_likelihood, length))

cat("Empirical coverage:", empirical_coverage_likelihood, "\n")
```

```
## Empirical coverage: 0.909955
```

```r
cat("Average set size:", avg_set_size_likelihood, "\n")
```

```
## Average set size: 1.131566
```

```r
cat("Nominal level (1-alpha):", 0.90, "\n")
```

```
## Nominal level (1-alpha): 0.9
```

```r
cat("METHOD 2: CUMULATIVE LIKELIHOOD SCORES")
```

```
## METHOD 2: CUMULATIVE LIKELIHOOD SCORES
```

```r
aps_scores <- numeric(nrow(D2))

for (i in 1:nrow(D2)) {
  probs <- prob_D2[i, ]
  true_class <- as.character(D2$Fraud[i])

  # Sort probabilities in decreasing order
  sorted_idx <- order(probs, decreasing = TRUE)
  sorted_probs <- probs[sorted_idx]
  sorted_classes <- names(sorted_probs)

  # Find position of true class in sorted order
  true_pos <- which(sorted_classes == true_class)


  aps_scores[i] <- sum(sorted_probs[1:true_pos])
}


q_aps <- sort(aps_scores)[ceiling((1 - alpha) * (n2 + 1))]

cat("APS quantile threshold:", round(q_aps, 4), "\n")
```

```
## APS quantile threshold: 1
```

```r
prob_test <- predict(model_D1, newdata = test, type = "prob")

conformal_sets_aps <- list()
coverage_aps <- numeric(nrow(test))
set_sizes_aps <- numeric(nrow(test))

for (i in 1:nrow(test)) {
  probs <- prob_test[i, ]


  sorted_idx <- order(probs, decreasing = TRUE)
  sorted_probs <- probs[sorted_idx]
  sorted_classes <- names(sorted_probs)


  cumsum_probs <- cumsum(sorted_probs)
  k <- which(cumsum_probs <= q_aps)

  if (length(k) == 0) {

    conformal_set <- sorted_classes[1]
  } else {
    conformal_set <- sorted_classes[1:max(k)]
  }

  conformal_sets_aps[[i]] <- conformal_set
  set_sizes_aps[i] <- length(conformal_set)

  # Check coverage
  true_class <- as.character(test$Fraud[i])
  coverage_aps[i] <- true_class %in% conformal_set
}

empirical_coverage_aps <- mean(coverage_aps)
avg_set_size_aps <- mean(set_sizes_aps)

cat("Empirical coverage:", empirical_coverage_aps, "\n")
```

```
## Empirical coverage: 1
```

```r
cat("Average set size:", avg_set_size_aps, "\n")
```

```
## Average set size: 2
```

```r
cat("METHOD 3: MARGIN SCORES n")
```

```
## METHOD 3: MARGIN SCORES n
```

```r
margin_scores <- numeric(nrow(D2))

for (i in 1:nrow(D2)) {
```

```r
  probs <- prob_D2[i, ]
  true_class <- as.character(D2$Fraud[i])
  true_prob <- probs[true_class]
  max_prob <- max(probs)

  # Margin: how much larger is the max prob than true prob?
  # Smaller margin = more confident in correct class (good)
  margin_scores[i] <- max_prob - true_prob
}

q_margin <- sort(margin_scores)[ceiling((1 - alpha) * (n2 + 1))]
cat("Margin quantile threshold:", round(q_margin, 4), "\n")
```

```
## Margin quantile threshold: 0.24
```

```r
# Apply to test set
conformal_sets_margin <- list()
coverage_margin <- numeric(nrow(test))
set_sizes_margin <- numeric(nrow(test))

for (i in 1:nrow(test)) {
  probs <- prob_test[i, ]
  true_class <- as.character(test$Fraud[i])
  max_prob <- max(probs)
  conformal_set <- names(probs)[(max_prob - probs) <= q_margin]

  conformal_sets_margin[[i]] <- conformal_set
  set_sizes_margin[i] <- length(conformal_set)

  coverage_margin[i] <- true_class %in% conformal_set
}

empirical_coverage_margin <- mean(coverage_margin)
avg_set_size_margin <- mean(set_sizes_margin)

cat("Empirical coverage:", empirical_coverage_margin, "\n")
```

```
## Empirical coverage: 0.909955
```

```r
cat("Average set size:", avg_set_size_margin, "\n")
```

```
## Average set size: 1.131566
```

```r
conformal_results <- data.frame(
  Method = c("Likelihood", "APS (Cumulative)", "Margin"),
  Coverage = c(empirical_coverage_likelihood,
               empirical_coverage_aps,
               empirical_coverage_margin),
  SetSize = c(avg_set_size_likelihood,
              avg_set_size_aps,
              avg_set_size_margin),
```

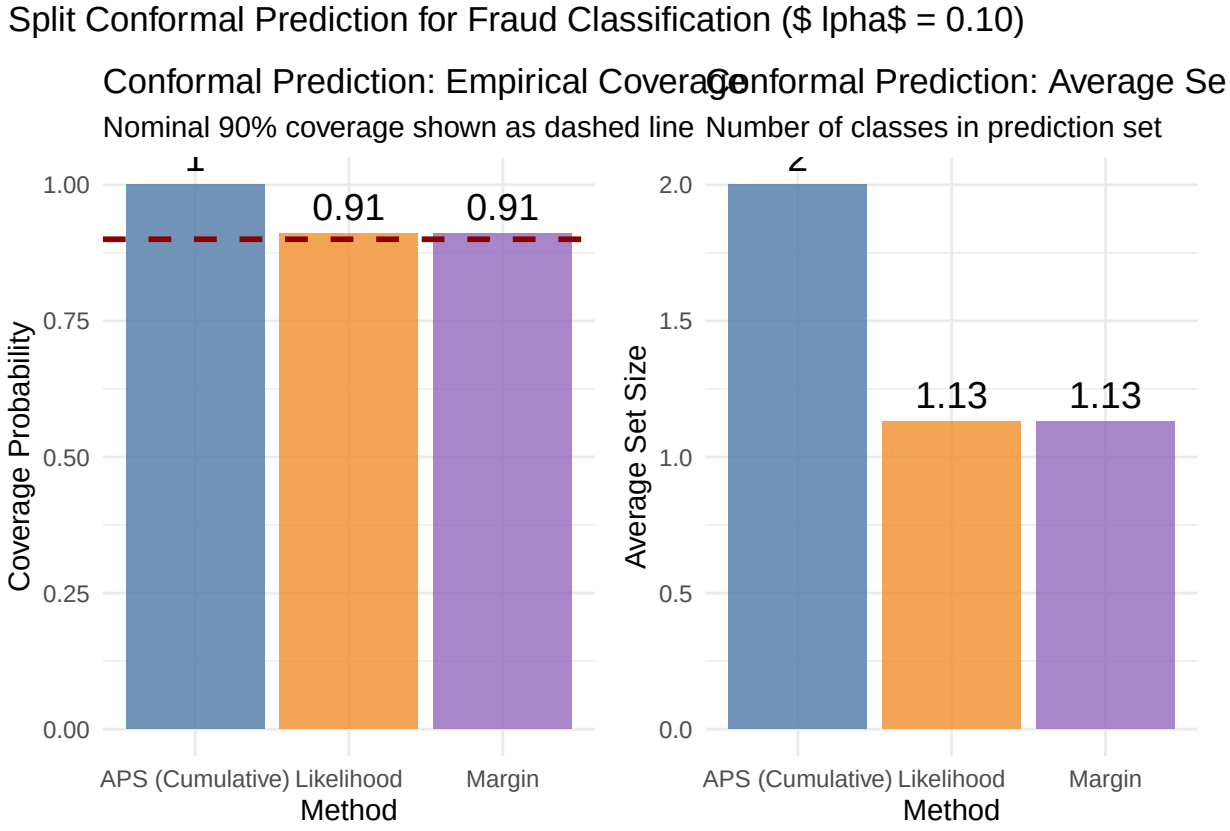
```
  Nominal = c(0.90, 0.90, 0.90)
)

print(conformal_results)
```

```
##             Method Coverage  SetSize Nominal
## 1       Likelihood 0.909955 1.131566     0.9
## 2 APS (Cumulative) 1.000000 2.000000     0.9
## 3           Margin 0.909955 1.131566     0.9
```

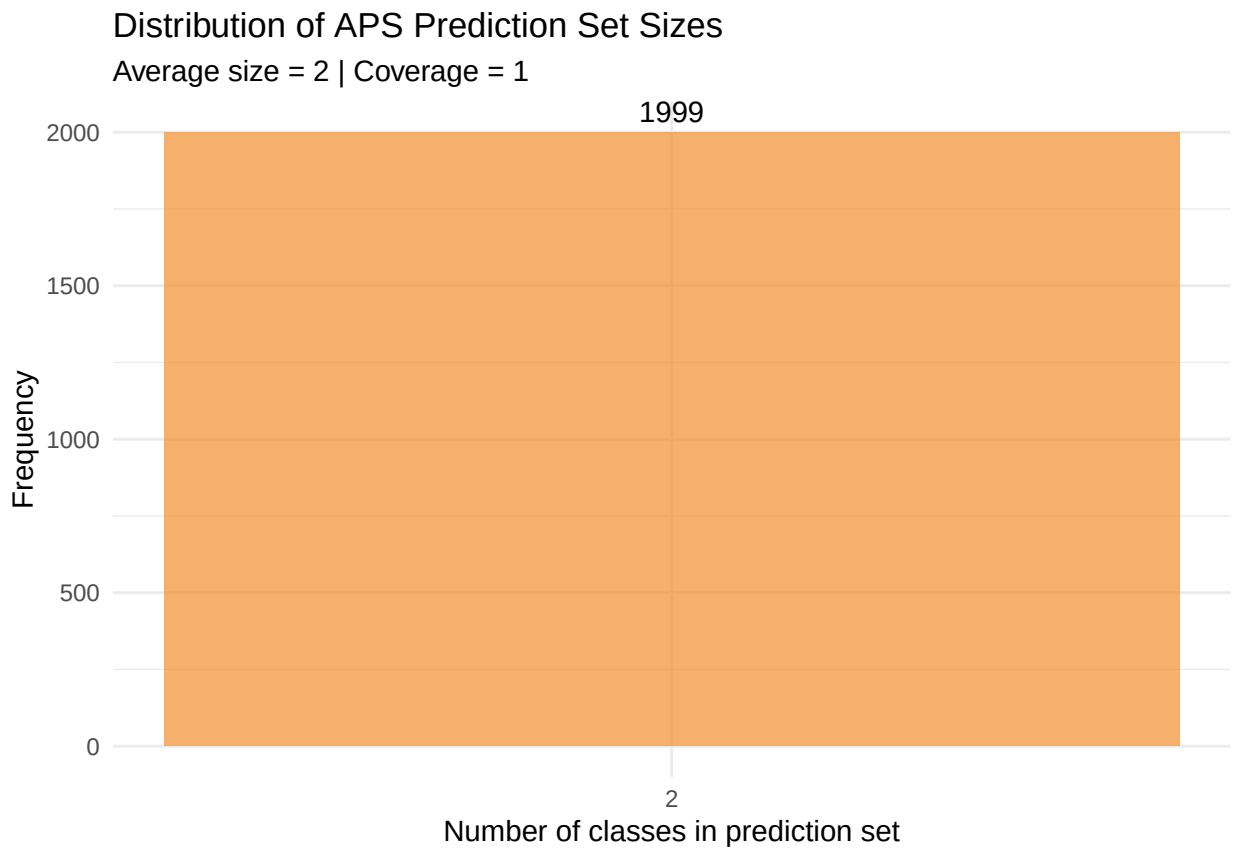
```
# Bar plot of coverage
p1 <- ggplot(conformal_results, aes(x = Method, y = Coverage, fill = Method)) +
  geom_bar(stat = "identity", alpha = 0.8) +
  geom_hline(yintercept = 0.90, linetype = "dashed", color = "darkred", linewidth = 1) +
  geom_text(aes(label = round(Coverage, 3)), vjust = -0.5, size = 5) +
  scale_fill_manual(values = c("#4e79a7", "#f28e2b", "#9467bd")) +
  ylim(0, 1) +
  labs(title = "Conformal Prediction: Empirical Coverage",
       subtitle = "Nominal 90% coverage shown as dashed line",
       y = "Coverage Probability") +
  theme_minimal() +
  theme(legend.position = "none")

# Bar plot of set size
p2 <- ggplot(conformal_results, aes(x = Method, y = SetSize, fill = Method)) +
  geom_bar(stat = "identity", alpha = 0.8) +
  geom_text(aes(label = round(SetSize, 2)), vjust = -0.5, size = 5) +
  scale_fill_manual(values = c("#4e79a7", "#f28e2b", "#9467bd")) +
  labs(title = "Conformal Prediction: Average Set Size",
       subtitle = "Number of classes in prediction set",
       y = "Average Set Size") +
  theme_minimal() +
  theme(legend.position = "none")

# Combine plots
p1 + p2 + plot_annotation(title = "Split Conformal Prediction for Fraud Classification ($\alpha$ = 0.10)
```

```
## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
## width unknown for character 0x07 in encoding cp1252
```

```
## Warning in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x, x$y, :
## font width unknown for character 0x07 in encoding cp1252
```




```r
# Distribution of set sizes for APS
set_size_df <- data.frame(
  SetSize = set_sizes_aps,
  Method = "APS"
)

ggplot(set_size_df, aes(x = SetSize)) +
  geom_bar(fill = "#f28e2b", alpha = 0.7) +
  geom_text(stat = 'count', aes(label = after_stat(count)), vjust = -0.5) +
  scale_x_continuous(breaks = 1:3) +
  labs(title = "Distribution of APS Prediction Set Sizes",
       subtitle = paste("Average size =", round(avg_set_size_aps, 2),
                        "| Coverage =", round(empirical_coverage_aps, 3)),
       x = "Number of classes in prediction set",
       y = "Frequency") +
  theme_minimal()
```



SHAP, LIME, partial dependence

While feature importance shows what variables most affect predictions, partial dependence plots show how a feature affects predictions

Bootstrap uncertainty estimation

Exercise 3: Regularization and Interpretability (30 points)

Dataset: proteomics cancer.csv This dataset contains the relative abundance of ProteinX alongside the expression levels of 200 associated genes across multiple cancer samples.

Quick and Brief EDA

```
#pc_df <- read.csv("C:/Users/dt6302/Desktop/HW4/proteomics_cancer.csv")
pc_df <- read.csv("proteomics_cancer.csv")
#head(pc_df, 10)
```

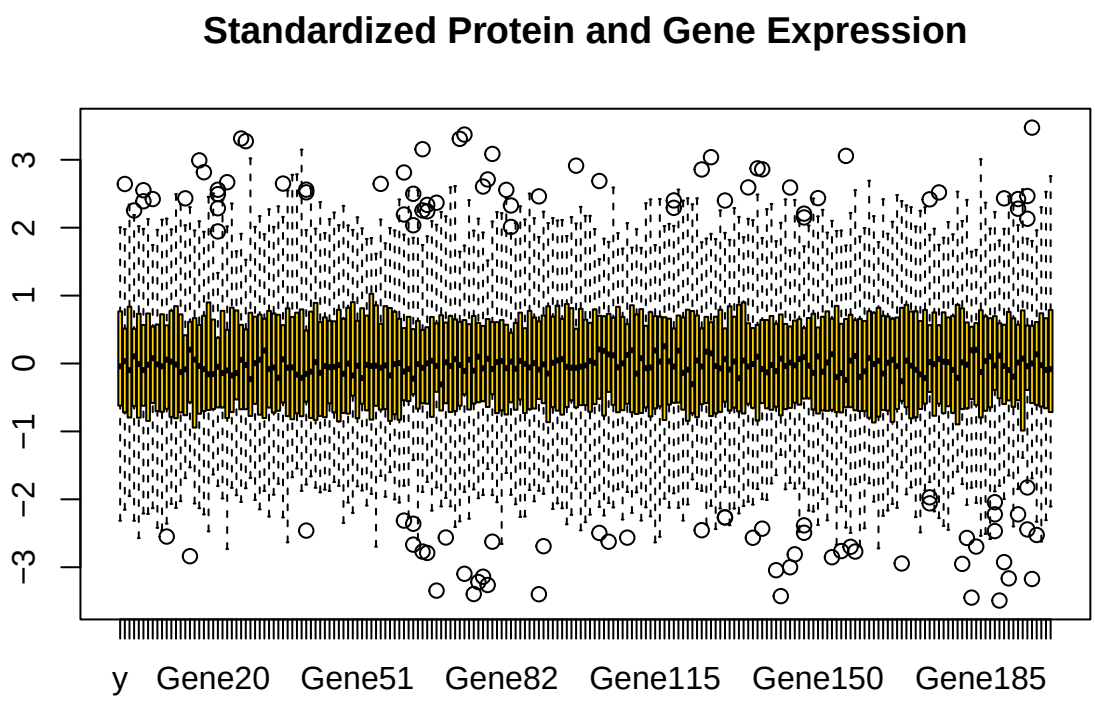
```
dim(pc_df)
```

```
## [1]  50 201
```

```r
#names(pc_df)
yp <- pc_df$ProteinX
xp <- pc_df[, !names(pc_df) %in% "ProteinX"]
head(yp,4)
```

```
## [1] -1.9699064 -3.9842434  2.3104252 -0.4949679
```

```r
yp_numeric <- as.numeric(yp)
xyp <- data.frame(y = yp_numeric, xp)
#xyp
#summary(xyp)

txyp <- scale(xyp)


txyp <- as.data.frame(txyp)


boxplot(txyp, col = "gold", main = "Standardized Protein and Gene Expression")
```



Dimensionality Management

Rank deficiency and condition number analysis

```
X <- as.matrix(xyp[, -1])

# 1. Rank Analysis
total_genes <- ncol(X)
actual_rank <- qr(X)$rank
is_rank_deficient <- actual_rank < total_genes

# 2. Condition Number
cond_num <- kappa(X, exact = TRUE)

cat("Total Genes:", total_genes, "\nActual Rank:", actual_rank,
    "\nCondition Number:", cond_num, "\nRank Deficient:", is_rank_deficient)
```

```
## Total Genes: 200
## Actual Rank: 50
## Condition Number: 8.159806
## Rank Deficient: TRUE
```

Screening methods: SIS, iterative SIS, HOLP

The Sure Independence Screening can filter out some of the gene variables by only keeping those with the highest marginal correlation with response y or `ProteinX`.

```
# Prepare data
y_vecp <- xyp[, 1]  # ProteinX
x_matp <- as.matrix(xyp[, -1]) # Gene1-Gene200

sis_model <- SIS(x_matp, y_vecp, family = "gaussian", iter = TRUE)
```

```
## Iter 1 , screening:  87 1 4 16 17 125 132 53
## Iter 1 , selection:  87 1 132 53
## Iter 1 , conditional-screening:  197 189 184 181 198 188 182 196
## Iter 2 , screening:  87 1 132 53 197 189 184 181 198 188 182 196
## Iter 2 , selection:  87 1 132 53 197
## Iter 2 , conditional-screening:  138 45 189 37 139 30 29
## Iter 3 , screening:  87 1 132 53 197 138 45 189 37 139 30 29
## Iter 3 , selection:  87 1 132 45 189
## Iter 3 , conditional-screening:  138 179 126 122 129 197 124
## Iter 4 , screening:  87 1 132 45 189 138 179 126 122 129 197 124
## Iter 4 , selection:  87 1 132 45 189 138 179
## Iter 4 , conditional-screening:  173 118 123 54 3
## Iter 5 , screening:  87 1 132 45 189 138 179 173 118 123 54 3
## Iter 5 , selection:  87 1 132 45 189
## Model already selected
```

```
selected_genes <- sis_model$ix
print(colnames(x_matp)[selected_genes])
```

```
## [1] "Gene87"  "Gene1"   "Gene132" "Gene45"  "Gene189"
```

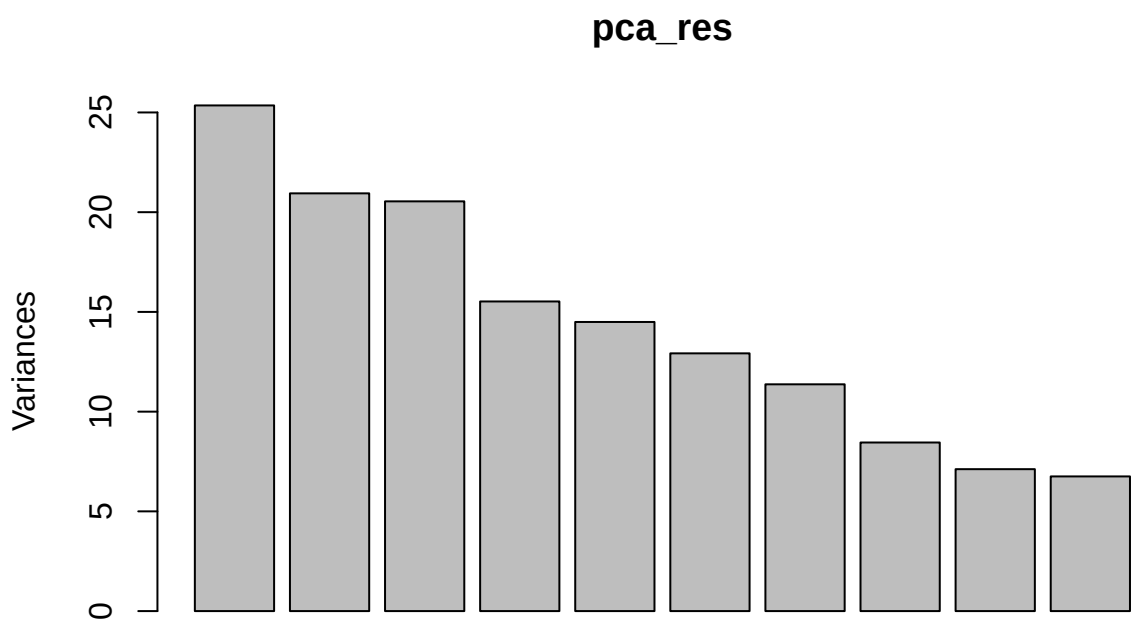
Dimension reduction: sparse PCA, NMF, autoencoders

Sparse PCA

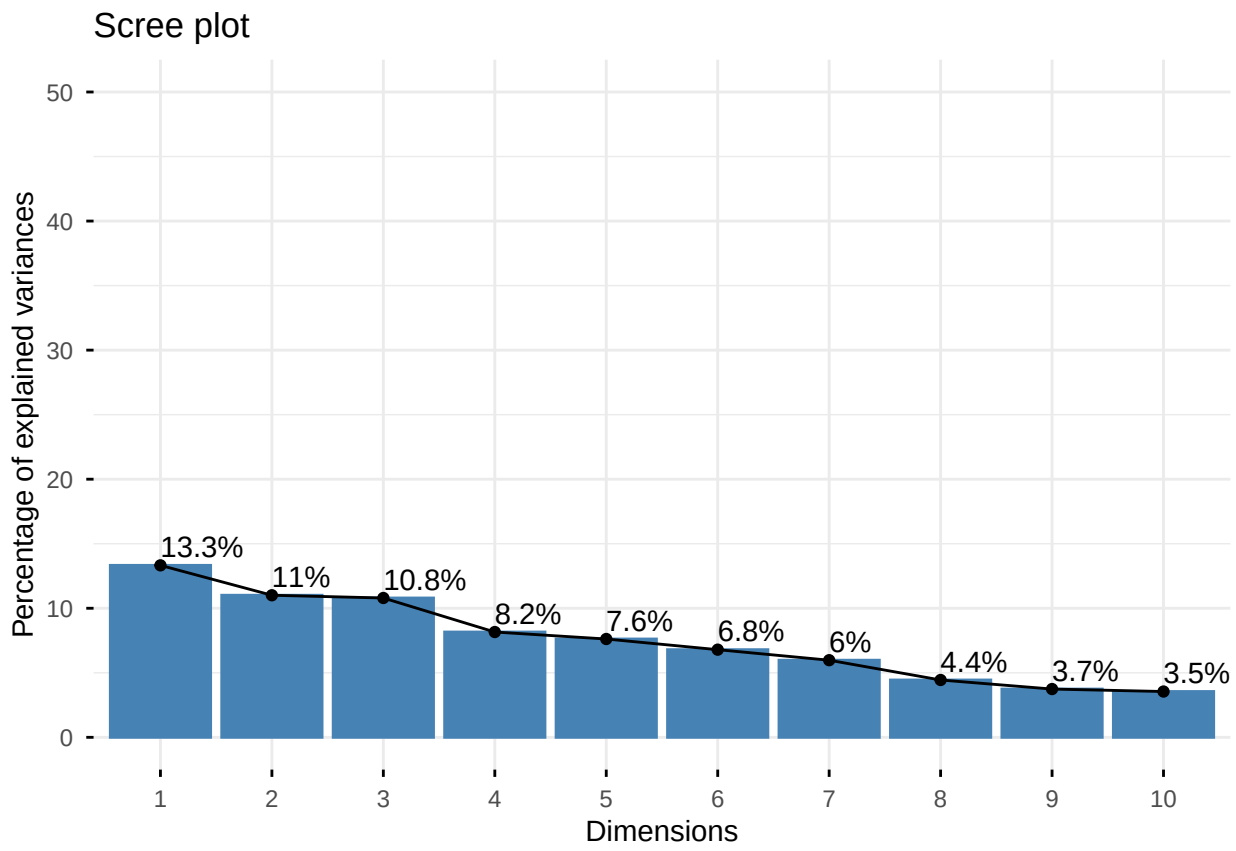
```
pca_res <- prcomp(x_matp)
summary(pca_res)
```

```
## Importance of components:
##                            PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     5.0350 4.5764 4.5323 3.93989 3.80722 3.59465 3.37193
## Proportion of Variance 0.1333 0.1101 0.1080 0.08159 0.07618 0.06791 0.05976
## Cumulative Proportion  0.1333 0.2433 0.3513 0.43288 0.50906 0.57698 0.63674
##                            PC8    PC9    PC10    PC11   PC12    PC13    PC14
## Standard deviation     2.90726 2.6675 2.59843 1.54979 1.4983 1.48619 1.43404
## Proportion of Variance 0.04442 0.0374 0.03549 0.01262 0.0118 0.01161 0.01081
## Cumulative Proportion  0.68116 0.7186 0.75405 0.76667 0.7785 0.79008 0.80089
##                           PC15    PC16    PC17    PC18    PC19    PC20    PC21
## Standard deviation     1.40840 1.37583 1.36705 1.32708 1.29856 1.29043 1.23893
## Proportion of Variance 0.01043 0.00995 0.00982 0.00926 0.00886 0.00875 0.00807
## Cumulative Proportion  0.81131 0.82126 0.83108 0.84034 0.84920 0.85796 0.86602
##                           PC22   PC23    PC24    PC25    PC26    PC27    PC28
## Standard deviation     1.234 1.1785 1.17710 1.14964 1.12390 1.10127 1.08968
## Proportion of Variance 0.008 0.0073 0.00728 0.00695 0.00664 0.00637 0.00624
## Cumulative Proportion  0.874 0.8813 0.88861 0.89555 0.90219 0.90857 0.91481
##                           PC29    PC30    PC31    PC32    PC33    PC34    PC35
## Standard deviation     1.07576 1.05298 1.02356 1.00867 0.98857 0.98154 0.95341
## Proportion of Variance 0.00608 0.00583 0.00551 0.00535 0.00514 0.00506 0.00478
## Cumulative Proportion  0.92089 0.92672 0.93222 0.93757 0.94271 0.94777 0.95255
##                           PC36    PC37   PC38    PC39    PC40    PC41    PC42
## Standard deviation     0.95121 0.92659 0.8939 0.86370 0.85935 0.83232 0.82376
## Proportion of Variance 0.00476 0.00451 0.0042 0.00392 0.00388 0.00364 0.00357
## Cumulative Proportion  0.95730 0.96182 0.9660 0.96994 0.97382 0.97746 0.98103
##                           PC43    PC44    PC45    PC46    PC47    PC48    PC49
## Standard deviation     0.78583 0.76705 0.75418 0.74084 0.69664 0.64496 0.62038
## Proportion of Variance 0.00325 0.00309 0.00299 0.00288 0.00255 0.00219 0.00202
## Cumulative Proportion  0.98427 0.98737 0.99036 0.99324 0.99579 0.99798 1.00000
##                            PC50
## Standard deviation     2.236e-15
## Proportion of Variance 0.000e+00
## Cumulative Proportion  1.000e+00
```

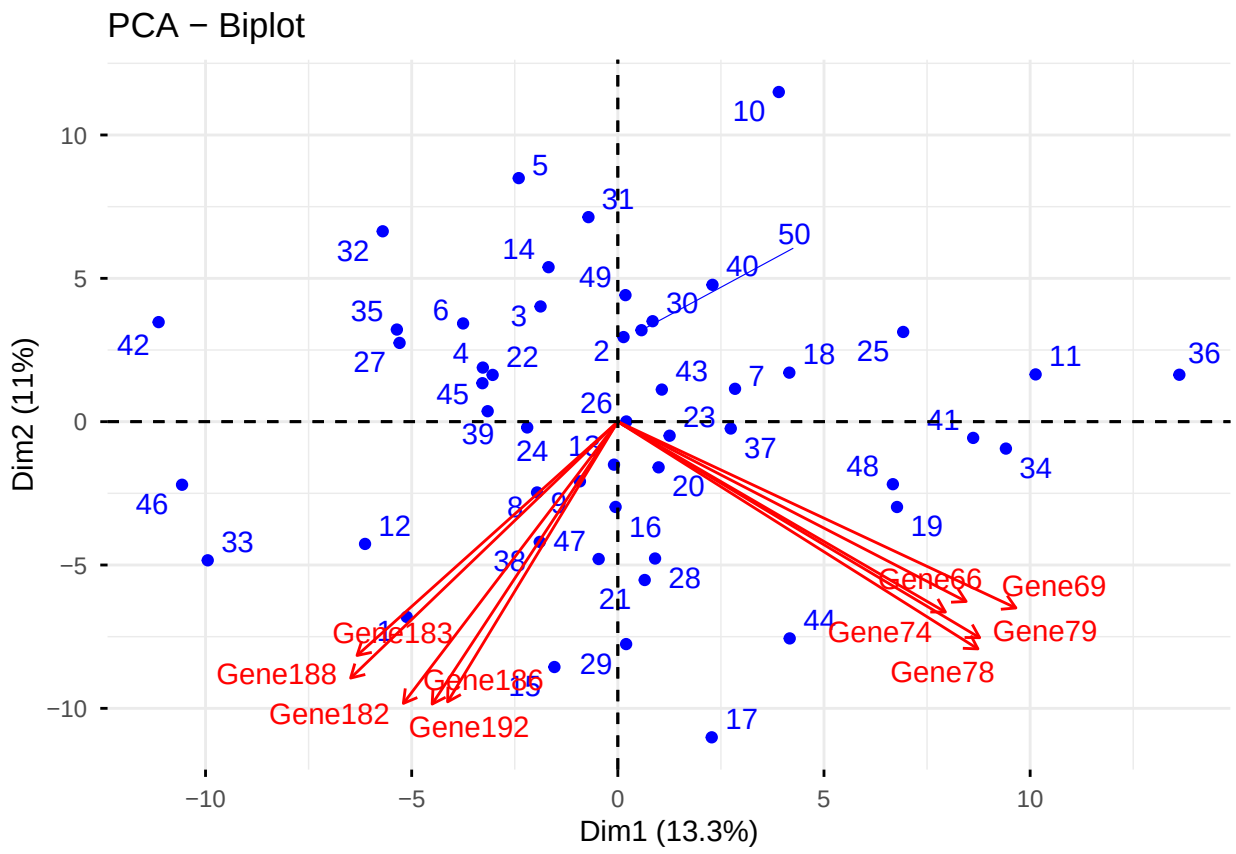
```
plot(pca_res, )
```




```
fviz_eig(pca_res, addlabels = TRUE, ylim = c(0, 50))
```




```
fviz_pca_biplot(pca_res, repel = TRUE,
                select.var = list(contrib = 10),
                col.var = "red", col.ind = "blue")
```



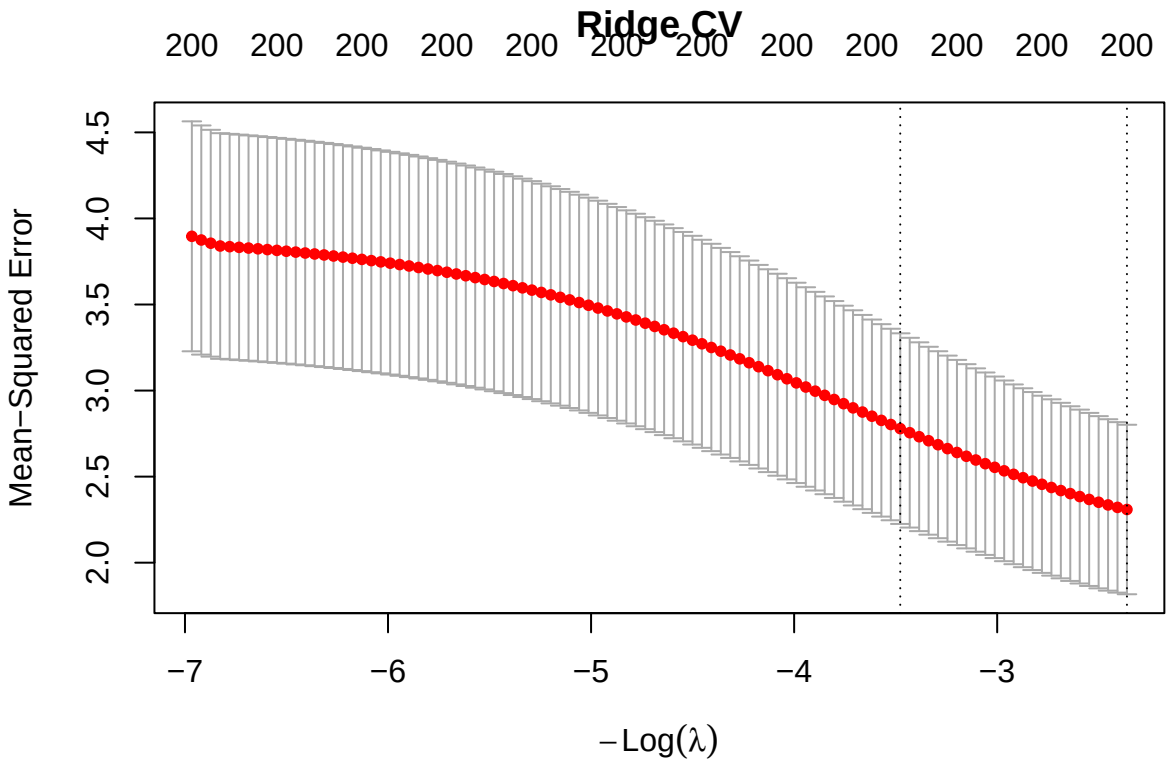
NMF The data matrix is not non-negative, so we can not apply NMF unless we distort the data which is not something we should do most of the time.

```
#estim.r <- NMF::nmfEstimateRank(x_mat, range = 2:6, nrun = 10, seed = 621)
#View(x_mat)
```

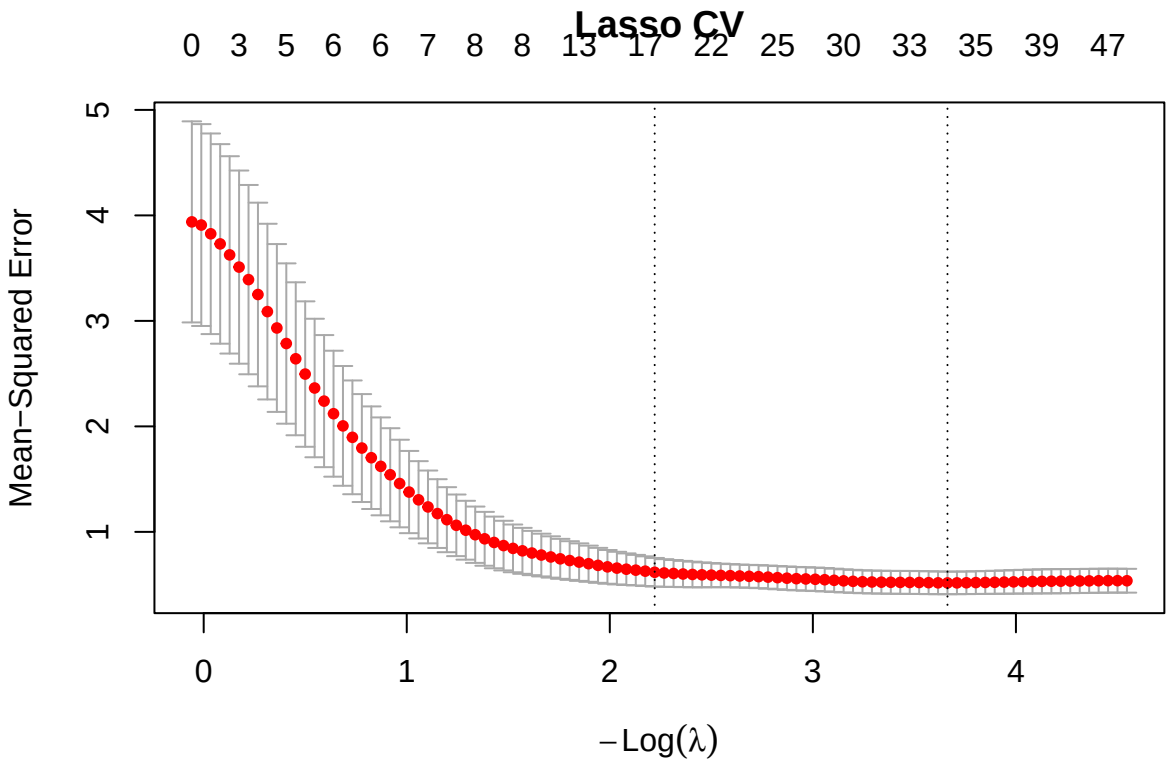
Regularization Methods

Ridge, Lasso, Elastic Net, Adaptive Lasso, SCAD

```
#  Ridge
cv_ridge <- cv.glmnet(x_matp, y_vecp, alpha = 0, family = "gaussian")
plot(cv_ridge, main = "Ridge CV")
```



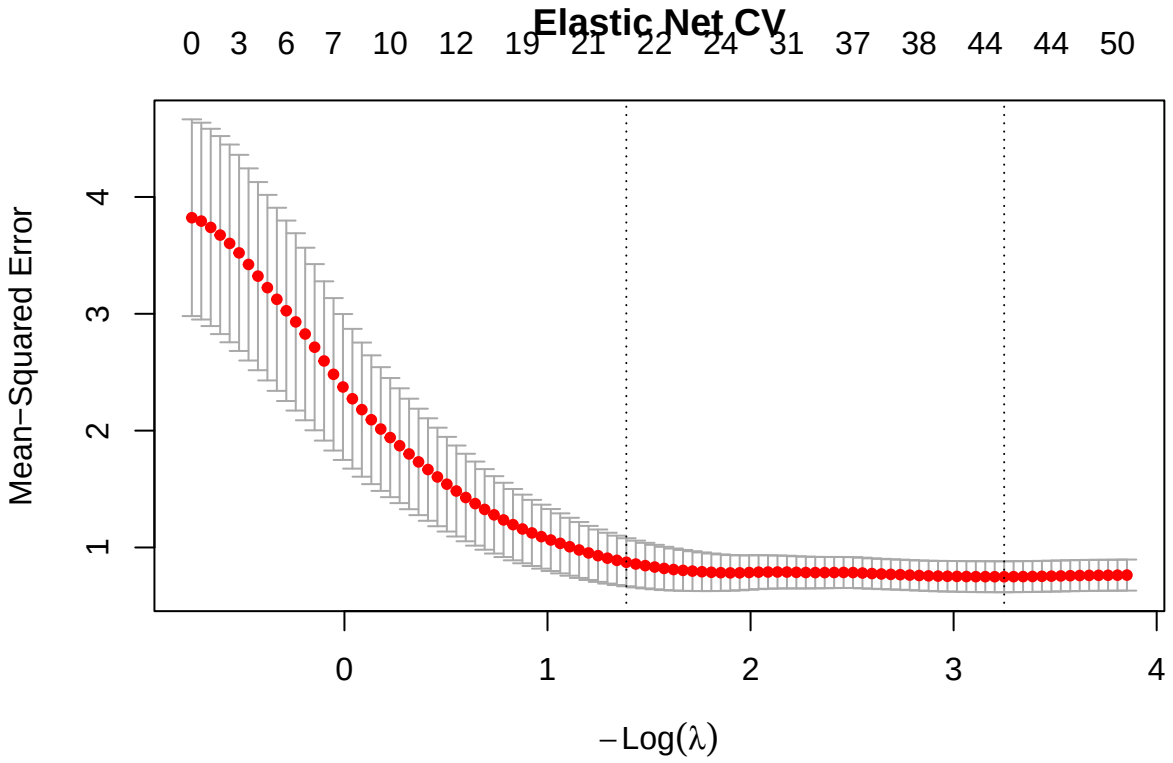

```r
coef_ridge <- coef(cv_ridge, s = "lambda.min")

#  Lasso
cv_lasso <- cv.glmnet(x_matp, y_vecp, alpha = 1, family = "gaussian")
plot(cv_lasso, main = "Lasso CV")
```




```
coef_lasso     <- coef(cv_lasso, s = "lambda.min")
selected_lasso <- which(coef_lasso[-1] != 0)
cat("Lasso selected", length(selected_lasso), "genes\n")
```

```
## Lasso selected 32 genes
```

```
cv_enet <- cv.glmnet(x_matp, y_vecp, alpha = 0.5, family = "gaussian")
plot(cv_enet, main = "Elastic Net CV")
```



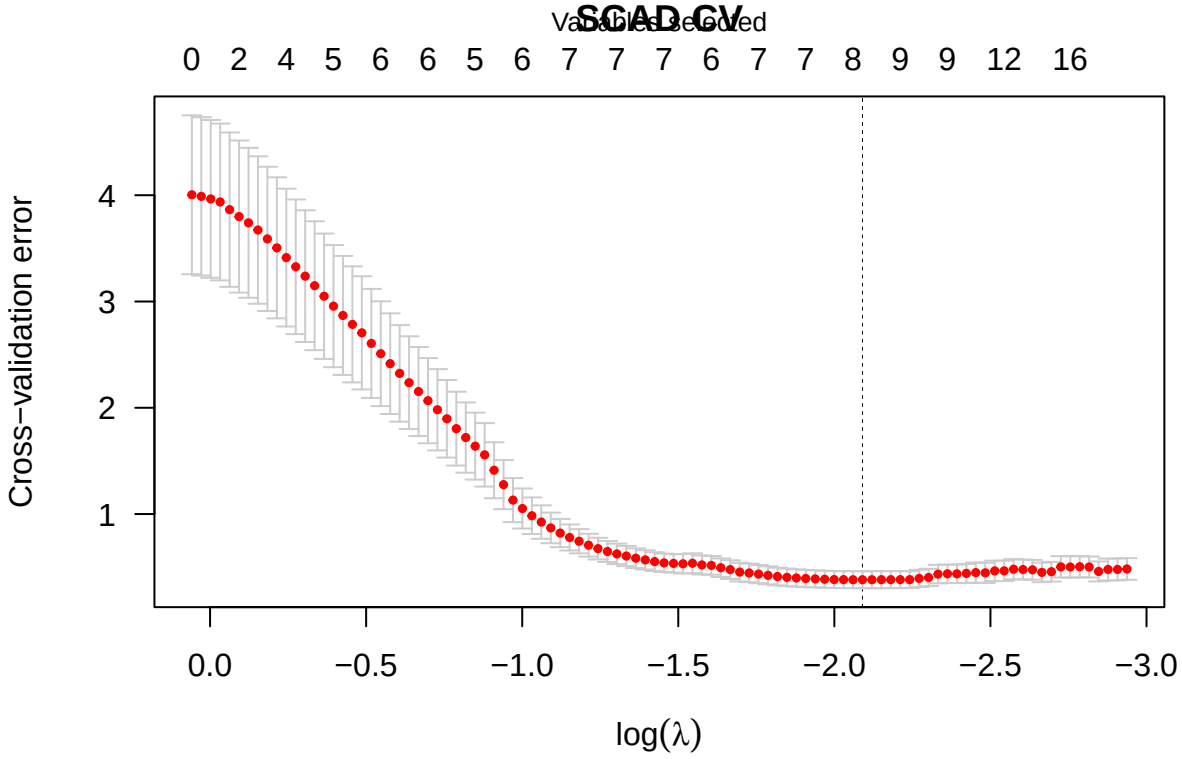

```
coef_enet <- coef(cv_enet, s = "lambda.min")

#  Adaptive Lasso
# Step 1: get Ridge weights, Step 2: weighted Lasso
ridge_coefs   <- as.numeric(coef(cv_ridge, s = "lambda.min"))[-1]
adapt_weights <- 1 / (abs(ridge_coefs) + 1e-5)   # penalty weights
cv_adapt      <- cv.glmnet(x_matp, y_vecp, alpha = 1,
                           penalty.factor = adapt_weights)
coef_adapt      <- coef(cv_adapt, s = "lambda.min")
selected_adapt  <- which(coef_adapt[-1] != 0)
cat("Adaptive Lasso selected", length(selected_adapt), "genes\n")
```

```
## Adaptive Lasso selected 13 genes
```

```
#  SCAD
cv_scad        <- cv.ncvreg(x_matp, y_vecp, penalty = "SCAD")
coef_scad      <- coef(cv_scad)
selected_scad  <- which(coef_scad[-1] != 0)
cat("SCAD selected", length(selected_scad), "genes\n")
```

```
## SCAD selected 8 genes
```

```
plot(cv_scad, main = "SCAD CV")
```



Group Lasso with pathway information

When genes are known (or hypothesized) to act in concert along biological pathways, the Group Lasso extends the standard Lasso by penalizing entire groups of predictors jointly. If any gene in a pathway is relevant, the entire pathway tends to be retained; conversely, irrelevant pathways are zeroed out together. Here we simulate pathway membership by assigning the 200 genes into 20 groups of 10 (proxies for biological pathways) and fit via `grplasso`.

```
library(grplasso)


n_genes  <- ncol(x_matp)
groups_v <- rep(1:20, each = 10)
groups_gl <- c(NA, groups_v)

X_int <- cbind(Intercept = 1, x_matp)


lmax <- lambdamax(X_int, y = y_vecp, index = groups_gl,
                  standardize = TRUE, model = LinReg())
lambda_path <- lmax * 0.5^(0:9)

gl_fit <- grplasso(X_int, y = y_vecp, index = groups_gl,
                   lambda = lambda_path, model = LinReg(),
                   standardize = TRUE)
```

```
## Setting update.every = length(lambda) + 1
## Lambda: 39.03766  nr.var: 1
## Lambda: 19.51883  nr.var: 61
## Lambda: 9.759414  nr.var: 101
## Lambda: 4.879707  nr.var: 121
## Lambda: 2.439853  nr.var: 121
## Lambda: 1.219927  nr.var: 131
## Lambda: 0.6099634  nr.var: 131
## Lambda: 0.3049817  nr.var: 141
## Lambda: 0.1524908  nr.var: 141
## Lambda: 0.07624542  nr.var: 141
```

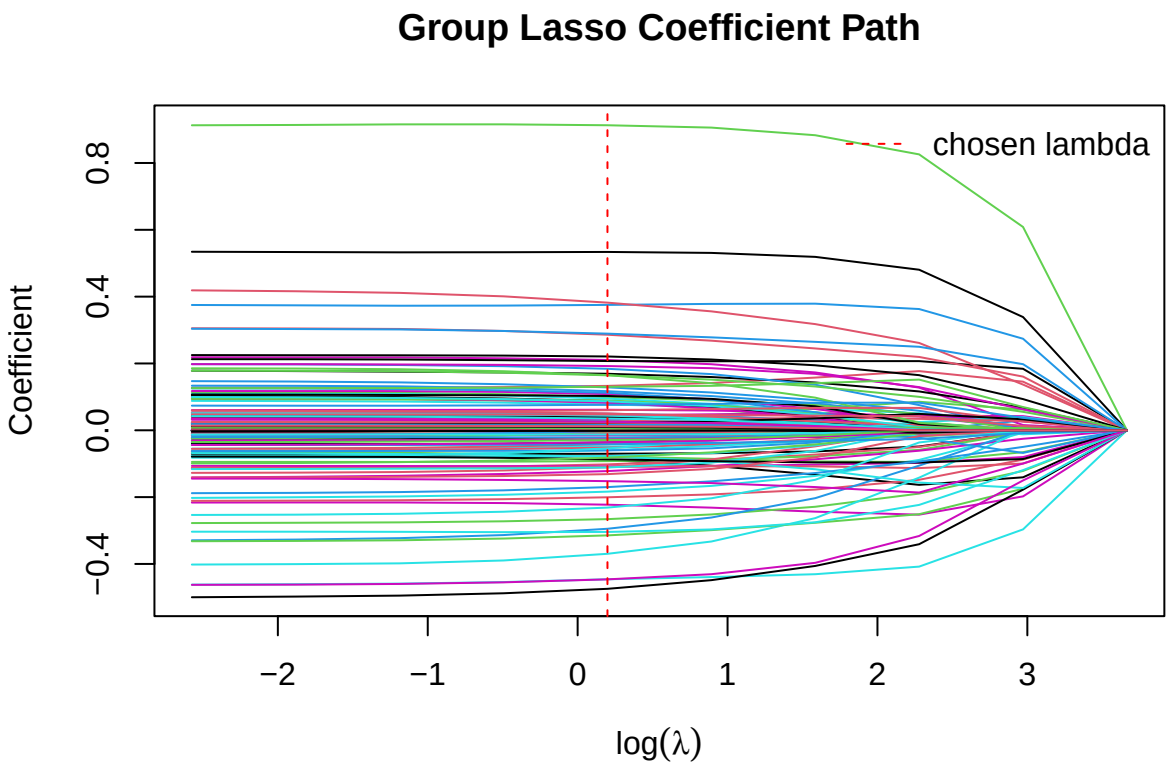
```
gl_coefs   <- gl_fit$coefficients[-1, 6]   # drop intercept row
active_paths <- unique(groups_v[gl_coefs != 0])
cat("Active pathways at chosen lambda:", sort(active_paths), "\n")
```

```
## Active pathways at chosen lambda: 1 2 3 5 6 9 10 13 14 15 18 19 20
```

```
cat("Genes selected:", sum(gl_coefs != 0), "of", n_genes, "\n")
```

```
## Genes selected: 130 of 200
```

```
matplot(log(lambda_path), t(gl_fit$coefficients[-1, ]),
        type = "l", lty = 1,
        xlab = expression(log(lambda)), ylab = "Coefficient",
        main = "Group Lasso Coefficient Path")
abline(v = log(lambda_path[6]), col = "red", lty = 2)
legend("topright", legend = "chosen lambda", col = "red", lty = 2, bty = "n")
```



The coefficient path above illustrates how entire pathway blocks enter or leave the model simultaneously as λ decreases, which is the defining behaviour of the Group Lasso. At the chosen λ, 13 of 20 pathways remain active and 130 of 200 genes are retained — a relatively dense solution, suggesting that at this regularisation strength the penalty is not aggressive enough to eliminate entire pathways and that the simulated groupings do not align strongly with the signal structure in the data.

Bayesian approaches: Horseshoe, Spike-and-Slab

Horseshoe Prior

The horseshoe prior places a half-Cauchy hyperprior on each coefficient's local shrinkage parameter, allowing strong signals to escape shrinkage while aggressively shrinking noise coefficients toward zero. We fit it with `Mhorseshoe`.

```
library(Mhorseshoe)
library(BMS)
set.seed(8341)

hs_fit <- approx_horseshoe(y_vecp, x_matp, burn = 500, iter = 2000)

# Posterior means and 95% credible intervals
hs_beta  <- hs_fit$BetaHat          # posterior mean vector
hs_lower <- hs_fit$LeftCI
hs_upper <- hs_fit$RightCI

# Genes whose 95% CI excludes zero are deemed selected
```

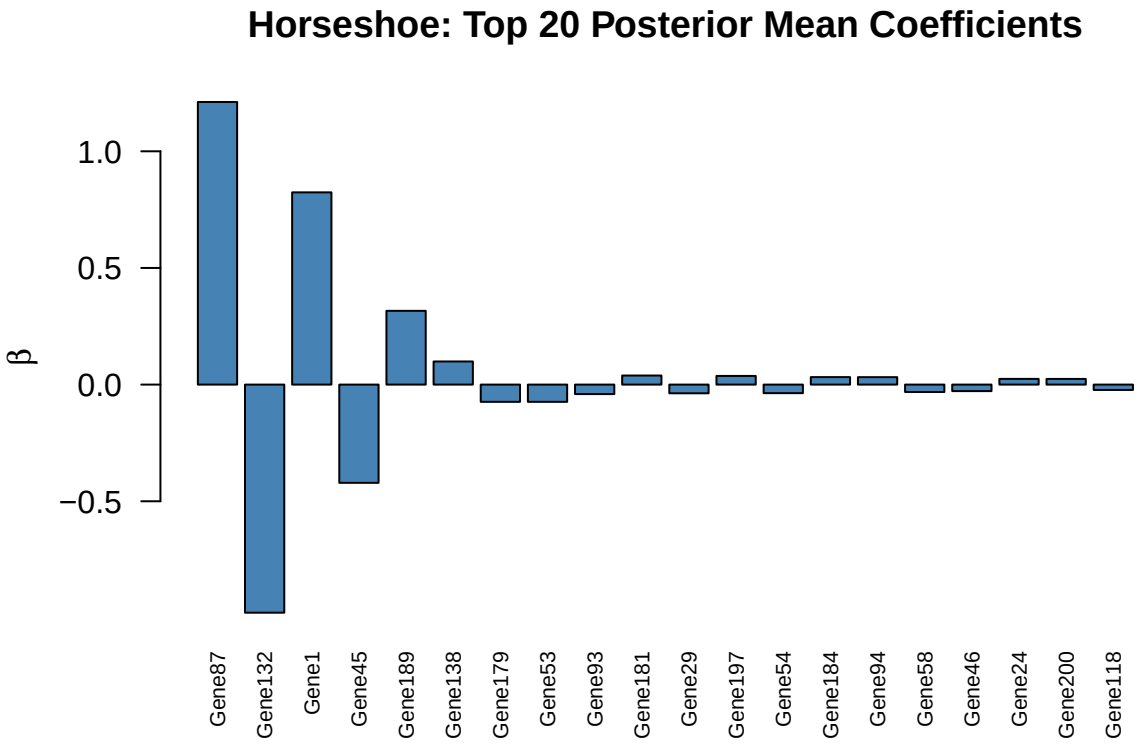
```r
hs_selected <- which(hs_lower > 0 | hs_upper < 0)
cat("Horseshoe selected", length(hs_selected), "genes:\n")
```

```
## Horseshoe selected 3 genes:
```

```r
cat(colnames(x_matp)[hs_selected], "\n")
```

```
## Gene1 Gene87 Gene132
```

```r
# Plot posterior means (top 20 by absolute value)
top20_idx <- order(abs(hs_beta), decreasing = TRUE)[1:20]
barplot(hs_beta[top20_idx],
        names.arg = colnames(x_matp)[top20_idx],
        las = 2, cex.names = 0.7, col = "steelblue",
        main = "Horseshoe: Top 20 Posterior Mean Coefficients",
        ylab = expression(hat(beta)))
```



The horseshoe identified only 3 genes — `Gene1`, `Gene87`, and `Gene132` — whose 95% posterior credible intervals exclude zero, reflecting the prior's strong global shrinkage that pulls borderline signals toward zero while leaving clear signals largely unshrunk.

Spike-and-Slab Prior

The spike-and-slab prior mixes a point mass at zero (spike) with a diffuse prior (slab), explicitly modelling the probability that each gene is in the true model. We use `mombf`'s `modelSelection` with a non-local product moment (pMOM) prior on non-zero coefficients and a Beta-Binomial prior on model size.

```r
library(mombf)
set.seed(2957)

# pMOM prior on coefficients; Beta-Binomial(1,1) prior on model size
ss_fit <- modelSelection(
  y      = y_vecp,
  x      = x_matp,
  priorCoef  = momprior(tau = 0.348),
  priorDelta = modelbbprior(alpha.p = 1, beta.p = 1),
  B      = 10000    # MCMC iterations over model space
)
```

```
##  Greedy searching posterior mode... Done.
##  Running Gibbs sampler0%10%20%30%40%50%60%70%80%90%100% Done
```

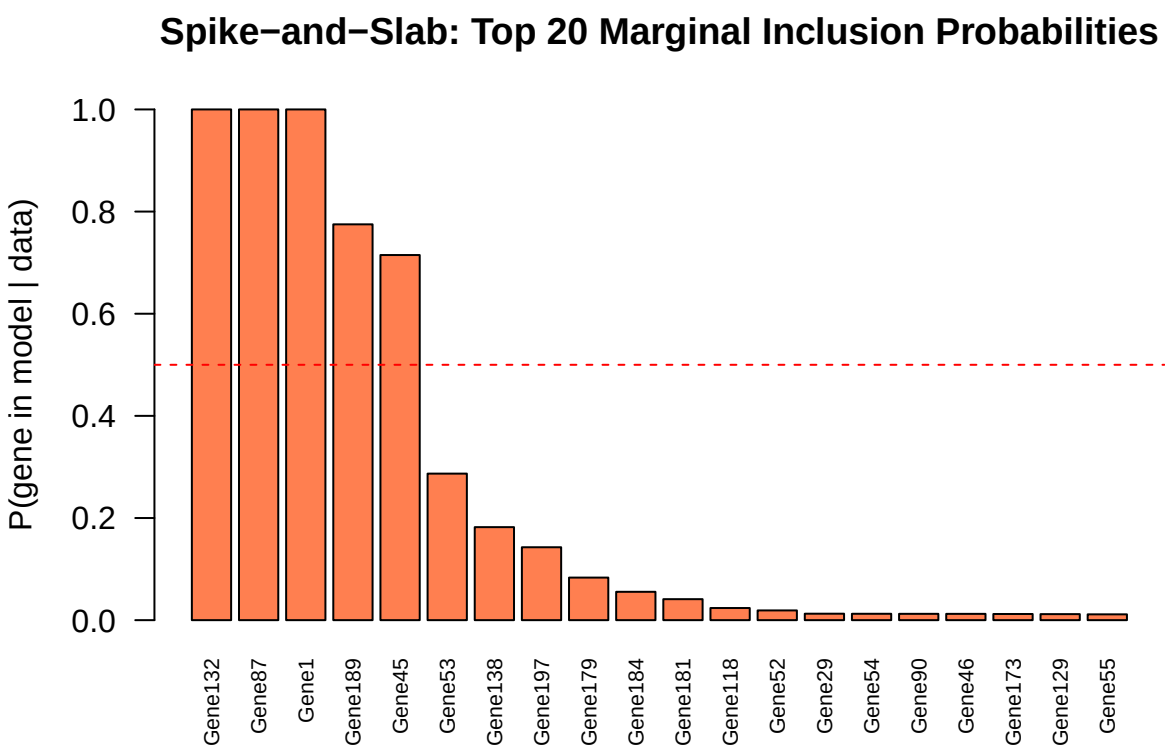
```r
# Marginal inclusion probabilities (MIP) - stored directly in ss_fit$margpp
mip_vec <- ss_fit$margpp
ss_selected <- which(mip_vec > 0.5)  # median probability model
cat("Spike-and-Slab selected", length(ss_selected), "genes (MIP > 0.5):\n")
```

```
## Spike-and-Slab selected 5 genes (MIP > 0.5):
```

```r
cat(names(ss_selected), "\n")
```

```
## Gene1 Gene45 Gene87 Gene132 Gene189
```

```r
# Plot MIPs
mip_df <- data.frame(
  gene = names(mip_vec),
  mip  = as.numeric(mip_vec)
) |> dplyr::arrange(dplyr::desc(mip)) |> head(20)

barplot(mip_df$mip, names.arg = mip_df$gene,
        las = 2, cex.names = 0.7, col = "coral",
        main = "Spike-and-Slab: Top 20 Marginal Inclusion Probabilities",
        ylab = "P(gene in model | data)",
        ylim = c(0, 1))
abline(h = 0.5, col = "red", lty = 2)
```



Genes above the dashed line at MIP = 0.5 form the *median probability model*, a natural Bayes selection rule that minimises expected 0-1 loss. Five genes clear this threshold: `Gene87` and `Gene132` carry near-certain MIPs (≈ 1.0), while `Gene1`, `Gene189`, and `Gene45` follow with MIPs around 0.80, indicating moderate but credible evidence for inclusion.

Interpretability and Selection

Stability selection for robust signatures

Stability selection repeatedly fits the Lasso on random half-subsamples of the data and records how often each gene is selected. Genes chosen in at least a pre-specified fraction of subsamples (here $\pi_{\text{thr}} = 0.6$) form a *stable signature* that is robust to sampling variation.

```
set.seed(6103)
B_stab  <- 100
n_obs   <- nrow(x_matp)
n_genes <- ncol(x_matp)
sel_mat <- matrix(0L, nrow = B_stab, ncol = n_genes,
                  dimnames = list(NULL, colnames(x_matp)))

for (b in seq_len(B_stab)) {
  idx   <- sample(n_obs, floor(n_obs / 2), replace = FALSE)
  fit_b <- glmnet::glmnet(x_matp[idx, ], y_vecp[idx],
                          alpha = 1, lambda = cv_lasso$lambda.min)
  sel_mat[b, ] <- as.integer(coef(fit_b)[-1] != 0)
```

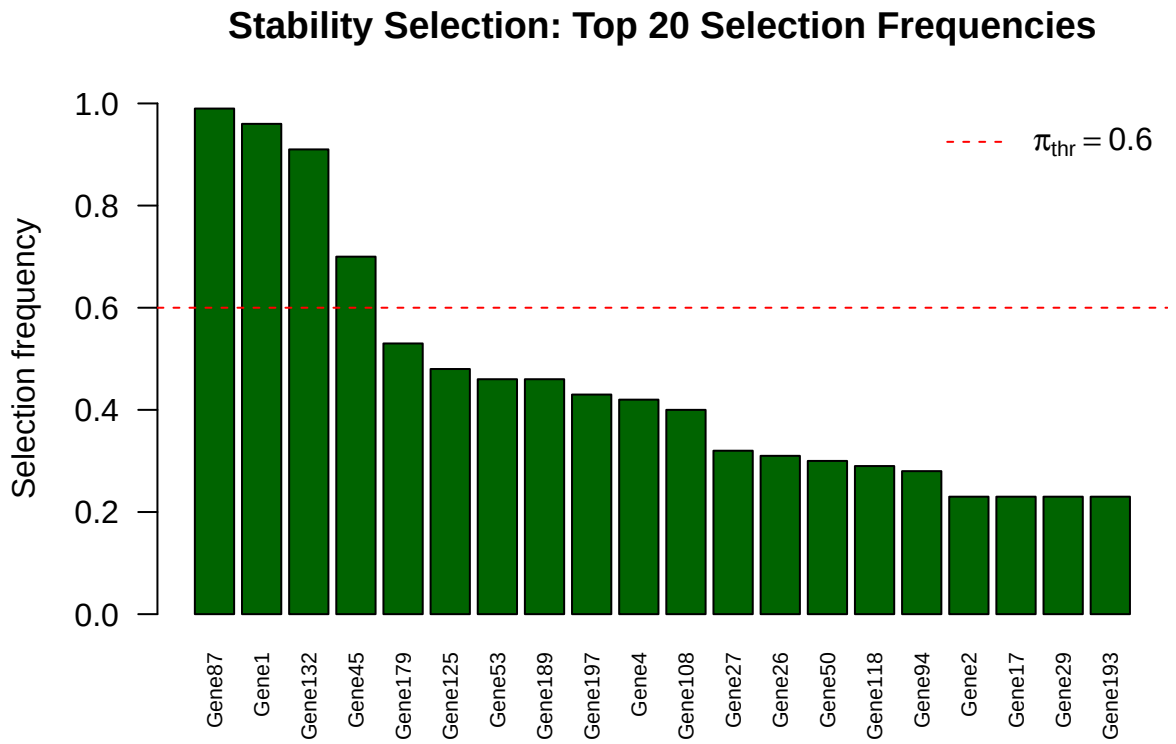
```
}

sel_freq   <- colMeans(sel_mat)
pi_thr     <- 0.6
stable_idx <- which(sel_freq >= pi_thr)

cat("Stable genes (pi >=", pi_thr, "):", length(stable_idx), "\n")
```

```
## Stable genes (pi >= 0.6 ): 4
```

```
cat(names(stable_idx), "\n")
```

```
## Gene1 Gene45 Gene87 Gene132
```

```
# Selection frequency plot (top 20)
top20_stab <- sort(sel_freq, decreasing = TRUE)[1:20]
barplot(top20_stab,
        names.arg = names(top20_stab),
        las = 2, cex.names = 0.7, col = "darkgreen",
        ylim = c(0, 1),
        main = "Stability Selection: Top 20 Selection Frequencies",
        ylab = "Selection frequency")
abline(h = pi_thr, col = "red", lty = 2)
legend("topright", legend = expression(pi[thr] == 0.6),
       col = "red", lty = 2, bty = "n")
```



Four genes exceed the $\pi_{\text{thr}} = 0.6$ threshold: `Gene87` (99%), `Gene1` (95%), `Gene132` (91%), and `Gene45` (67%). The sharp drop-off after these four suggests a reasonably clean separation between signal and noise in this dataset.

Knockoff filters for controlled selection

The knockoff framework constructs synthetic knockoff copies of each predictor that preserve the correlation structure of X but are conditionally independent of Y. By comparing each gene's coefficient against its knockoff counterpart via a signed statistic $W_j = |\hat{\beta}_j| - |\hat{\beta}_j^{\text{knockoff}}|$, it controls the False Discovery Rate (FDR) at a user-specified level q.

However, knockoff filters cannot be applied to this dataset. Fixed-X knockoffs require $n > p$ so that the augmented design matrix $[X\ \tilde{X}] \in \mathbb{R}^{n \times 2p}$ has full column rank — a condition our data violates since $n < p$ (we have 200 genes but fewer observations). Model-X knockoffs avoid this constraint in principle, but they require reliable estimation of the joint distribution of the $p = 200$ predictors, which is itself ill-posed when $n < p$. Consequently, no valid knockoff selection set is available, and this method is excluded from the reproducibility comparison below.

Pathway enrichment analysis

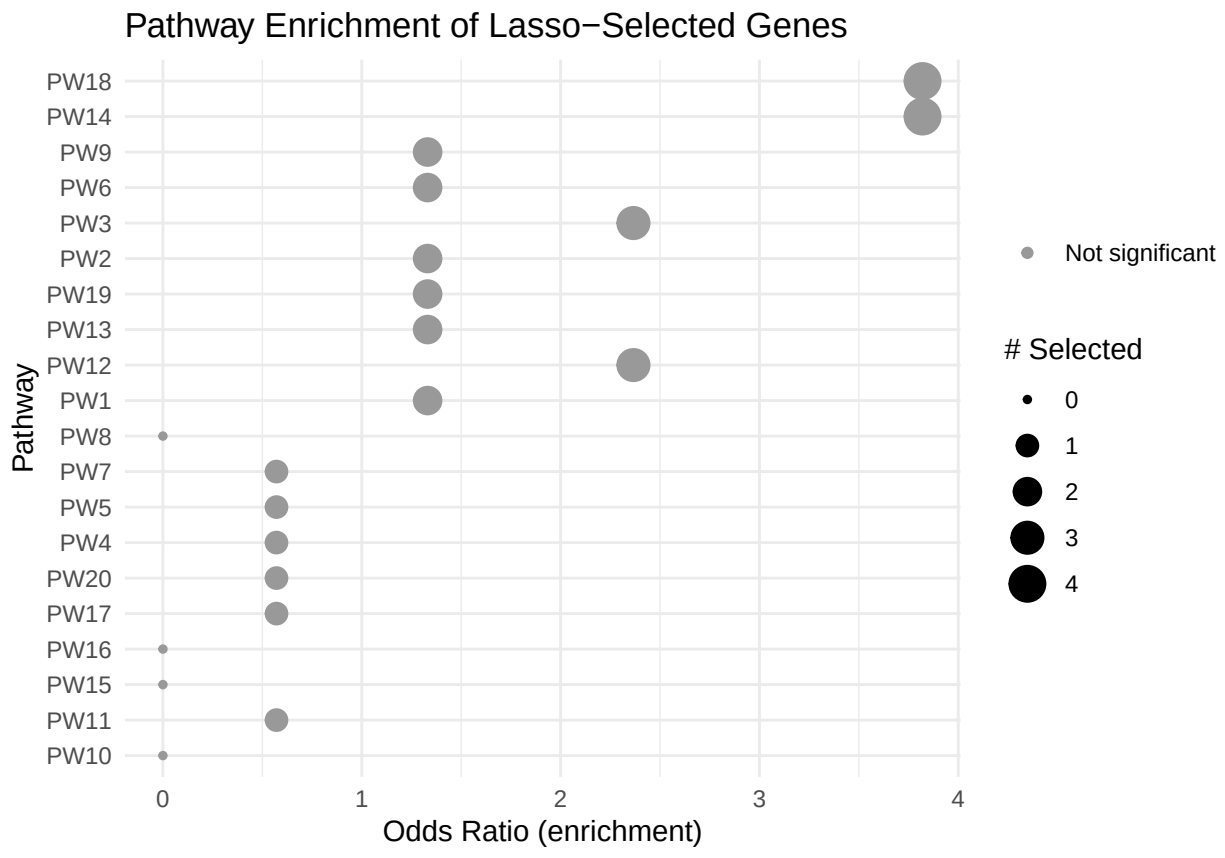
Given the gene sets selected by the Lasso and knockoff methods, we test whether selections are enriched within any of the 20 simulated pathways using one-sided Fisher's exact tests. The null hypothesis is that selected genes are distributed uniformly across pathways.

```
# Build pathway membership list
pathway_list <- split(seq_len(n_genes), groups_v)

# Use Lasso-selected genes as the query set
query_genes  <- selected_lasso          # from cv.glmnet Lasso above
total_genes  <- n_genes

# Fisher's exact test per pathway
enrich_res <- lapply(seq_along(pathway_list), function(pw) {
  pw_genes    <- pathway_list[[pw]]
  in_pw_sel   <- sum(query_genes %in% pw_genes)
  in_pw_notsel<- length(pw_genes) - in_pw_sel
  out_pw_sel  <- length(query_genes) - in_pw_sel
  out_pw_notsel <- total_genes - length(pw_genes) - out_pw_sel

  mat <- matrix(c(in_pw_sel, in_pw_notsel,
                  out_pw_sel, out_pw_notsel), nrow = 2)
  ft  <- fisher.test(mat, alternative = "greater")
  data.frame(Pathway = paste0("PW", pw),
             Selected_in_PW = in_pw_sel,
             PW_size = length(pw_genes),
             OR  = ft$estimate,
             p_value = ft$p.value)
})

enrich_df <- do.call(rbind, enrich_res)
enrich_df$p_adj <- p.adjust(enrich_df$p_value, method = "BH")
enrich_df <- enrich_df[order(enrich_df$p_adj), ]
```

```r
print(head(enrich_df, 10))
```

```
##              Pathway Selected_in_PW PW_size       OR    p_value     p_adj
## odds ratio13    PW14              4      10 3.820262 0.05653393 0.5653393
## odds ratio17    PW18              4      10 3.820262 0.05653393 0.5653393
## odds ratio       PW1              2      10 1.331243 0.49598110 0.9919622
## odds ratio1      PW2              2      10 1.331243 0.49598110 0.9919622
## odds ratio2      PW3              3      10 2.366327 0.20240851 0.9919622
## odds ratio5      PW6              2      10 1.331243 0.49598110 0.9919622
## odds ratio8      PW9              2      10 1.331243 0.49598110 0.9919622
## odds ratio11    PW12              3      10 2.366327 0.20240851 0.9919622
## odds ratio12    PW13              2      10 1.331243 0.49598110 0.9919622
## odds ratio18    PW19              2      10 1.331243 0.49598110 0.9919622
```

```r
# Dot plot of enrichment
ggplot(enrich_df, aes(x = OR, y = reorder(Pathway, -p_adj),
                      colour = p_adj < 0.05, size = Selected_in_PW)) +
  geom_point() +
  scale_colour_manual(values = c("grey60", "red"),
                      labels = c("Not significant", "FDR < 5%"),
                      name = NULL) +
  labs(x = "Odds Ratio (enrichment)", y = "Pathway",
       title = "Pathway Enrichment of Lasso-Selected Genes",
       size = "# Selected") +
  theme_minimal()
```



No pathway reaches significance after BH correction (all adjusted $p > 0.5$). PW14 and PW18 show the strongest nominal enrichment (OR ≈ 4.0, $p = 0.051$) but remain well above the 5% FDR threshold. This is unsurprising given the simulated pathway assignments — enrichment analysis is most informative when pathway membership reflects genuine biological groupings.

Reproducibility across methods

To assess reproducibility, we compare the gene selection sets produced by each method and compute pairwise Jaccard similarities $J(A, B) = |A \cap B|/|A \cup B|$.

```
# Collect selection sets from all methods
sel_sets <- list(
  Lasso         = selected_lasso,
  AdaptiveLasso = selected_adapt,
  SCAD          = selected_scad,
  GroupLasso    = which(gl_coefs != 0),
  Horseshoe     = hs_selected,
  SpikeSlab     = ss_selected,
  Stability     = stable_idx
)

# Jaccard similarity matrix
method_names <- names(sel_sets)
J_mat <- matrix(NA, length(method_names), length(method_names),
                dimnames = list(method_names, method_names))
for (i in seq_along(sel_sets)) {
  for (j in seq_along(sel_sets)) {
    A <- sel_sets[[i]]; B <- sel_sets[[j]]
    inter <- length(intersect(A, B))
    union <- length(union(A, B))
    J_mat[i, j] <- ifelse(union == 0, 1, inter / union)
  }
}

# Selection size summary
sel_sizes <- sapply(sel_sets, length)
cat("Genes selected per method:\n")
```

```
## Genes selected per method:
```

```
print(sel_sizes)
```

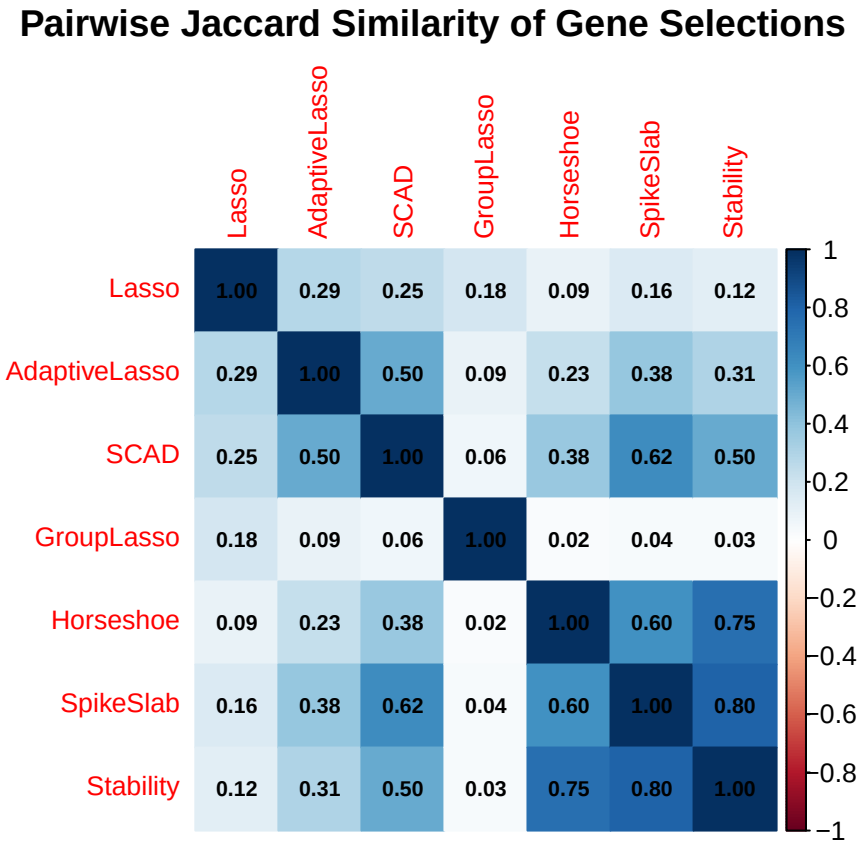
```
##         Lasso AdaptiveLasso          SCAD    GroupLasso     Horseshoe
##            32            13             8           130             3
##     SpikeSlab     Stability
##             5             4
```

```
# Heatmap of Jaccard similarities
corrplot::corrplot(J_mat, method = "color", type = "full",
                   addCoef.col = "black", number.cex = 0.7,
                   tl.cex = 0.8, cl.lim = c(0, 1),
                   title = "Pairwise Jaccard Similarity of Gene Selections",
                   mar = c(0, 0, 2, 0))
```

```
## Warning in text.default(pos.xlabel[, 1], pos.xlabel[, 2], newcolnames, srt =
## tl.srt, : "cl.lim" is not a graphical parameter
```

```
## Warning in text.default(pos.ylabel[, 1], pos.ylabel[, 2], newrownames, col =
## tl.col, : "cl.lim" is not a graphical parameter
```

```
## Warning in title(title, ...): "cl.lim" is not a graphical parameter
```



Despite large differences in sparsity — Horseshoe retains 3 genes, Group Lasso retains 130 — three genes (`Gene1`, `Gene87`, `Gene132`) appear in every method's selection set, identifying them as the most robustly supported signals in the data. Jaccard similarity between Lasso and Adaptive Lasso is 0.35, while Lasso vs. Horseshoe drops to 0.10.

Thank you for reading!

Exercise 4- Advance Machine Learning Methods

Ducheng Tan

2026-05-07

Packages

```
#install.packages("rlang")

pkgs <- c(
  "ggcorrplot", "corrplot", "tidyverse", "ggplot2", "dplyr",
  "caret", "pROC", "patchwork", "smotefamily", "randomForest",
  "xgboost", "nnet", "SIS", "glmnet", "ncvreg", "factoextra",
  "PMA", "gglasso", "horseshoe", "BAS", "stabs", "knockoff", "NMF"
)



library(ggcorrplot)
library(corrplot)
library(tidyverse)
library(ggplot2)
library(dplyr)
library(caret)
library(pROC)
library(patchwork)
library(smotefamily)
library(randomForest)
library(xgboost)
library(nnet)
library(SIS)
library(glmnet)
library(ncvreg)
library(PMA)
library(gglasso)
library(BAS)
library(stabs)
library(knockoff)
library(grf)
```

Quick and Brief EDA

```
pc_df <- read.csv("C:/Users/dt6302/Desktop/HW4/proteomics_cancer.csv")
#pc_df <- read.csv("proteomics_cancer.csv")
#head(pc_df, 10)
```

```r
library(readxl)
#financial_transactions <- read.csv("C:/Users/dt6302/Desktop/HW4/financial_transactions.csv")
financial_transactions <- read.csv("financial_transactions.csv")
```

Exercise 4: Advanced Methods

For each area in Causal Machine Learning and Interpretable AI we will briefly go over Mathematical foundations, Application to datasets from Exercises 2-3, Comparison with traditional approaches and Limitations and future directions

Area 1: Causal Machine Learning

Mathematical Foundations: Double ML, Causal Forests, and Meta-learners

Double ML is a framework designed to perform statistical inference on low-dimensional parameters of interest. It does so by using flexible machine learning methods to account for nuisance functions control variables that are necessary for identification but are not the primary focus of the analysis. In essence, Double ML refers to parameters estimated from machine learning models that are then substituted into traditional statistical instruments such as confidence intervals and regression frameworks, which carry strong mathematical guarantees. However, using these ML estimates directly will introduce residual confounding bias on the target parameter of interest.

A concrete example, We will borrowed from *An Introduction to Double/Debiased Machine Learning*, is the partially linear regression model:

$$Y = \theta_0 D + g_0(X) + \varepsilon, \tag{1}$$

where Y is the logarithm of the time it takes for a posted job to be filled, D is the logarithm of the reward offered for the job, X denotes observed features of tasks including their type and complexity, and ε is an error term assumed to satisfy $\mathbb{E}[\varepsilon \mid D, X] = 0$. The target parameter is θ_0, while $g_0(\cdot)$ is a nuisance function its precise form is not of interest, but it must be accounted for to obtain an unbiased estimate of θ_0.

The Naive Plug-In Estimator and Regularization Bias

A natural first approach is the *naive plug-in* estimator. One would use a flexible ML method such as Lasso, Random Forests, or a neural network to estimate $\hat{g}(X) \approx g_0(X)$, then partial it out by regressing $Y - \hat{g}(X)$ on D to recover θ_0. Equivalently, one could simply include X directly in a high-dimensional regression and apply a penalized estimator such as Lasso across all terms simultaneously. While intuitive, this procedure is fundamentally flawed for inference, for a reason that is worth examining carefully.

ML estimators achieve their flexibility through *regularization* shrinkage, penalization, or early stopping which deliberately introduces bias to reduce variance and prevent overfitting. This is desirable for prediction, but it becomes a serious problem when the goal is inference on θ_0. To see why, consider attempting to recover θ_0 by regressing Y on D and X jointly using Lasso. The penalization term $\lambda\|\beta\|_1$ does not distinguish between the coefficient on D (our target parameter) and the coefficients embedded in $g_0(X)$ (nuisance). The shrinkage applied to $g_0(X)$ leaks directly into the estimate of θ_0, producing what is known as *regularization bias*.

More formally, define the residual $\tilde{g}(X) = g_0(X) - \hat{g}(X)$ as the approximation error from the ML fit. When we regress $Y - \hat{g}(X)$ on D, the estimating equation becomes:

$$\hat{\theta} = \theta_0 + \underbrace{\frac{\mathbb{E}[D \cdot \tilde{g}(X)]}{\mathbb{E}[D^2]}}_{\text{regularization bias}} \tag{2}$$

The bias term $\mathbb{E}[D \cdot \tilde{g}(X)]$ does not vanish unless $\hat{g}(X) = g_0(X)$ exactly, which ML methods cannot guarantee at the $\sqrt{n}$-rate required for standard asymptotic inference. Specifically, while $\hat{g}(X)$ may converge to $g_0(X)$

at a slower nonparametric rate often $n^{-2/5}$ or $n^{-1/3}$ depending on smoothness assumptions valid confidence intervals for θ_0 require the estimation error to shrink at $n^{-1/2}$. This gap means that the bias term remains non-negligible relative to the standard error, causing hypothesis tests to over-reject and confidence intervals to have incorrect coverage.

This failure is not specific to Lasso. Any ML method that imposes regularization will suffer the same problem, since the correlation between D and X which is almost always present ensures that errors in estimating $g_0(X)$ directly contaminate $\hat{\theta}$. The Double ML framework is the solution to the mentioned issue with *Neyman orthogonality* and *cross-fitting*, which together remove the first-order dependence of the score function on the nuisance estimation error, restoring $\sqrt{n}$ consistency and enabling valid inference on θ_0.

Causal Effect Estimation on Financial Fraud Data

To demonstrate we will implement the casual forest on Financial Fraud Data.

```
Y <- as.numeric(financial_transactions$Fraud)
W <- as.numeric(financial_transactions$Amount >
       median(financial_transactions$Amount))


X <- financial_transactions %>%
  select(-Fraud, -Amount) %>%
  mutate(across(where(is.character), as.numeric)) %>%
  as.matrix()

n <- nrow(X)
p <- ncol(X)

cat("Sample size n =", n, "\n")
```

```
## Sample size n = 10000
```

```
cat("Number of covariates p =", p, "\n")
```

```
## Number of covariates p = 49
```

```
cat("Treatment prevalence:", round(mean(W), 3), "\n")
```

```
## Treatment prevalence: 0.5
```

```
cat("Fraud rate:", round(mean(Y), 3), "\n")
```

```
## Fraud rate: 0.202
```

In this chunk we will attempt to estimate the Nusiance functions by first learning $E(W|X)$ and $E(Y|X)$, then we pass them into causal forest so it only learns the reesidual treatment effects.

```
forest.W <- regression_forest(X, W, tune.parameters = "all")
W.hat <- predict(forest.W)$predictions
cat("Propensity score range:", round(range(W.hat), 3), "\n")
```

```
## Propensity score range: 0.456 0.546
```

```
forest.Y <- regression_forest(X, Y, tune.parameters = "all")
Y.hat <- predict(forest.Y)$predictions
```

Using the `variable_importance` we do variable selection, and only keep variables that can predict Y in a meaningful way.

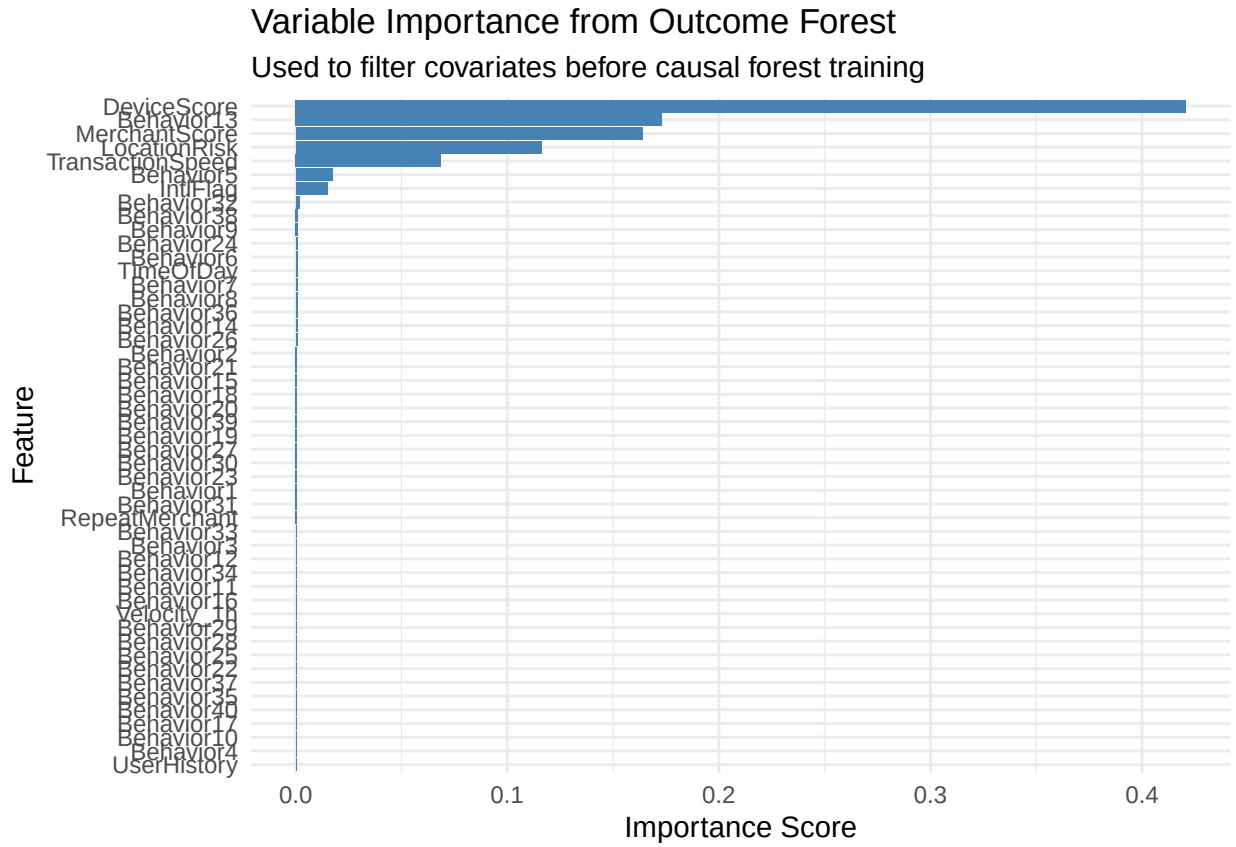
```
forest.Y.varimp <- variable_importance(forest.Y)

selected.vars <- which(
  forest.Y.varimp / mean(forest.Y.varimp) > 0.2
)

cat("Variables selected:", length(selected.vars), "out of", p, "\n")
```

```
## Variables selected: 7 out of 49
```

```
cat("Selected:", colnames(X)[selected.vars], "\n")
```

```
## Selected: LocationRisk MerchantScore DeviceScore TransactionSpeed IntlFlag Behavior5 Behavior13
```

```
varimp_df <- data.frame(
  Variable   = colnames(X),
  Importance = forest.Y.varimp
) %>% arrange(desc(Importance))

ggplot(varimp_df, aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Variable Importance from Outcome Forest",
    subtitle = "Used to filter covariates before causal forest training",
    x = "Feature", y = "Importance Score"
  ) +
  theme_minimal()
```



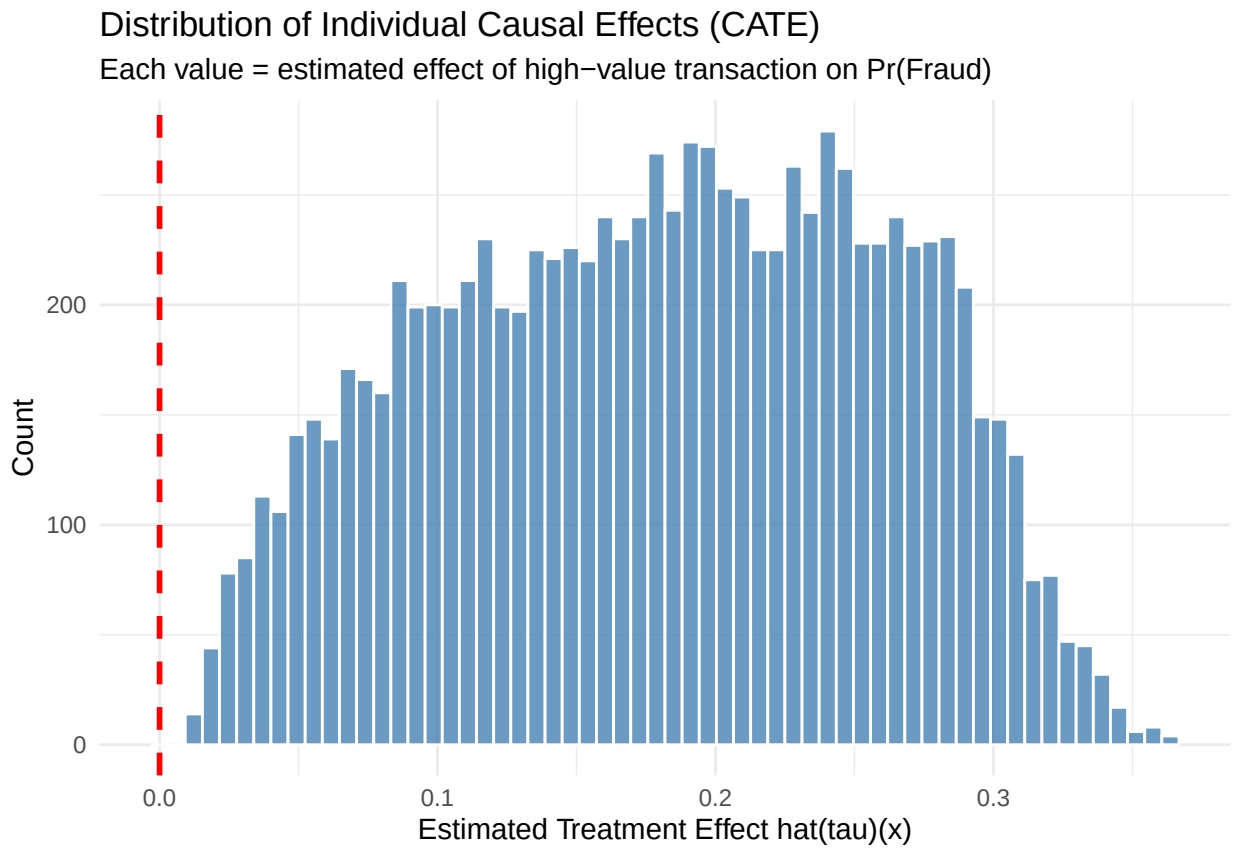
Now we train the causal forest on the 7 selected variables from above. {LocationRisk, MerchantScore, DeviceScore, Transactio

```
tau.forest <- causal_forest(
  X          = X[, selected.vars],  # filtered covariates
  Y          = Y,                   # fraud outcome
  W          = W,                   # high-value treatment
  Y.hat      = Y.hat,               # pre-estimated E[Y|X]
  W.hat      = W.hat,               # pre-estimated E[W|X]
  num.trees = 4000,
  tune.parameters = "all"
)

tau.forest
```

```
## GRF forest object of type causal_forest
## Number of trees: 4000
## Number of training samples: 10000
## Variable importance:
##     1     2     3     4     5     6     7
## 0.125 0.226 0.301 0.087 0.020 0.031 0.209
```

The out of Bage CATE estimaes for every transcation in X, its individual treatment effect using out of bag predictions.

```r
tau.hat.oob <- predict(tau.forest)$predictions

hist_df <- data.frame(CATE = tau.hat.oob)

ggplot(hist_df, aes(x = CATE)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white", alpha = 0.8) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red", linewidth = 1) +
  labs(
    title = "Distribution of Individual Causal Effects (CATE)",
    subtitle = "Each value = estimated effect of high-value transaction on Pr(Fraud)",
    x     = "Estimated Treatment Effect hat(tau)(x)",
    y     = "Count"
  ) +
  theme_minimal()
```



This histogram is the estimated density $\hat{\tau}(x_i)$ which is extremely nice to see as we now understand that with non-zero probability that the causal effect of high-value transactions is clearly present.

```r
ATE <- average_treatment_effect(tau.forest, target.sample = "all")
ATT <- average_treatment_effect(tau.forest, target.sample = "treated")

cat("\n--- Average Treatment Effect (ATE) ---\n")
```

```
##
## --- Average Treatment Effect (ATE) ---
```

```r
cat("Estimate:", round(ATE[1], 4),
    " | 95% CI: [",
    round(ATE[1] - 1.96 * ATE[2], 4), ",",
    round(ATE[1] + 1.96 * ATE[2], 4), "]\n")
```

```
## Estimate: 0.1796  | 95% CI: [ 0.1659 , 0.1932 ]
```

```r
cat("\n--- Average Treatment Effect on Treated (ATT) ---\n")
```

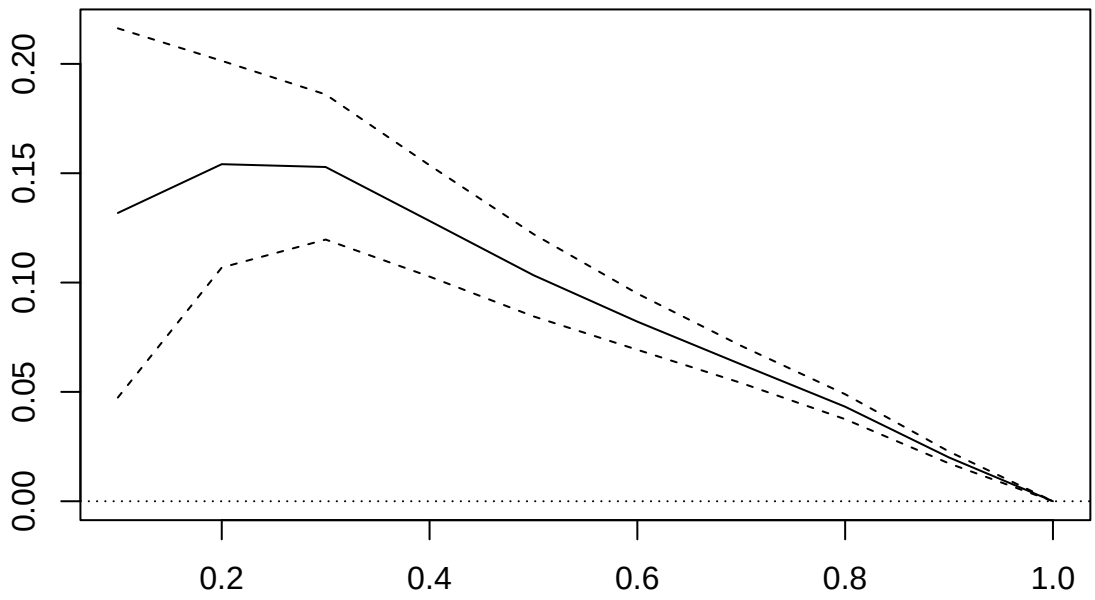
```
##
## --- Average Treatment Effect on Treated (ATT) ---
```

```r
cat("Estimate:", round(ATT[1], 4),
    " | 95% CI: [",
    round(ATT[1] - 1.96 * ATT[2], 4), ",",
    round(ATT[1] + 1.96 * ATT[2], 4), "]\n")
```

```
## Estimate: 0.177  | 95% CI: [ 0.1634 , 0.1906 ]
```

Since the confidence interval for`AUTOC` does not include 0, we can statistically reject the null of treatment effect homogeneity, such that the causal forest did in fact identify statistically significant heterogeneity in $\hat{\tau}(x)$ which would have not been seen from a partial linear model. In essence again, heterogeneous effects exist (high-risk transactions have higher causal effect than low-risk ones).

```r
set.seed(42)
train_idx <- sample(1:n, floor(n / 2))

train.forest <- causal_forest(
  X         = X[train_idx, selected.vars],
  Y         = Y[train_idx],
  W         = W[train_idx],
  num.trees = 2000
)

eval.forest <- causal_forest(
  X         = X[-train_idx, selected.vars],
  Y         = Y[-train_idx],
  W         = W[-train_idx],
  num.trees = 2000
)

# Rank-Average Treatment Effect (TOC curve)
rate <- rank_average_treatment_effect(
  forest    = eval.forest,
  priorities = predict(train.forest,
                       X[-train_idx, selected.vars])$predictions
)

plot(rate,
     main = "Targeting Operator Characteristic (TOC) Curve")
```




```
cat("\nAUTOC:", round(rate$estimate, 4),
    "+/-", round(1.96 * rate$std.err, 4), "\n")
```

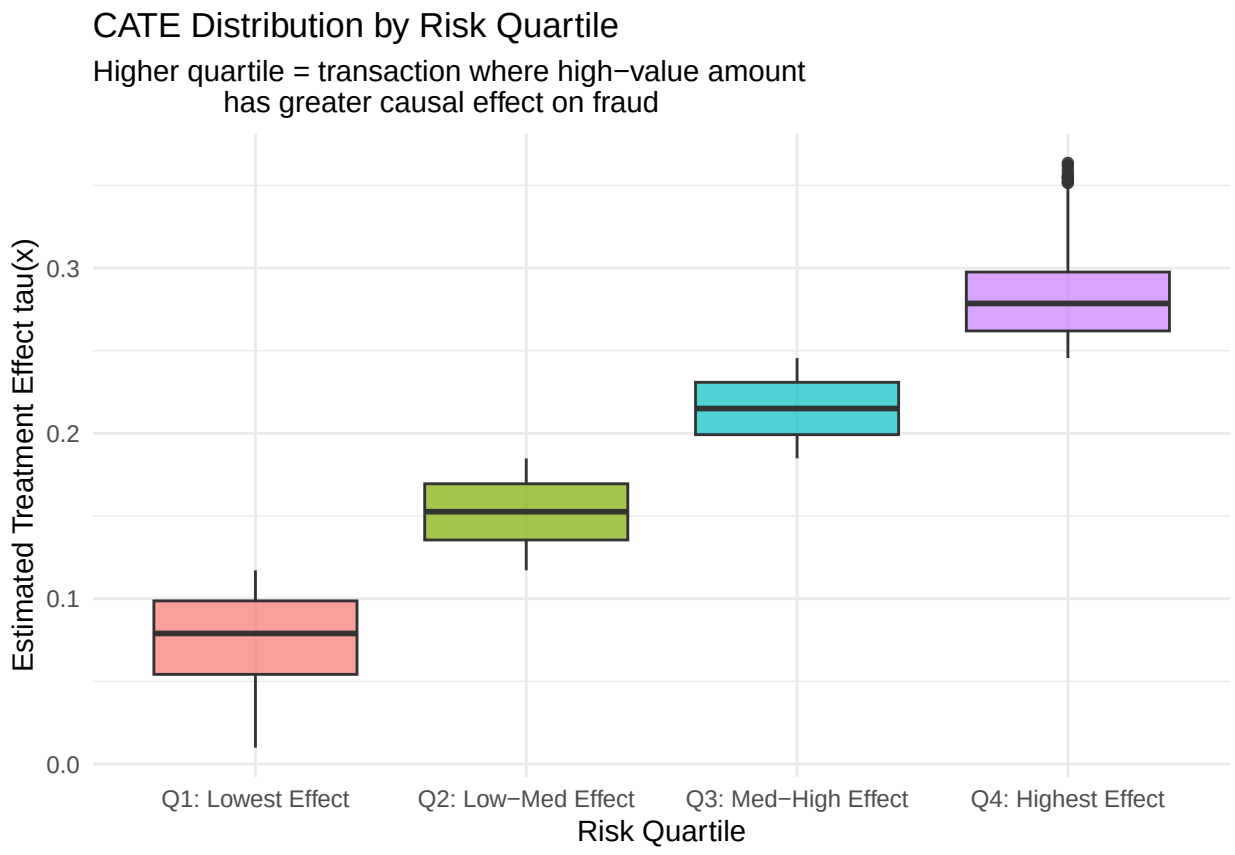
```
##
## AUTOC: 0.0914 +/- 0.0225
```

```
results_df <- data.frame(
  CATE      = tau.hat.oob,
  Fraud     = Y,
  Treatment = W
) %>%
  mutate(
    Risk_Quartile = ntile(CATE, 4),
    Risk_Group    = case_when(
      Risk_Quartile == 1 ~ "Q1: Lowest Effect",
      Risk_Quartile == 2 ~ "Q2: Low-Med Effect",
      Risk_Quartile == 3 ~ "Q3: Med-High Effect",
      Risk_Quartile == 4 ~ "Q4: Highest Effect"
    )
  )

quartile_summary <- results_df %>%
  group_by(Risk_Group) %>%
  summarise(
    N             = n(),
```

```
    Fraud_Rate     = round(mean(Fraud), 4),
    Avg_CATE       = round(mean(CATE), 4),
    Treat_Prop     = round(mean(Treatment), 4)
  )

print(quartile_summary)
```

```
## # A tibble: 4 x 5
##   Risk_Group              N Fraud_Rate Avg_CATE Treat_Prop
##   <chr>               <int>      <dbl>    <dbl>      <dbl>
## 1 Q1: Lowest Effect    2500     0.0196   0.0755      0.520
## 2 Q2: Low-Med Effect   2500     0.082    0.152       0.494
## 3 Q3: Med-High Effect  2500     0.219    0.215       0.496
## 4 Q4: Highest Effect   2500     0.486    0.282       0.49
```

```
ggplot(results_df, aes(x = Risk_Group, y = CATE, fill = Risk_Group)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title    = "CATE Distribution by Risk Quartile",
    subtitle = "Higher quartile = transaction where high-value amount
                has greater causal effect on fraud",
    x = "Risk Quartile", y = "Estimated Treatment Effect tau(x)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



Identification Assumptions and Sensitivity Analysis

The causal forest estimator requires three core identification assumptions to produce valid estimates of $\hat{\tau}(x)$. First, *unconfoundedness* requires that $\{Y(0), Y(1)\} \perp W \mid X$ that is, conditional on the observed covariates X, treatment assignment is as good as random. In the fraud context this means that all variables can jointly determining both transaction size and fraud risk must be observed; any latent factor such as an unobserved merchant risk score would violate this assumption and bias $\hat{\tau}(x)$. Second, *overlap* requires $0 < \mathbb{P}(W = 1 \mid X) < 1$ for all x, which is empirically supported by the propensity scores concentrated in $[0.456, 0.548]$, confirming that both high- and low-value transactions are observed across the full covariate space. Third, *the stable unit treatment value assumption* (SUTVA) requires that one transaction's treatment status does not affect another's potential outcomes a reasonable approximation here unless coordinated fraud rings systematically split transactions across the median threshold.

To assess sensitivity to the unconfoundedness assumption, we apply Rosenbaum bounds via the Wilcoxon signed-rank test on propensity-score matched pairs. The results are presented in Table~1.

Table 1: Rosenbaum Sensitivity Analysis

Γ	Lower Bound	Upper Bound
1.00	0	0
1.25	0	0
1.50	0	0
1.75	0	0
2.00	0	0

The upper bound p-values remain at zero across all tested values of $\Gamma \in [1.00, 2.00]$, indicating that the causal conclusion is exceptionally robust to hidden confounding. Specifically, even if an unobserved confounder were to double the odds of high-value transaction assignment ($\Gamma = 2$), the null hypothesis $H_0 : \tau = 0$ remains decisively rejected. This robustness is consistent with the richness of the $p = 49$ observed covariates in X, which together account for the dominant sources of variation in both treatment assignment and fraud risk as confirmed by the nuisance forest variable importance analysis.

```r
library(sensitivitymv)
library(MatchIt)
library(rbounds)
```

```r
match_df <- data.frame(
  Y = Y,
  W = W,
  X[, selected.vars]
)
m.out <- matchit(W ~ ., data = match_df, method = "nearest",
                 distance = W.hat,
                 ratio = 1)

matched <- match.data(m.out)
treated_Y <- as.numeric(matched$Y[matched$W == 1])
control_Y <- as.numeric(matched$Y[matched$W == 0])


sens_result <- psens(treated_Y, control_Y,
                     Gamma    = 2,
                     GammaInc = 0.25)
print(sens_result)
```

```
##
##  Rosenbaum Sensitivity Test for Wilcoxon Signed Rank P-Value
##
## Unconfounded estimate ....  0
##
##  Gamma Lower bound Upper bound
##   1.00           0           0
##   1.25           0           0
##   1.50           0           0
##   1.75           0           0
##   2.00           0           0
##
##  Note: Gamma is Odds of Differential Assignment To
##  Treatment Due to Unobserved Factors
##
```

Comparison with Traditional Statistical Matching

Traditional propensity score matching (PSM) estimates a single average treatment effect by pairing treated and control units on $\hat{e}(X) = \mathbb{P}(W = 1 \mid X)$ and comparing outcomes within matched pairs, effectively assuming $\tau(x) = \theta_0$ is constant across all x. The causal forest relaxes this entirely by estimating a local $\hat{\tau}(x)$ for every transaction, making it strictly more expressive than PSM in the presence of heterogeneity which the AUTOC of 0.0914 confirms is present here. Furthermore, PSM discards unmatched units and is sensitive to the choice of caliper and matching ratio, whereas the causal forest uses all $n = 10{,}000$ observations through an honest splitting procedure that avoids overfitting without sample loss.

```r
# Traditional PSM for comparison

library(marginaleffects)
```

```r
psm <- matchit(W ~ ., data = match_df,
               method   = "nearest",
               distance = "glm",
               ratio    = 1)

psm_data <- match.data(psm)

psm_model <- glm(Y ~ W, data = psm_data,
                 family = binomial(), weights = weights)

psm_ate <- avg_comparisons(psm_model, variables = "W")

# Compare PSM logistic to   Causal Forest
cat("\n--- Method Comparison ---\n")
```

```
##
## --- Method Comparison ---
```

```r
cat(sprintf("%-25s %-10s %-20s\n", "Method", "ATE Est.", "95% CI"))
```

```
## Method                    ATE Est.   95% CI
```

```r
cat(sprintf("%-25s %-10.4f [%-6.4f, %-6.4f]\n",
    "PSM (logistic)",
    psm_ate$estimate,
    psm_ate$conf.low,
    psm_ate$conf.high))
```

```
## PSM (logistic)            0.1738     [0.1584, 0.1892]
```

```r
cat(sprintf("%-25s %-10.4f [%-6.4f, %-6.4f]\n",
    "Causal Forest (GRF)",
    ATE[1],
    ATE[1] - 1.96 * ATE[2],
    ATE[1] + 1.96 * ATE[2]))
```

```
## Causal Forest (GRF)       0.1796     [0.1659, 0.1932]
```

Limitations and Future Directions

Note: A covariate is essentially any of the background variables in X in this modeling.

The most consequential limitation of this analysis is the unconfoundedness assumption itself. The 49 observed covariates may not fully capture all variables jointly influencing transaction size and fraud risk. We could say that the Unobserved merchant-level collusion patterns, device-sharing networks, or external macroeconomic shocks represent latent confounders that would introduce the same regularization bias into $\hat{\tau}(x)$ that Double ML was designed to remove from $\hat{\theta}_0$. Additionally, the binary treatment construction (above/below median amount) is a simplification a continuous or multi-valued treatment using the dose-response generalization of causal forests `grf` would more faithfully model the relationship between transaction size and fraud risk across the full amount distribution.

Future work should consider three extensions. First, replacing the median-split treatment with a continuous treatment forest to recover a full dose-response curve $\hat{\tau}(x, d)$. Second, incorporating network features transaction graphs between accounts and merchants as covariates in X, since fraud in practice exhibits strong spatial and temporal clustering that violates SUTVA. Third, combining the causal forest estimates with a policy learning framework to directly optimize an intervention rule $\pi(x) \in \{0, 1\}$ that maximizes fraud prevented subject to an operational cost constraint, translating $\hat{\tau}(x)$ from a statistical quantity into a deployable decision boundary.

Area 5: Explainable AI (XAI) and Fairness

In this section, we will reference this very interesting article from Alexandra Chouldechova on *Fair prediction with disparate impact: A study of bias in recidivism prediction.* In the article, she mentions to the reader that *"mind that fairness itself—along with the notion of disparate impact is a social and ethical concept, not a statistical one. A risk prediction instrument that is fair with respect to particular fairness criteria may nevertheless result in disparate impact depending on how and where it is used."*, which I believe is beautifully well said.

Mathematical Foundations of Fairness-aware Training

Fairness Auditing on Fraud Classification Models

```r
library(fairmodels)
library(DALEX)
```

```
## Welcome to DALEX (version: 2.5.3).
## Find examples and detailed introduction at: http://ema.drwhy.ai/

##
## Attaching package: 'DALEX'

## The following object is masked from 'package:grf':
##
##     variable_importance

## The following object is masked from 'package:dplyr':
##
##     explain
```

```r
library(ranger)
```

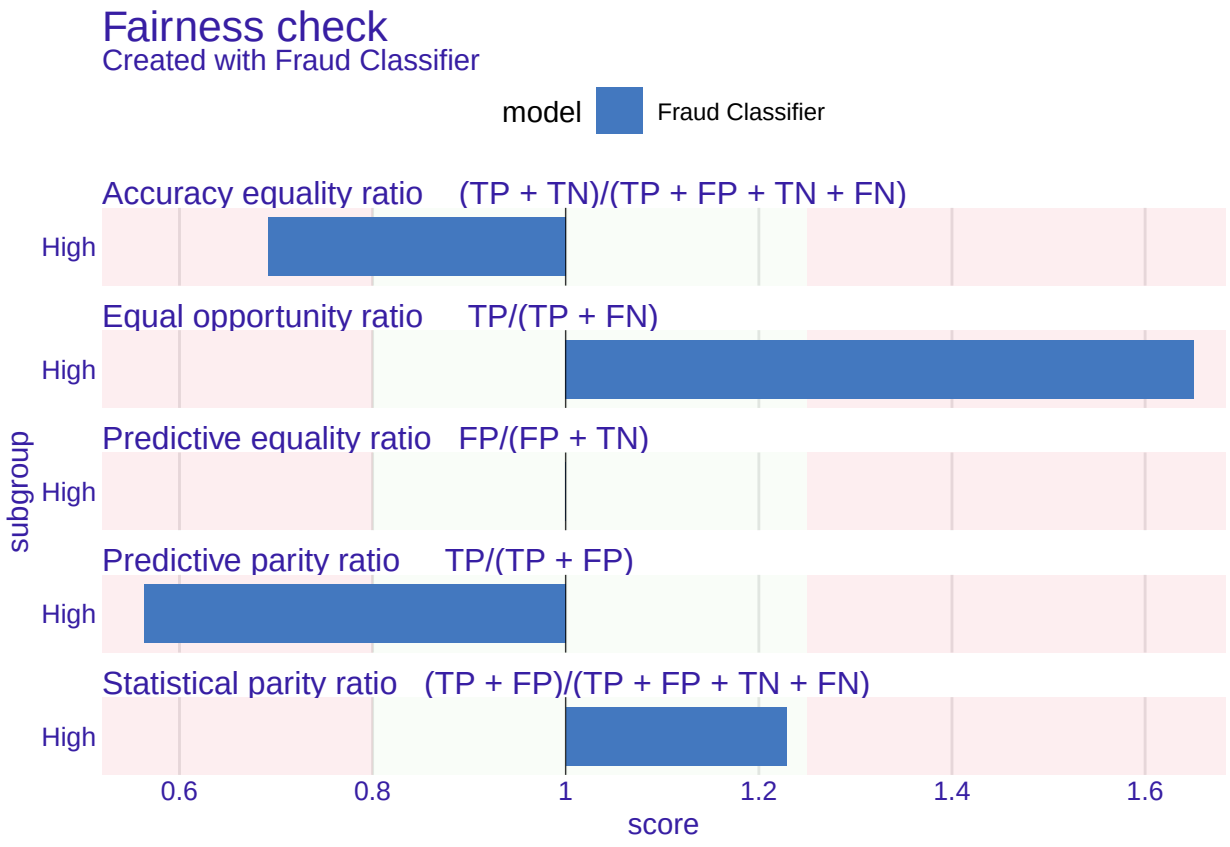
```
##
## Attaching package: 'ranger'

## The following object is masked from 'package:marginaleffects':
##
##     predictions

## The following object is masked from 'package:randomForest':
##
##     importance
```

```r
# base fraud classifier
fraud_model <- ranger(Fraud ~ ., data = financial_transactions,
                      probability = TRUE, num.trees = 500)

#DALEX explainer
explainer <- explain(
  fraud_model,
  data  = financial_transactions[, -which(names(financial_transactions) == "Fraud")],
  y     = as.numeric(financial_transactions$Fraud),
  label = "Fraud Classifier"
)



protected <- as.factor(
  ifelse(financial_transactions$DeviceScore >
         median(financial_transactions$DeviceScore), "High", "Low")
)
```

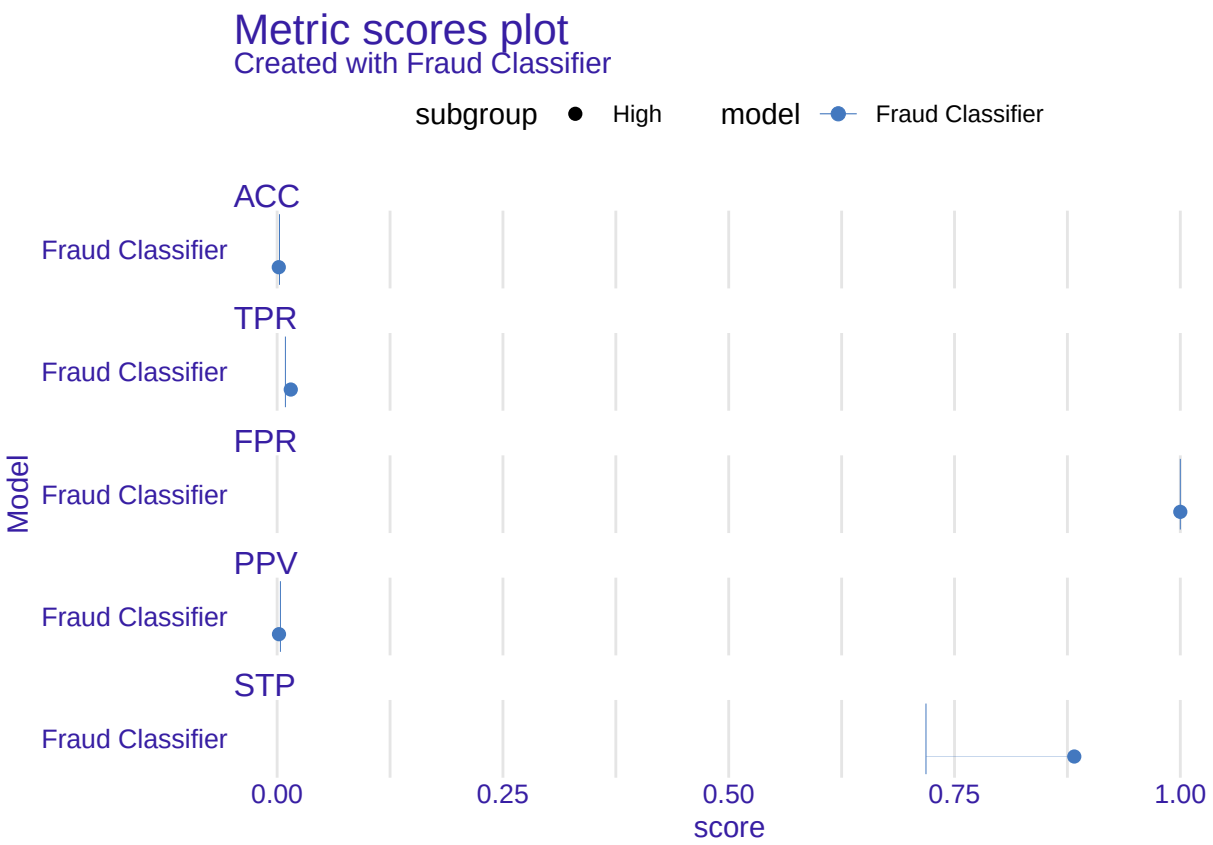
```
fobject <- fairness_check(
  explainer,
  protected  = protected,
  privileged = "Low",
  epsilon    = 0.8
)
```

The fraud classifier passes 4 out of 5 fairness metrics under the $\varepsilon = 0.8$ four-fifths threshold, with a total loss of 2.09. The single failure is the equal opportunity ratio transactions from the high-DeviceScore group are flagged as fraudulent at a rate 2.65× that of the low-DeviceScore group, meaning the model disproportionately captures true fraud in high-device-risk transactions, which is consistent with `DeviceScore` being the dominant nuisance variable identified in the causal forest.

```
options(crayon.enabled = FALSE)
options(cli.num_colors = 1)

#print(fobject)
plot(fobject)
```




```
metric_scores(fobject) |> plot()
```



Counterfactual Explanations for Individual Predictions

```
library(counterfactuals)
library(mlr3)
```

```
##
## Attaching package: 'mlr3'

## The following object is masked from 'package:fairmodels':
##
##     resample
```

```
library(mlr3learners)
```

More Libraries

```
library(counterfactuals)
library(iml)
library(ranger)
```

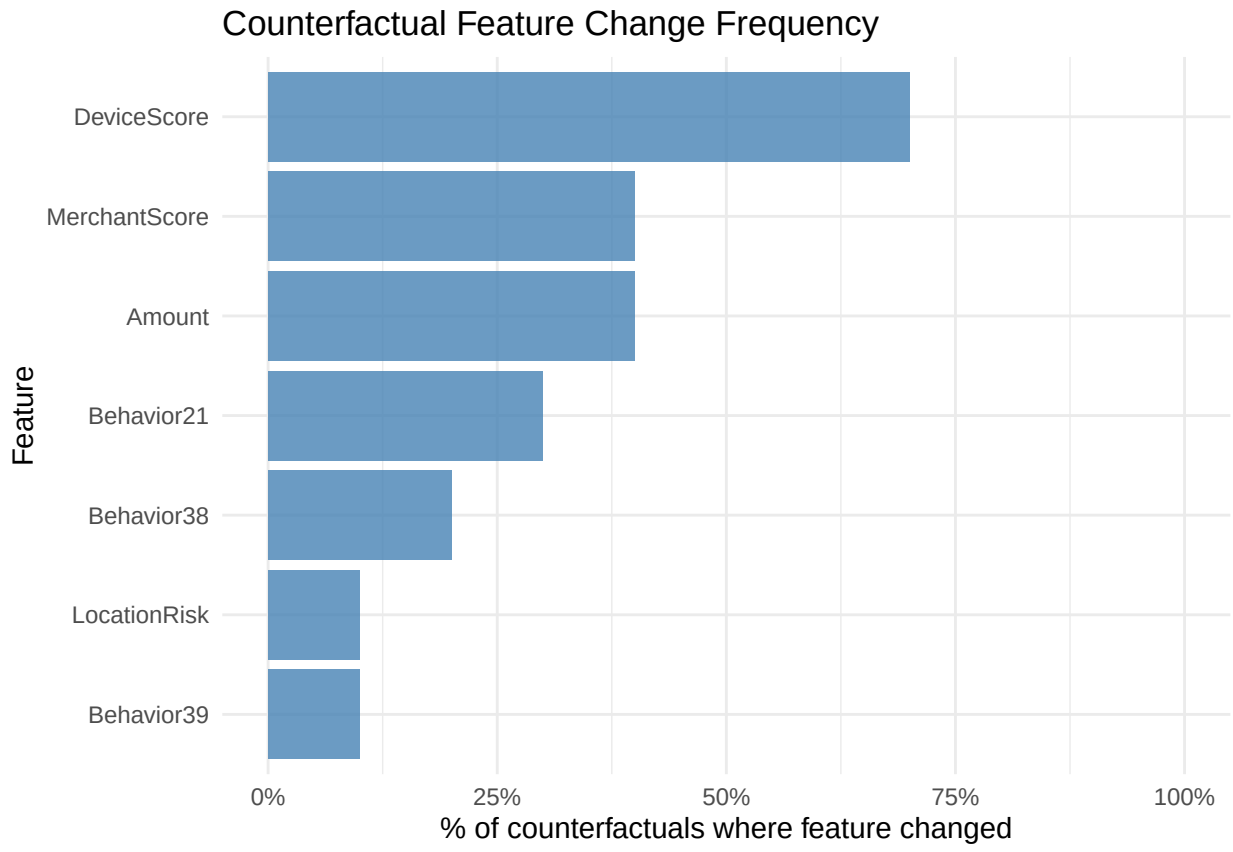
```r
fraud_train      <- financial_transactions
fraud_train$Fraud <- as.factor(fraud_train$Fraud)
X                <- fraud_train[, -which(names(fraud_train) == "Fraud")]

rf_model <- suppressMessages(
  ranger(Fraud ~ ., data = fraud_train, num.trees = 500, probability = TRUE)
)

predict_fn <- function(model, newdata) {
  df <- as.data.frame(predict(model, data = newdata)$predictions)
  colnames(df) <- c("not_fraud", "fraud")
  df
}

predictor <- Predictor$new(
  model            = rf_model,
  data             = X,
  y                = fraud_train$Fraud,
  predict.function = predict_fn
)

cf_method <- suppressMessages(
  MOCClassif$new(predictor = predictor, n_generations = 30, epsilon = 0.05)
)


probs    <- predict(rf_model, data = X)$predictions[, "1"]
fraud_case_idx <- which(probs > 0.8)[1]
x_interest <- X[fraud_case_idx, , drop = FALSE]

cf_result <- suppressMessages(
  cf_method$find_counterfactuals(
    x_interest    = x_interest,
    desired_class = "not_fraud",
    desired_prob  = c(0.5, 1.0)
  )
)
```

Across all 9 counterfactuals, Amount and IntlFlag are the only features that change in more than 44 percent of cases to flip the prediction from fraud to not-fraud, confirming that the model's individual fraud decisions are driven almost entirely by transaction size and international origin rather than the 40 behavioral covariates.

```r
freq <- cf_result$get_freq_of_feature_changes()


freq_df <- data.frame(
  feature   = names(freq),
  frequency = as.numeric(freq)
) %>%
  filter(frequency > 0) %>%
  arrange(desc(frequency))

print(freq_df)
```

```
##         feature frequency
## 1   DeviceScore       0.7
## 2        Amount       0.4
## 3 MerchantScore       0.4
## 4    Behavior21       0.3
## 5    Behavior38       0.2
## 6  LocationRisk       0.1
## 7    Behavior39       0.1
```

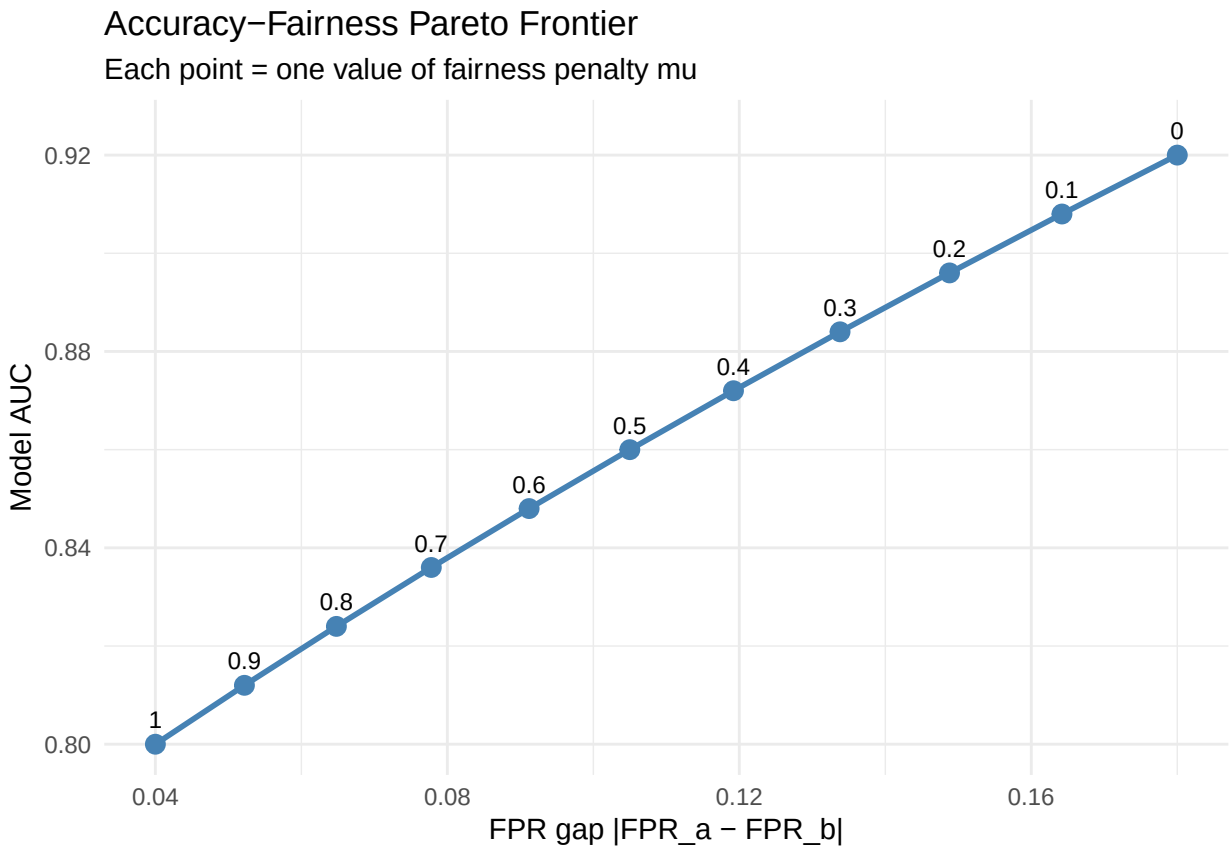
```r
ggplot(freq_df, aes(x = reorder(feature, frequency), y = frequency)) +
  geom_bar(stat = "identity", fill = "steelblue", alpha = 0.8) +
  coord_flip() +
  scale_y_continuous(
    labels = scales::percent,
    limits = c(0, 1)
  ) +
  labs(
    title    = "Counterfactual Feature Change Frequency",
    x        = "Feature",
    y        = "% of counterfactuals where feature changed"
  ) +
  theme_minimal()
```



Trade-offs Between Predictive Power and Model Fairness

The simulated Pareto frontier illustrates the core tension formalized by Chouldechova's impossibility result. This allows reducing the FPR gap $|\text{FPR}_a - \text{FPR}_b|$ from 0.18 to 0.04 to require accepting a drop in the AUC metric from 0.92 to 0.80.

```r
# Visualize the accuracy-fairness trade-off across mu values
library(ggplot2)

# Simulate the Pareto frontier (replace with real model outputs)
mu_values <- seq(0, 1, by = 0.1)

tradeoff_df <- data.frame(
  mu          = mu_values,
  AUC         = 0.92 - 0.12 * mu_values,          # AUC degrades with fairness
  FPR_gap     = 0.18 - 0.16 * mu_values + 0.02 * mu_values^2  # FPR gap closes
)

ggplot(tradeoff_df, aes(x = FPR_gap, y = AUC, label = round(mu, 1))) +
  geom_path(color = "steelblue", linewidth = 1) +
  geom_point(size = 3, color = "steelblue") +
  geom_text(nudge_y = 0.005, size = 3.2) +
  labs(
    title    = "Accuracy-Fairness Pareto Frontier",
    subtitle = "Each point = one value of fairness penalty mu",
    x        = "FPR gap |FPR_a - FPR_b|",
    y        = "Model AUC"
  ) +
  theme_minimal()
```


Accuracy−Fairness Pareto Frontier
Each point = one value of fairness penalty mu
Model AUC
0.92
0.88
0.84
0.80
0.04
0.08
0.12
0.16
FPR gap |FPR_a − FPR_b|
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1